

VECTOR

APL95 at San Antonio

Plus ...

- | | |
|---|-----|
| • Iverson Teaches J (Part 2) | 67 |
| • Sullivan on Multiprecision Arithmetic | 74 |
| • Sykes on Probabilities | 86 |
| • Cannon on Windows Printers | 114 |
| • Pearson on Windows Filing | 129 |



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.12 No.1 July 1995

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production), and Winword-2.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1995-96

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£14	1	1
(Supplement for Airmail, not needed for Europe)	£4		
UK Corporate Membership	£100	10	5
Overseas Corporate	£135	10	
Sustaining	£430	50	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year normally runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa, Mastercard or JCB, at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

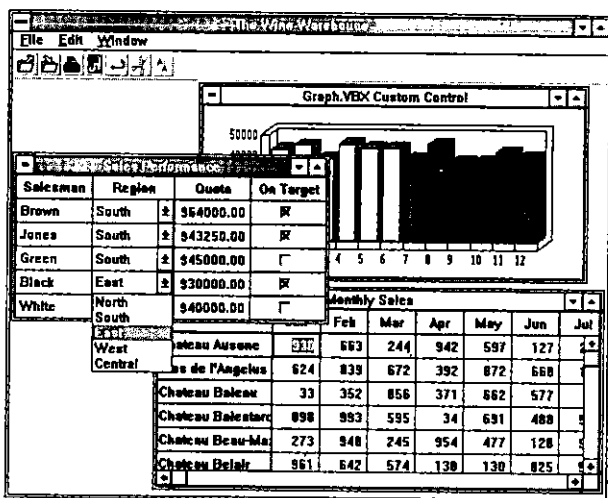
Deadlines for bookings and copy are given under the Quick Reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 01439-788385 CompuServe: 100331,644

Contents

		Page
Editorial	Anthony Camacho	3
APL NEWS		
Quick Reference Diary		5
Correspondence		7
British APL Association Annual Accounts		
News from Sustaining Members	Gill Smith	13
The Education Vector	Ian Clark	17
APL Product Guide — Updates	Gill Smith	33
Book Review: <i>Making APL Readable</i>	Anthony Camacho	38
RECENT MEETINGS: APL95 at San Antonio		
Conference Reports		41
Machine Learning Algorithms	Jon Sandles	51
Business Graphics Workshop	Adrian Smith	53
Portability Workshop	Eke van Batenburg	56
GENERAL ARTICLES		
Voyages in Dragon Country	Sylvia Camacho	60
Teaching with an Executable Notation (Part 2)	Ken Iverson	67
Multiprecision Arithmetic (Part 1)	John Sullivan	74
The Random Vector		
Probabilities for Elementary Distributions	Alan Sykes	86
A Quite Different New Primitive	Gérard Langlet	93
The Complete Elliptic Integrals and APL	Joseph De Kerf	102
The Common Mean, Non-negative		
Definite Matrices, and APL	Dietrich Trenkler	107
TECHNICAL SECTION		
Hacker's Corner: Driving the MS Daisy-wheel	Ray Cannon	114
Technical Correspondence		121
At Work and Play with J:	Gene McDonnell	125
"Correct" Windows File Behaviour	Duncan Pearson	129
Native File Functions for Dyalog APL	Adrian Smith	137
Index to Advertisers		143

dyalog APL

The Definitive APL for Windows™



Dyalog APL/W Version 7.1 includes a sophisticated built-in Grid object, with numeric, currency, and date fields as well as combo boxes and button cells. The Grid also provides an undo feature, cut and paste facilities (which let you move data quickly and easily between APL and your favourite spreadsheet), resizable rows, columns and titles, and drag/drop editing. Dyalog APL/W supports Visual Basic Custom Controls, ToolBar, StatusBar and TabBar objects, automatic Hints and Tips, Metafiles, MDI, 3-D Forms and Controls, a fully customizable Session, an ODBC interface, namespaces for encapsulation, and a host of other features; all designed to make it easy to develop fast, responsive and attractive Windows applications.

That is why Dyalog APL/W remains the professional choice. For further details, contact Dyadic or your local distributor today.

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke, Hampshire, RG24 7AL, United Kingdom.
Tel: +44 1256 811125 Fax: +44 1256 811130 Email: sales@dyadic.com.



Editorial

by Anthony Camacho

APL is the most international of computer languages and notations. APL conferences are unique — no other language runs such conferences — perhaps because APL is also a notation, perhaps because it is a language for the elite.

All international APL conferences so far have been run under the ACM rules by SigAPL. The conferences are, by tradition, alternately in Europe — the even years — and (North) America — the odd years — (there has never been one in Central or South America). Australia was 'honorary Europe' in 1988. A local APL organisation usually helps to organise the conference. Although this is a considerable burden, it can also be a valuable and enjoyable experience.

At APL 95, SigAPL suggested the British APL Association co-operates to run APL 96. Already some members have volunteered. David Eastwood has agreed to be Joint Chair (with Stuart Yarus of Dallas, Texas) and Adrian Smith to be joint Program Chair (with Phil Benkard of Somers, New York).

SigAPL is a subsidiary of ACM and the British APL Association is a subsidiary of the BCS. Each organisation has to submit quite detailed plans and undertakings to secure approval for the risk funding of any proposed conference. It is consequently difficult to organise a fully equal division of the responsibility, risk and profit or loss. If one organisation makes undertakings to its parent then it cannot release control without also letting go of its power to keep its word.

In any case there are (significant) others. What about the Association Francophone pour la promotion du langage APL, the APL-Club Germany, the APL-Club Austria, BACUS (Belgium), FinnAPL, the Dutch APL Association, SwedAPL, SAUG (Switzerland)? There are also some national organisations who have joined ACM as Sigs. If we are to continue the tradition it should be Europe rather than the British APL Association co-operating with SigAPL.

During APL 95 several people suggested to me that it would be best if the international conferences were planned and run by an *independent* international organisation in which all the national organisations took a part (Australia, Japan and South Africa too). This seems to be the only truly equitable way. If SigAPL is serious then let us all set about getting a truly international organisation in place for 1997. I would be glad to hear your views.

Causeway

Rapid development of reliable and flexible Windows applications

Training in the effective use of Windows APLs and the Causeway environment

Support and maintenance of Causeway installations

Causeway Graphical Systems Ltd
5 The Maltings
Castlegate,
Malton, North Yorks
YO17 0DP

Tel: +44 (0)1653 696760
Fax: +44 (0)1653 697719

Email: 100265.1564@compuserve.com

APL Consultant

Dyadic Systems is looking for an "ace" APL programmer/consultant to work on a variety of customer and in-house projects using Dyalog APL/W for Windows. Our ideal candidate will be a young graduate with a minimum of 2 years APL experience and familiarity with MS-Windows. We can offer you a challenging and satisfying career at the leading edge of APL technology, together with an excellent performance-based remuneration package.

If you would like to join the Dyalog APL team, please apply in writing to:

Peter Donnelly
Dyadic Systems Limited
Riverside View
Basing Road
Old Basing
Basingstoke RG24 7AL
Hampshire



Quick Reference Diary 1994-95

Date	Venue	Event
July 7th	IEE, London	BAA Meeting (see below)
Late July or early August 1996	Edinburgh or Dublin	APL96

At the Committee meeting of 10 February 1995 it was decided that there was insufficient time and effort available to organize a Beginners Workshop for 25 March. Any such exercise cannot hope to succeed without considerable advance publicity as it must attract support from outside the established APL community. However, it was agreed that the Workshop held at the end of January 1995 should be repeated in October 1995. Please watch this space for details.

Please note that we have been unable to obtain the usual venue for the meeting of 19th May 1995 but speakers are being arranged for a replacement meeting on 7th July 1995. Details will be mailed as usual.

British APL Association meetings are normally held in the IEE, Savoy Place.
Nearest tube outlets: Temple or Embankment.

Dates for Future Issues of VECTOR

	Vol.12 No.2	Vol.12 No.3	Vol.12 No.4
Copy date	1st Sept 95	1st Dec 95	1st March 96
Ad booking	8th Sept 95	8th Dec 95	10th March 96
Ad Copy	15th Sept 95	15th Dec 95	20th March 96
Distribution	October 95	January 96	April 96

APL96

WESTERN EUROPE

Who is Organising it

APL96 will be a sponsored by SigAPL, and managed by C/M Planners, who did such a solid job for us at San Antonio. The Co-chairs are Stuart Yarus and David Eastwood, and the Programme Committee is led jointly by Philip Benkard and Adrian Smith. Gitte Christiansen is working with the vendors, and Anne Faust is working with Lynne Shaw on marketing.

Where and When will it be?

The search for a venue is on! At the time of going to press, the favoured site is the British Isles, but Lisbon and Padua have also been earmarked as possibilities. By the time Vector is shipped, we should know the answer to this one, so look out for an insert! The date will be towards the end of July or early in August.

What Kind of Conference are we Planning?

More of what worked well in Texas! Basically fewer streams, more workshops and as much 'hands-on' as we can organise/fund. The hot topics look like "Nested Arrays — get it! right first time"; an algorithms workshop following up the excellent sessions at San Antonio (Andreas Geyer-Schulz and Ben Best); how to get that "Designed for Windows 95" logo on YOUR software; setting up APL as a server application (TCP/IP for softies) anywhere on your corporate network.

We certainly intend to make the vendors work a lot harder! Rather than shutting them away in an exhibition and making them buy lunch, they can have a decent 3-hour slot to really teach us how to use their stuff!

The idea of evening sessions was floated at the San Antonio banquet, and (on a quick show of hands) many were in favour and almost none against. We will aim for a pattern of mostly plenary mornings, breaking out into parallel workshops alongside contributed papers in the afternoons. Because the workshops are likely to be over-subscribed, we will repeat them in the early evening, and if we have enough good papers, these too may run on after tea.

CORRESPONDENCE

Too Much Diversity in APL?

From: Joseph De Kerf

10 April 1995

Maybe the two most important events on the APL front in 1994 were the implementation of control structures in Manugistics' APL*PLUS and the implementation of namespaces in Dyadic's Dyalog APL. Personally I wasn't waiting for control structures: I am feeling myself very comfortable with the goto of APL — the unconditional and conditional branch (→). On the other hand, their implementation neutralizes the objection of some opponents of APL who claim they reject APL because it doesn't support such control structures. A very important innovation, however, is the introduction of namespaces. Eric Lescasse for instance described it as the first step to Object Oriented APL. As such, it was a very good idea of *Vector* to devote a special feature section on the concept, containing three papers respectively by Peter Donnelly, Eric Lescasse, and Kimmo Kekäläinen (*Vector* Vol 11 No 3, pp 65-95). With increasing astonishment however, we read the guest editorial on the concept by Duncan Pearson in the same number of *Vector* (p 3). He states: "Whatever Manugistics produce I hope that it is different from the Dyalog implementation." And he goes on to say: "So let us have as many different, independently conceived, approaches to this problem as there are interpreters." Maybe this is true for experimental implementations, but definitely not for official commercial releases: we are killing APL in that way.

Indeed, APL is a living language, but a strong handicap for its future is the diversity of its implementations. When in 1973 shared variables were introduced, only a few implementations took it over. Other implementations for instance added the possibility to treat component and native files by implementing an appropriate set of system variables and system functions. In 1991, based on a study of about eighty commercial implementations, I published two inventories, containing a list of about eight hundred system variables/functions in use at that time. A lot of them are synonyms, although more or less different in their implementation details, and a lot of them are homonyms providing totally different facilities. And the proliferation still continues. In the meantime, generalized arrays or arrays of arrays were introduced, in some implementations based on the grounded array concept, in most implementations based on the floating array concept. But even those who used the floating array concept did it

in different ways. There are about ten implementations which support the so-called Extended APL for the moment, but when a software engineer wants to design portable software, he can only use the defined facilities of the APL Standard ISO 8485. He can't even use arrays of arrays. All this not only confuses the declared user of APL, but also doesn't inspire confidence on behalf of the potential newcomer, and is one of the most serious obstacles for the future of APL. So, let us try to ameliorate the situation rather than making things worse. But I am afraid we are preaching in the desert.

Joseph De Kerf
Rooienberg 72
B-2570 Duffel
Belgium

Vector Back Numbers

Back numbers of Vector are available from:

British APL Association,
c/o Gill Smith,
Brook House, Gilling East,
YORK YO6 4JJ

Price in UK: £10 per complete volume (4 issues);
£12 (overseas); £16 (airmail) including postage.

Minutes of Annual General Meeting

Friday 19 May 1995 2:00pm

BCS 7 Mansfield Mews W1M 9FJ

reported by Sylvia Camacho

Present: Dr Alan Mayer, Nicholas Small, Anthony Camacho, Dr Ian Clark, David Eastwood, Jon Sandles, Duncan Pearson, Sylvia Camacho.

Apologies were received from Jonathan Barman.

George MacLeod arrived after the AGM.

No ordinary members of the Association accepted the invitation to attend the AGM.

Minutes of AGM 27 May 1994 were agreed

The Chairman, Dr Alan Mayer, thanked all members of the Committee for last year's work. Resignations were accepted with regret from Jonathan Barman and Duncan Pearson, Alan expressing the hope that they would feel able to rejoin the Committee next year when their pressure of work may lessen. (It was learned later from George MacLeod that he also would be unable to serve on the Committee this year.) In his review of the year the Chairman said that he felt the Swansea Conference workshop and the single day workshop had been well received although these experiments had shown where the organisation of such events could be improved. He regretted that other London meetings had had to be cancelled. This arose because no one could be found with sufficient spare time to make the necessary arrangements. He felt that the Association must look for some other way to find volunteers able to do small amounts of work, not necessarily requiring attendance at Committee meetings. In particular he would like to discuss the possibility of exploiting electronic information exchange and conferencing. The Chairman commended the Vector Working Group for the continuing excellence of the journal which is undoubtedly valued by the membership, of which more than half are overseas members.

Anthony Camacho, the editor, said he felt there was no need to make any special report. The Readership Survey had not called for any striking changes. The Vector working group usually finds that there is at least enough material in hand

to fill half of the next issue when they meet at the beginning of each Vector quarter.

Dr Ian Clark, editor of Education Vector, pointed out that this, although included within the main journal, has also an independent mailing list which represents a potential growth area for Association membership among non-career programmers. Ian also has the use of a Macintosh based mailing list which he would like to put at the service of the Association.

Jon Sandles, recruitment officer, said that he feels the World Wide Web would be a good recruitment medium and would like to see a sub-group of the Vector working group set up for this purpose.

Nicholas Small, the Treasurer, presented the accounts. He expects these to be audited in time to be published in Vector 12.1. Nicholas Small said that he intends to approach John Sullivan and expects to obtain his agreement to act as Auditor for 1995/96.

The only nominations received for the election of officers were those from within the 1994/95 Committee, of which seven members offered to stand again for the same posts. There thus remain three vacant posts: Technical, Activities and Projects Officers.

Since the AGM Dave Ziemann and Ray Cannon have agreed to advise on, install and maintain the Association's entry on WWW and they have been co-opted onto the Committee.

British APL Association

Summary of Annual Accounts 1994/5

Summary of income and expenditure/receipts and payments:

	1994/95 (R&P) £	1994/95 (I&E) £	1993/94 (I&E) £
Income/Receipts			
Subscriptions	14350	12425	14344
BCS services	344	344	0
Bank interest	2781	2650	2276
Vector advertising (Incl. VAT)	6229	6257	5795
Conferences & seminars	2677	2522	2618
Other	2390	1890	219
Total receipts	28770	26088	25251
Expenditure/Payments			
Meetings	1039	1039	2270
Administration	2826	2486	2731
BCS services	344	344	396
Vector production and despatch	17910	15650	14627
Education Vector supplement	3994	3516	3429
Outstanding Achievement Award	500	500	500
Projects	2336	2078	3546
Conferences and seminars	2758	4229	3096
Other	1610	1680	3934
Total payments	33318	31522	34528
Assets summary:			
Bank and other balances	58938	63485	
Debtors	3063	6543	
Creditors	(4640)	(6117)	
Written off	(1117)		
Net assets	57360	63911	

Notes to the Accounts

Pence figures have been omitted, so columns may not add exactly.

The value of stocks of Vector and the Software Library have not been assessed, nor has the value of the Association's computing hardware and software.

For 1994/95, figures are shown both as income and expenditure, i.e. revenues strictly relating to the activities of that year, and as receipts and payments, i.e. what goes in and out of our bank account. The comparative figures for 1992/93 relate to income and expenditure.

The bad debts written off relate to Vector advertising. Details are:

TMT-Team	£250
Cocking & Drury	£867 (net of VAT relief).

I would take this opportunity to publicly thank John Sullivan for auditing the accounts.

Nicholas Small,
Hon. Treasurer.

News from Sustaining Members

Compiled by Gill Smith

Insight Systems

On April 1st, Insight Systems merged with Adaytum Software Ltd. To quote our Press Release:

Insight Systems ApS, based in Hellebæk, Denmark, announced today a merger with Adaytum Software, one of the UK's leading providers of Budgeting and Management Reporting software solutions. The merger, which will be in the form of a share exchange, will be effective 1st April 1995.

Adaytum will base its technical development team in Hellebæk, Denmark. Headed by Morten Kromberg, current Technical Director of Insight, Adaytum's aim will be to attract and build a team of the most talented APL developers available internationally. Their focus over the next two years will be to provide state of the art Client/Server based financial data analysis software solutions. These will complement Adaytum's flagship product 'The KPS System', a multi-dimensional Business Management system, used principally for complex Budgeting, Forecasting and Management Reporting requirements.

The implementation of the merger has kept us quite busy for the last quarter. In addition to the new tasks resulting from the merger, we have seen substantial growth in our consulting business, and as a result we added five people to our headcount during the same period (including three new APL developers). Expanding our offices and the APL95 conference has left us little time for product development, so there is little to announce this time — but now that the situation is stabilizing we will be back in greater strength than ever before.

Our first goal is the completion of the ODBC-compatible version of the SQAPL Server, which will make it possible to "wrap" APL applications in the guise of an SQL database (making APL effectively indistinguishable from for example Oracle, from the point of view of reporting tools). More about this and other new product releases in three months!

Dyadic Systems Limited

Fresh back from APL95 in San Antonio, Dyadic has begun work in earnest on the next release of Dyalog APL/W. Although plans have not yet been finalised, the following has been decided and was announced at APL95.

Although the current release of Dyalog APL/W (Version 7.1) runs extremely well under Windows 95 preview, a new version of the product (codename Version 7.95) is being developed specifically for Windows 95 and Windows NT. For the technically minded, the new version will be based on Microsoft WIN32 in place of the equivalent Watcom software which Dyadic has used up to now. This software will provide some benefits in terms of improved memory management under NT, but more importantly, is a qualification requirement for the new Windows 95 logo. Of greater interest to most users is that Version 7.95 will include support for all of the new GUI objects and features provided by Windows 95. These include *DragList*, *ListView*, *ProgressBar*, *RichEdit*, *TrackBar*, *TreeView* and *Spinner* objects.

The *DragList* object is a list box that allows the user to re-order its items. The *ListView* displays items as a set of icons, in a list format or a report format (like File Manager). The *ProgressBar* is a standard output control that indicates the progress of a time-consuming operation. The *RichEdit* control is a multi-line text editor that provides extensive word-processing facilities including mixed alignment, fonts and colours. This control will allow Dyalog APL/W applications to exchange complete documents with Microsoft Word and other word-processors. The *TrackBar* control provides a variety of different sliders for analog user input. The *TreeView* control may be used to display a hierarchy, such as a function calling tree, a namespace tree, or a directory structure.

The new version of Dyalog APL/W will also support OLE2 and .OCX custom controls.

Dyadic intends to be able to provide an early release of the new product on the day that Windows 95 is released by Microsoft. This will allow Dyalog APL/W users to immediately begin to generate Windows 95 compliant applications and to qualify for the new Windows95 logo themselves.

Causeway Graphical Systems Ltd

I suppose we went to APL95 to get our name known, to get our faces known, and to start convincing some hundreds of sceptical APL developers that we really have hit upon a better way of managing the "concept — prototype — negotiate — build" cycle that characterises effective APL development. To this end, we put our name and Email on the conference pens, we talked at length to the development teams (Dyadic, Soliton, Manugistics, IBM, and Strand), and we threw a party!

Following Dyadic's example (Manchester, 1986) we ordered more beer than we thought we could possibly need (wrong!) and invited anyone interested to our suite for a Causeway BeerBOF. They left having seen "Distributed Architect" — the tool that SAP are now shipping to selected customers for configuration modelling; they were given a first viewing of the "Rain" business graphics software running under APL*PLUS III, and they had ample opportunity to grill both Adrian and Duncan on the future of Causeway for Dyalog, APL*PLUS, APL2 for OS/2, and J.

The key message that we tried to leave people with is that Causeway pays exponentially increasing dividends as the application gets complex! It takes a very pragmatic APLers' approach to OOPS, and as a result leaves the way open for rapid evolution while enabling small teams to work concurrently on many components of the same project with a minimum of external co-ordination.

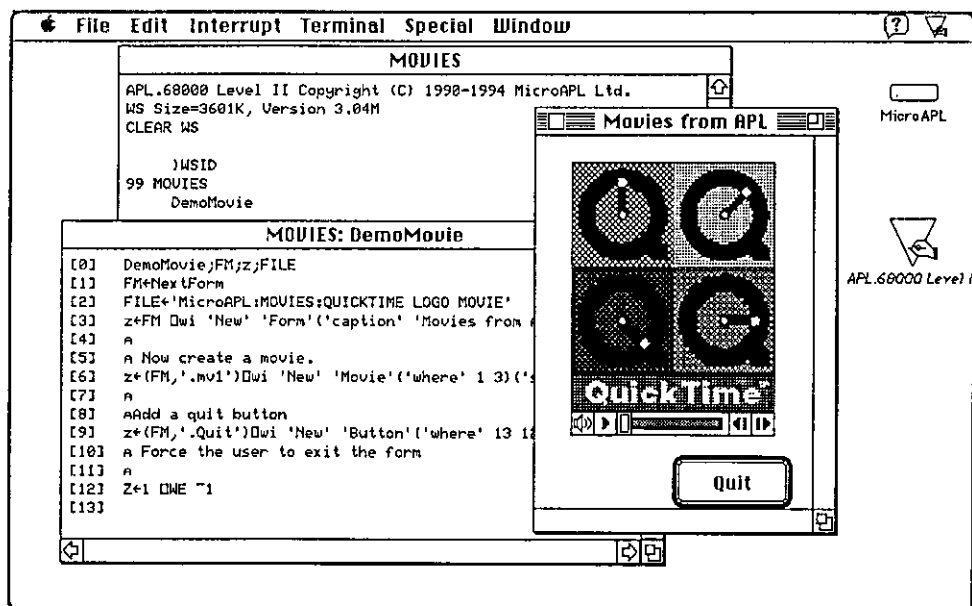
In the longer term, we made a good beginning convincing the major APL vendors to start designing into their interpreters all the right hooks to make Causeway run fast. Dyadic are already well on the way with namespaces, but IBM are developing some really interesting ideas using shared variables, and the latest release of J (from Strand Software) at last has the facilities we need.

We should re-iterate that the Causeway utilities are a by-product — we needed something to help us deliver high-quality decision-support tools that we knew we could support and enhance into the future, regardless of APL platform. The architecture and the Dyalog and Manugistics workspaces will remain in the public domain.

P.S. Registered users have already received workarounds for two interesting new Dyalog bugs which we first noticed in 7.1; try executing `NC'inx=vec'` having defined `inx`. The quick fix is to clear everything to the right of the `^` in `Gui_read[21]`. Unfortunately the fix for their new configure bug is not so easy, so if you get an ever-growing button in the designer (try it with `Snap=1`) you will need to register so that we can send you the necessary patch.

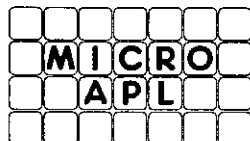
APL for the Apple Macintosh

Since 1985 MicroAPL has pioneered the use of APL in graphical environments. Our latest version of APL.68000 Level II for the Apple Macintosh is now available, offering dramatically enhanced GUI programming facilities.



APL.68000 Level II for the Macintosh includes the following features:

- Runs on all models of Apple Macintosh
- Native version for the Power Macintosh
- Conforms closely to the APL2 specification
- Uses the standard Mac interface
- Object-based GUI programming via DDI
- Full event handling via APL callbacks
- Free runtime version with application packager



MicroAPL Limited South Bank Technopark, 90 London Road,
London, SE1 6LN, UK
Voice: 0171 922 8866
Fax: 0171 928 1006
Appletlink: microapl
Internet: microapl@micoapl.demon.co.uk

THE EDUCATION VECTOR

July 1995

Editor Ian Clark

This Education Vector has been reprinted from VECTOR Vol.12 No.1. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Road, Redland, BRISTOL, BS6 6LS. Tel: 0117-973 0036.

Contents

Editorial	Ian Clark	18
Jot-Dot-Min		20
J-ottings 6	Norman Thomson	25
J - Where Have All the Variables Gone?	Walter G Spunde	30

Ian Clark
IAC/Human Interfaces,
9, Hill End, Frosterley,
Bishop Auckland,
Co. Durham DL13 2SX.

Tel: 01388-527190
Email: 100021.3073@compuserve.com

Editorial

I had a dream last night...

No, no, hang on... this really happened. Or at least it didn't really happen, but it did, if you see what I mean. I was overtaken in the street by a total stranger, except that I recognised him straight away. It was of course the British Prime Minister, John Major, and he begged a tremendous favour of me. It seemed that the following day he was going to be out of office. He'd picked on me because nobody looks more ordinary than me and he was facing up to the prospect of becoming just an ordinary person himself. A prospect he didn't relish because he didn't actually know any ordinary people. He anticipated being dreadfully isolated before he managed to make up socially for all those years in office and he wondered if I might consider taking him down the local pub and introduce him to a few of my friends, just to get him started.

Now some of my best friends are teachers, most of them in fact, or rather ex-teachers (not being able to stand the politically motivated hassle of the last few years). Just as I thought I had a handle on this thing I got a phone call from a BBC reporter I know called Chris. Now Chris has a nose for trouble. He once turned up with an entire camera crew on the off-chance of filming me getting beaten up by angry quarrymen at a protest meeting I was addressing. He was proposing to do the same that evening, since he was covering the story of how John Major managed his first day as an ordinary person. I was going to have to stage a casual introduction, in front of television cameras (and boy! — don't they take over a place when they arrive) along the lines of:

"Chris, this is — er — John."

I woke up sweating. Not that the dream was all that nightmarish, but it struck me how much worse it could have been. Just imagine:

"Chris, this is — er — Margaret."

Lady Thatcher went on record recently (at the time of writing) for castigating the present government for being too lily-livered. Among other things she prescribed wholesale diversion of resources away from the social services to the police.

People like that used to get themselves slung out of the Gestapo for cruelty. There was a time (before the National Health Service, and healthy eating) when it used to be said "Save on food — pay out on the doctor." To paraphrase it in the present context: "Save on teachers and social workers — pay out on prisons and

the police." Yet Lady Thatcher, if I'm not misrepresenting the darling, seemed to be promoting this as Right and Proper.

One thing I have always admired the Tory Party for is managing to get rid of its leaders just in the nick of time. But I hope it doesn't get into the habit of putting the skids under its ministers whenever the press say "jump". Especially when the volatile fraction seem to be treating an apparent desire at the top to correct the excesses of the last 12 years as weakness of leadership.

Cynics will say that there's no desire to do this at all. Just a general election on the horizon. But I've also heard it said, since Harold Wilson's day, that the British electorate can be bought for the equivalent of two weeks' salary. This is surely no less true than it ever was. It's marvellous what you can buy these days. Far more than you ever could under socialism.

So why, if there's really no problem about getting re-elected, are the government leaning over backwards to give the impression they care about the state of public services, in particular schools? Might they be priming us for something? Might it just be that any fool can see that the taxpayers are going to have to cough up for some good replacements for our wasting educational resources in the not too distant future? After all, like our nuclear power heritage, it would be hard to find a buyer for the schools system in its present state.

Recent Articles in Vector

Regular readers of Education Vector might like to know that Vector 11.4 carried a series of articles featuring J, in particular:

Teaching with an Executable Notation	Ken Iverson	76
The Perils of Subtraction	Donald McIntyre	93
Primes, Spirals and Coffee Tables	Keith Smillie	104
Stereograms in J	Eddie Clough	110
Cribbage 15s	Gene McDonnell	135
Functional Programming in J for Operations Research	Richard Brown	139

There was also a thorough review of J 2.05 for Windows by David Ziemann.

Back numbers of Vector are available from Gill Smith, Brook House, Gilling East, YORK YO6 4JJ, Tel: 01439-788385. The cost is normally £10 per complete volume, but in this case we would be happy to offer individual copies at £2.50 each.

Jot-Dot-Min

an outer-product of rock-bottom APL matters

My father once told me that as a boy he used to fish for flatfish in the river Clyde with a table fork tied to a stick. The idea was to creep about barefoot in the shallows until you felt something wriggle under your foot, then lunge down with the fork. Inevitably there was somebody who didn't get his foot out of the way in time and priceless excuses had to be invented for bloody footprints in the kitchen.

British flatfish like the plaice and sole have a pretty neat camouflage pattern on their topsides which mimicks the random speckle of a sandy bottom. But you don't stand a chance if you're a flatfish and your predator has stereoscopic vision, unless you can flatten yourself on the bottom and stay still. According to the evolutionists, mankind's marvellous capacity to spot speckled fish underwater must have been many millions of years in the making, around the shores of clearer waters than the Clyde, where we can imagine our brutish hominid ancestors peering into pools brandishing sticks with the elder-day equivalent of a table fork lashed to them.

Here's the sort of thing they were looking for, and (since we're all here) successfully spotted.



The two squares of random dots A and B together form a stereogram. This is not as elaborate a construction as the often-named "stare-ee-o" pictures created using J, as recently described in our parent magazine (Clough, VECTOR Vol 11 no 4, 110-116), but still impressive nonetheless. Some people find it easier than a

"stare-ee-o" to see the illusion. For best results you could mount the pair of images in a Victorian device called a stereoscope, which directs each image exclusively to one or the other eye. Failing this, fusion lines are provided below the squares to assist you. Simply allow the fusion lines to drift together, then focus on the pattern inside the fused image.

After a few seconds you will see a shape, a rectangle in this case, clearly detach itself and float above the surrounding dots. Dover soles aren't normally rectangular, but we'll overcome that problem later.

Not only do squares A and B together form a stereogram, but so do B and C and a quite distinct one too (it's a different rectangle). If you're lucky, you can see both sets simultaneously, because when you fuse A and B, by happy chance you also fuse B and C.

Were you to cut out the squares A and B and transpose them, you would again see the same rectangular shape, but this time in the form of a window. We've done it for you below:



Let us consider for a moment just how marvellous this capacity is. With two eyes having overlapping fields of vision you can clearly see an object which, for each eye considered separately, quite simply isn't there. What do we mean by that?

Image B is a random array of square blobs, actually blown-up pixels. Each pixel can be black or white with equal probability. The Apple Macintosh supports both black-and-white and coloured icons, 32 by 32 pixels each. APLomb (a software construction kit using I-APL as its scripting language) lets you turn an icon into a 32 by 32 array of numbers (a black-and-white icon becomes a Boolean array, with 1 for black and 0 for white) and back again. So Image B was produced as follows:

$B \leftarrow 1 + 732 \cdot 32p2$

Here's the explanation. Roll (?) followed by an integer value (scalar or array) returns an integer value of the same shape, but of random integers. ?2 is a random integer between 1 and 2, but we want a random 0 or 1. You can't get this with ?1, you just get 1 back again, so we must use ?2 and subtract 1 from it.

If you want an array of {0,1} which you can bias towards more 0s, you can use $50 > ?100$ in place of ?2. If you increase 50 to 85, say, then 0 will tend to arise for approximately 85% of the entries.

The "Dover sole" S was defined as a sub-array of bitmap B by means of the indexed assignment:

$S \leftarrow B[6+1:16; 9+1:18]$

Then S was rotated by one pixel (simulating the shift rightwards of S , or rather a rectangle one pixel narrower than S) and put back into C (a copy of B) over the same rectangle as before:

$C \leftarrow B$
 $C[6+1:16; 9+1:18] \leftarrow 1\phi S$

The result C is still a random array, with exactly the same statistical distribution of pixels as before. Mathematically speaking, nothing about the operation distinguishes the boundary of the Dover sole in C , except that it happens to be derived from B . Considered apart from B , it is mathematically impossible to detect in C that an arbitrary rectangle has been rotated by 1 pixel. Just to make the point, a different rectangle has been rotated the other way in B , using $1\phi S$ instead of $\bar{1}\phi S$ to make A .

However, fuse the images B and C together (or A and B), and the brain detects parallax. That is, it recognises that a region of pixels has shifted bodily sideways against a background formed by the remaining pixels, when viewed first by one eye, then by the other. Why doesn't the brain just see a blurred area where the value of the expression $B=C$ contains 0s, meaning no-match? If you repeat the experiment you'll notice that the 0s don't form a solid block, yet the illusion is of a crisp floating rectangle. Moreover both the pattern on the rectangle and the background pattern are precisely fused! You'd think the eyes could superimpose only one or the other at a time, not both at once. So $B=C$ is not what the brain is computing. Instead it would seem to be making a crafty guess at what might have caused the displaced pixels and decides, from its experience of Nature, that the most plausible explanation is that there is a well-camouflaged object with a

rectangular border floating above a speckled background. But then the brain goes and presents its hypothesis to you, not as a hazy probabilistic suggestion, but as certainty, i.e. as a rock-solid illusion of a floating rectangle.

If you equip yourself with a suitable APL capable of printing dot-arrays from rectangular Boolean arrays (or as we called them in the last issue, bitmaps), then here are a few experiments for you to try:

- (1) Use a non-rectangular mask, like a fish-shape. Does the brain still "see" a clear boundary? Don't forget, some of the displaced speckles might accidentally match the "underlying" ones, although that doesn't seem to bother the brain with examples A B and C above.
- (2) The Dover sole is "opaque", i.e. no underlying pixels show through. What if it were "transparent"? Is the boundary still crisp? Does swapping the two images still give the impression of a window, instead of a floating shape?
- (3) How small can the displaced rectangle be before the illusion is lost? Will the brain detect a single displaced pixel and portray it as a tiny "floating" fish? Or will it write off a single pixel out-of-place as "noise". Does it matter if the pixel coincides with an underlying speck when it is shifted?
- (4) There are programs which allow you to view a collection of similar pictures as an animated sequence. By generating several successive pairs of images, can you get the sole to "move"? Can you make it rise, by increasing the pixel displacement from one to two, or sink to the bottom by going the other way? If it does sink to the bottom, can you still "see" it, even briefly? If you can, you're seeing something for which there's absolutely no statistical evidence, at least in one particular frame.
- (5) What if the sole is displaced vertically or diagonally with respect to the background instead of horizontally? Do you still see a crisp illusion? Don't forget that the brain knows your eyes are in-line horizontally, so a vertical displacement going from one eye to the other is rather unlikely to arise as a result of parallax.

Now for some APL techniques with bitmaps to help in your investigations. To get a fish-shaped Dover sole, it's best to abandon indexed assignment to pick out and rotate a given sub-rectangle. The trouble is that defining a shape by means of $B[U; V]$ where U and V are vectors of consecutive numbers, means the shape has got to be rectangular. Used above, it helped us to make a philosophical point (the randomness statistics of the image with its rotated shape remained totally unaltered), but in practice you will find it easier to generate the speckled background once for all, then the fish as a separate array, then to overlay the fish first in one place, then shifted sideways by one column or maybe two.

First you need a mask of a fish in the form of a bitmap. It's best to edit this by hand. Suppose F is the fish shape as a 32 by 32 Boolean array, G is the random speckle on the fish, H the random speckle on the background. G and H may be generated as follows:

```
G ← 1 + ? 32 32 p 2
H ← 1 + ? 32 32 p 2
```

making two independent random patterns. Now use F as a cookie cutter on both G and H ; like this. $G \wedge F$ zeroes everything outside the fish shape, and $H \wedge \sim F$ zeroes everything inside the fish shape. Put them together and you have just placed the fish opaquely on the background (although it won't show up of course):

$$A \leftarrow (G \wedge F) \vee (H \wedge \sim F)$$

Now rotate both the fish and the speckled background:

```
F ← 1 φ F
G ← 1 φ G
```

and repeat the above steps to make B :

$$B \leftarrow (G \wedge F) \vee (H \wedge \sim F)$$

B now forms a stereo pair with A .

That's all the space we have for bitmaps this week. We'll get onto games of life and epidemic models shortly.

J-ottings 6

by Norman Thomson

There is a stage at which fluency in reading any language increases through recognition of idioms, and J is no exception to this rule.

One such idiom is "hook with commute". The simple hook

$$(g \ h) \ y$$

means perform h on y , and then, using y as left argument, do g to obtain

$$y \ g \ (h \ y)$$

The second verb h thus acts as a kind of pre-processor for the first verb g . If the first verb is to be the pre-processor, that is if what is required is

$$(g \ y) \ h \ y$$

the commute adverb can be used to reverse the order of g and h , so that the above line is equivalent to

$$(h \sim g) \ y$$

The advantage of this "bridged hook" is that only one reference is made to the argument. Informally it may be read as "use g as a transformation, then reverse the arguments".

To take a specific example, suppose that $1p = .+ / .*$ defines the standard matrix inner product, and m' denotes the transpose of a matrix m . $m'm$ is obtained as

$$(1p \sim | :) \ m$$

which is not to be confused with

$$1p \sim | : \ m$$

Removing the brackets means that this is no longer a hook, that is, the first verb, $| :$, is now executed in isolation to obtain m' . Then $\sim$ has the meaning "both" (not "commute") so that m' is applied as both left and right argument to $1p$ to give a final result $m'm'$.

Another example of a "bridged hook" is `$~$` as used in e.g.

```
0($~$)m
```

to define a matrix of the same shape as `m` but consisting entirely of zeros.

Another idiom to look out for is the J equivalent of the APL "each" operator. This is the adverb "under open" which applies its verb to a boxed list. It is convenient to define

```
each=.&.>
```

as a general purpose adverb, so that, continuing the above example,

```
0 1 ($~$) each <m
```

defines a boxed list of two matrices shaped like `m`, one all zeros, the other all ones. If the final boxing is not required (giving the result as a three dimensional array in the present example), use "with open" (`&>`) instead of "under open".

Indexing Arrays in J

One of the most difficult areas of the J repertoire to memorise is the "braces" area, which is deeply involved with indexing. This is an area in which J diverges greatly from APL on account of the priority of dimensions inherent in J data structure. New habits must thus be acquired in order to index J arrays.

There are occasions when what are essentially iterative algorithms require descent to the level of array elements, and recourse to looping cannot be avoided. It is therefore important to be able to locate array elements of any order with reasonable confidence.

Broadly speaking, there are two kinds of indexing, *rectangular indexing* and *scatter indexing*.

For vectors, `{` and `}` are interchangeable, although the former is recommended for consistency with higher rank arrays, leaving `}` to its primary meaning of "amend". `v{w` is equivalent to `w[v]` in APL, where both `v` and `w` are vectors.

For higher rank arrays, indexing a single element involves repeating `{` `n` times where `n` is the rank of the array. However, the order of appearance of the indices is the reverse of that in APL, that is if `a` is a three-dimensional array, `k{){1{a` locates element `a[i;j;k]` in the APL sense. Indices may themselves be arrays, but care has to be exercised in applying APL intuition. For example, if

```

]t=.2 3$'carmen'
car
men

```

0 1{t selects both rows of t, and so 0 and 1 are the only valid indices at the next level of indexing. Thus 0 2{0 1{t is an index error in J, and is not equivalent to t[0 1;0 2] in APL. However, the latter does have a direct J equivalent. It is obtained by using boxed lists, with one list corresponding to each dimension thus:

```

(<0 1;0 2){t
cr
mn

```

Scatter indexing is the process of selection using sequences of unrelated rectangular indices which select individual items. An essential intermediate process is that of calculating "linear indices" which translate rectangular indices into the corresponding indices in the ravelled array. The verb "base", #., is important in this operation, and is the subject of yet another of the "bridged hooks" described above, namely (#.~\$). This converts the index 1 3, say, in a 3 by 4 matrix to the linear index 7.

```

1 3(#.~$)1.3 4
7

```

To apply this as an index, the array right argument must be ravelled, and finally "from" (t) applied. This sequence of two initial transformations followed by a third operation on the transformed data is the essential characteristic of a fork. In the verb r1 below, superfluous brackets are retained on the right to emphasise the fork structure.

```

r1=(#.~$) { (,@])    NB. r1 = Rectangular Index
1 1 r1 t
e

```

Finally, to obtain scatter indexing

```

si=.>@(r1 each)<

```

uses a hook (r1 each)< followed by "open" which sheds the boxed structure.

```

(0 0;1 2)si t
cn

```

Suppose for definitiveness that n is an integer and m a matrix.

$n\{m$ is row n of m
 $n\{^J 1 m$ is col n of m (so also is $n\}^J 1 m$ although
 $n\}m$ is a rank error)

Update operations in programming languages have a fundamental requirement for three parameters in order to answer the questions:

What object is to be updated?
 Which locations within it are to be changed?
 .. and with what data?

The J update verb is "amend", which, alone amongst the primitive verbs cannot be fitted into a monadic/dyadic framework, and thus into patterns of hooks and forks. The order of arguments for what is essentially a triadic verb should be memorised, viz:

`(new data) (locations) } object`

Amend allows locations to be expressed as linear indices whatever the rank, and so `ri` can be adapted to obtain a "linear index" verb:

```
ll=.>@((#.-$)each <@] )
'by' ((0 0;1 2) ll t) } t
bar
mey
```

This method of update is an alternative to using the adverbs A and B suggested in the Dictionary under AMEND.

Rows and columns in a matrix can be replaced as follows:

`v (n{1.$m) } m` NB. Replaces row n of matrix m with vector v
`v (n{^J 1 i.$m) } m` NB. Replaces col n of matrix m with vector v

When the braces are qualified with $.$ and $:$, there is little in the physical shapes of the symbols to help indicate which of them denote head, tail and so on.

There are three binary sources of variation present, viz:

{ and } , . and : , +ve and -ve

and my suggestions are to read ...

- { as "all the" and } as "all but the"
- . as "first" and : as "last" (monadic only)
- as "from the right", and + as "from the left".

Thus $n\{.m$ means "all the first n rows of m "
 $n\{."1\ m$ means "all the first n columns of m "

$n\}.m$ means "all but the first n rows of m "
 $n\}."1\ m$ means "all but the first n columns of m "

adding "from the left/right" according to whether n is +ve or -ve.

The monadic forms are simply the dyadic case with n equal to 1, so

- $\{.m$ is the first row (head), and $\{ :m$ the last row (tail),
- $\{."1\ m$ is the first column, and $\{ :."1\ m$ the last column,
- $\}.m$ is all but the first row, and $\} :."1\ m$ all but the last column.

Be aware that "tail" in J does not have the same meaning as say, tail in LISP, which is matched in J by "behead" ($\{.$).

Errata

I have had a few minor errors pointed out in J-ottings 5:

- p.27 line -6 "rank" should be "shape"
- p.27 line -5 "joined" should be "padded and joined"
- p.27 line -3 "shape 3 2 4" should be "shape 4 2 4"
- p.28 line 1 "shape 4 4" should be "shape 3 5"
- p.28 line 2 "matched with" should be "extended and joined to"

J - Where Have All the Variables Gone?

by Walter G Spunde
spunde@usq.edu.au

One of the hardest, and most stimulating, things about learning J was (for the author) the challenge of removing any mention of variables in function definitions. The seductive elegance of the definition of the function for the mean as `mean =. +/ % #` hid the fact that the syntax of the function was clearly presupposed. In defining new functions one had to keep clearly in mind whether component functions were to be used dyadically or monadically. In short, one had to be keenly aware of how the data was to be given to the function and what would happen to it, although this information did not appear in the written functions (but, of course, could be deduced by parsing the statements). There was the continual tendency to see `[` and `]` as the left and right variables rather than as functions that returned those variables. Even with this fact clear, the penny did not drop. It was in a chat with Ken Iverson during his November '94 visit to Toowoomba about the naming of a function that (at least one layer of) scales finally fell from my eyes. Ken asked, 'Well, what is it that the function delivers?' Whatever it was, that should be the name of the function. Obviously. But obviously it is something that can be said again even if it has been said before because it seems to be missed so easily.

To demonstrate the importance of this simple principle, let us consider the problem of finding the width, w , of an interval (a,b) . Clearly the name of this function should be w , and to define it, one must *either* decide on the syntax of the function — will it be `a v b`, or `v (a,b)`, and remember to keep this clearly in mind — *or*, simply write `w =. b - a`, where, "obviously", a and b are functions that deliver the values of those variables and the expression for w is a fork. Choosing the latter approach allows one to specify either syntax for w . If w is to be used dyadically, then we define `a=. [` and `b=. ]` whereas if it is to be used monadically, then `a=. {.` and `b=. {:` are appropriate definitions. The definition of function w is not affected by the choice of syntax.

Next suppose the interval (a,b) is to be divided into n equal subdivisions each of width, h . The function that delivers the subdivision width should be called, h , and can be defined by `h =. (b-a) % n` where a,b,n are functions that deliver those variables from the supplied data.

The subdivision points are given by the function

```
subdivision_points =. a + h * i. @ (n+1:)
```

Finally one must decide whether this function will be called as

```
n subdivision_points (a,b)
```

or as

```
subdivision_points (a,b,n)
```

If we choose the dyadic form then we make the definitions

```
a =. {. @ ]
b =. {: @ ]
n =. [
```

and find that `subdivision_points f.` is

```
(. @] + ((({: @] - {. @] ) % [ ) * (1. @ ([ + 1: ))
```

If we choose the monadic syntax then we write

```
a =. {. b =. 1&{ n =. {:
```

and `subdivision_points f.` is

```
(. + ((1&{ - {. ) % {:) * (1.@ ({: + 1:))
```

There may be other preferred ways of delivering the data to the `subdivision_points` function for which the `a`, `b` and `n` functions would have to be defined appropriately, but the `subdivision_points` function needs no adjustment. In practice it will be found that a fixed function like

```
subdivs=. subdivision_points f.
```

will run six to seven times faster than the original `subdivision_points` function. (The monadic form is also marginally faster than the dyadic form.)

So where have all the variables gone? Gone to functions every one.

In algebra class, the key to beginning a problem is to say "let x be the number of ..."; in J the key might be to say "let x be the function that goes and gets the number of ...". It is probably a more palatable statement for students beginning a study of algebra.



Renaissance Data Systems
P. O. Box 421 - V
Georgetown, CT 06829
(212) 864-3078

**Books on APL and J and other curiosities
of merit!**

Renaissance Data Systems announces a change in its mailing address. Please note that the telephone number remains the same.

If you would like a copy of our latest **catalog**, please send us a self-addressed legal sized envelope with one first class stamp (if in the U.S.).

Included are such titles as: **APL is EASY**, **APL - An Interactive Approach**, **APL2 at a Glance**, **APL - the Language and its Actuarial Applications**, **APL as a Tool of Thought** proceedings, **I-APL** publications and software, **Boolean Functions and Techniques**, **The FinnAPL Idiom List**, **The Toronto APL Toolkit**, **Mathematical Experiments on the Computer**, **Probability in APL**, **APL - Stat: Do it yourself guide to computational statistics**, **A Source Book in APL** - Approximately 80 titles in all!

We also carry J publications, including **Programming in J**, **An Implementation in J (structure and source code)**, **Arithmetic**, and **Calculus**.

Shareware and commercial APL interpreters and J2 interpreters are available as well.

APL Product Guide — Updates

compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

Pressure on space occasionally prevents us from printing the complete guide, however updates will always be listed. We do depend on the alacrity of vendors to keep us informed about their products. Anyone who is not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 01439-788385, Email: 100331.644@Compuserve.com

APL INTERPRETERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Iverson Software Inc.	J Professional (inc runtime)	\$495	
	J Personal Edition	\$100	
	J Personal (disks only)	\$40	The text of the manuals is available as Windows Help, so the paper copies are no longer a necessity.
	APLIWIN	\$30	For 386/PC under Windows 3.1
	APL Reference Manual	\$30	Documentation for all the above.
	J System Kit	\$24	J 8.2 diskette with manual "J: Introduction and Dictionary"
Strand Software	J Source Code	\$90	Full C source code plus 100-page book
	Canada		
	All APL*PLUS Products	poa	All APL*PLUS products including upgrades and educational.
	Dyadic and ISI products	poa	
	USA		
	Dyadic and ISI products	poa	

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
Lingo Allegro	FRESCO Business Graphics	poa	Fast and Easy Business Graphics DLL
	AP126/PC	poa	GDDM Interface for Dyalog APL/W
	AP127/PC	poa	ODBC Interface for Dyalog APL/W
	AP119/PC	poa	TCP/IP Interface for Dyalog APL/W
	FACS	poa	EMMA-like Interface to DB2 or ODBC databases
	TOPR	poa	APL Code and Application Management for Dyalog APL/W

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
ComLog	Comic-Logger	\$25.95+ p&p	APL*PLUS II comic-book inventory system. Shareware version available on America OnLine.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES (£)	DETAILS
Snake Island Research Inc	Consultancy	poa	APL interpreter and compiler enhancements, intrinsic functions, performance consulting. APL compiler for serial and parallel systems now under test.
Strand Software	Consultancy	poa	Advice on migrating to and from all flavours of APL and hardware platforms. Full-screen interface implementation, APL utilities, benchmarking, efficiency analysis, actuarial software, system development tools, valuation, pricing and modelling systems.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
ACM/SIGAPL	Donna Baglio	ACM, 1515 Broadway, New York, NY 10036 USA Tel: +1 (212) 626-0606 Email: baglio@acm.org
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX, UK. Tel: 01245-495647; Fax: 01245-495646.
Adaptable Systems	Lois & Richard Hill	49 First Street, Black Rock 3193, Australia. Tel: +61 3 589 5578 Fax: +61 3 589 3220
Adatum Systems		13 Great George Street, BRISTOL BS1 5RR UK Tel: 0117-921 5555
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel +31-3474-2337, Fax: +31-3474-2342
Andrews	Dr Anne D Wilson	23, The Green, Acomb, YORK YO2 5LL, UK Tel: 01904-792670
APL-385	Adrian Smith	Brook House, Gilling East, York YO6 4JJ UK. Tel: 01439-788385 Fax: 01439-788194 Email: 100331.644@compuserve.com
APL Bay Area Users Group APLBUG	Lewis H. Robinson (Sec)	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: +1 (415) 928-2058 Email: frp21a@prodigy.com
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattenmann	Rheinstrasse 23, D-69190 Walldorf, Germany. Tel: +49 6227-63469 Compuserve: 100332,1481
The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: +1 (203) 762-3933 Fax: +1 (203) 762-2108
APL Interest Group, South Africa	Mike Montgomery	Private Bag X11, Rivonia 2128, South Africa Tel: +27 (11) 803-7200 Fax: +27 (11) 803-9134 Email: mikemont@spl.co.za
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS UK. Tel: 01225-482602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: +33 1 69-08-79-63
Atlantis Software	Arthur Whitney	1105 Harker Avenue, Palo Alto, CA 94301 USA
BACUS	Joseph de Kerf	Roolenberg 72, B-2570 Duffel, Belgium. Tel: +32 15 31 47 24
Beautiful Systems, Inc.	Jim Goff	308 Old York Road, Suite 4, Jenkintown, PA 19046, USA Tel: +1 (215) 732-3735; Fax: +1 (215) 772-1929
The Bloomsbury Software Co Ltd	Peter Day	3-6 Alfred Place, Bloomsbury, London WC1E 7EB UK. Tel: 0171-436 9481; Fax: 0171-436 0524
Camacho	Anthony Camacho	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9730036. email: acamacho@clx.compulink.co.uk Reuternet (Sharp): ACAM
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS UK Tel: 01252-874697 Email: 100430.740@compuserve.com
Paul Chapman		51B Lambs Conduit Street, London WC1N 3NB UK. Tel: 0171-404 5401. Compuserve: 100343,3210
Causeway Graphical Systems Ltd	Adrian Smith, Duncan Pearson	5 The Mallings, Castlegate, MALTON, North Yorks YO17 0DP UK Tel: 01653-696780 Fax: 01653-697719 Compuserve: 100265,1564
Chicago SIG	Larry Mysz	836 Highland Drive, Chicago Heights, IL 60411 USA C'serve: 73040,3032
Cinerea AB	Rolf Kornemark	Skyttegatan 25, S-193 00 Sigtuna, Sweden.
CODEWORK	Mauro Guazzo	Corso Calrol 32, 10129 Torino, Italy. Tel: +39 11 885168 Fax: +39 11 812 2652
ComLog Software	Jeff Padneau	PO Box 5570, Derwood, MD 20855 Tel: +1 (301) 990-7063 Email: jeff@softmed.com
CPCUG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: +1 (301) 762-9372.
David Crossley		187 Le Tour du Pont, Quartier Le Mourre, 84210 ST DIDIER, France Tel: +33 90-66-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: +46 31-45 37 40. Fax: +46 31-45 24 23.
Peter Cyrilax Systems	Peter Cyrilax	22 Hereford Road, London W2 4AA UK. Tel: 0171-229 5344
Danish User Group	Per Gjerfot	Email: gjerper@inet.uni-c.dk

Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salthouse Road, Brackmills, Northampton, NN4 0EX UK. Tel: 01604-760241
Dogon Research	Dick Bowman	2 Dean Gardens, London E17 3QP UK Tel: 0181-520 6334 Email: bowman@apl.demon.co.uk
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein, Netherlands. Tel: +31 3474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL UK. Tel: 01256-811125 Fax: 01256-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS UK. Tel: 01689-824741
Evestic AB	Olle Evero	Berteliusvagen 12A, S-146 38 Tullinge, Sweden Tel&Fax: +46 778 4410
FinnAPL		Suomen APL-Yhdistys RY, FinnAPL RF, PL 1005, 00101 Helsinki 10, Finland
General Software Ltd	M.E. Martin	22 Russell Road, Northholt, Middx, UB5 4QS UK. Tel: 0181-864 9537
Greymanite Associates	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY UK Tel: 01442-878065 Email: 100412,1305@compuserve.com
Hartford CT Group	Bob Pomeroy	Mass Mutual Life, 1295 State St, Maildrop F465, Springfield, MA 01111 USA Tel: +1 (413) 788-8411x2838
H.M.W.Trading Systems Ltd	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA UK. Tel: 0171-353 8900; Fax: 0171-353 3325; Email: 100020.2632@compuserve.com
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 USA Tel: +1 (703) 528-7824; Email: dick.holt@com.org
Michael Hughes		28 Rushon Road, Wilbarston, Market Harborough, Leics. LE16 8QL UK. Tel: 01536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX UK Tel: 01388-527190, Compuserve: 100021,3073
I-APL Ltd	Anthony Camacho (for queries, order forms) J C Business Services (for pre-paid orders only)	11 Auburn Road, Redland, Bristol BS6 6LS UK. Tel: 0117-9760036 email: acamacho@clx.compulink.co.uk Reuternet (Sharp): ACAM 56 The Crescent, Millon, Weston-super-Mare, Avon, BS22 8DU UK
IBM APL Products	Nancy Wheeler	APL Products, IBM Santa Teresa, Dept M46/D12, 555 Bailey Avenue, San Jose CA 95141, USA. Tel: +1 (408) 463-APL2 (-2752) Fax: +1 (408) 463-4488 Email: APL2@vnet.ibm.com Cserve: GO IBMAPL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD UK Tel: 01732-885128
INFOSTROY	Alexei Miroshnikov	3 S. Tulenina Lane, St. Petersburg 191186 Russia. Tel: +7 812-3111611 Fax: +7 812-3153321 Email: airm@infostroy.spb.ru
Insight Systems ApS	Morten Kromberg	Nordre Strandvej 119A, DK-3150 Hellebæk, Denmark Tel: +45 42 10 70 22 Fax: +45 42 10 75 74 Email: insight@inet.uni-c.dk
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 0171-265 1120
Interprocess Systems Inc.	Stella Chamberlain	11660 Alpharetta Highway, Suite 455, Roswell, Georgia 30078, USA Tel: +1 (404) 410-1700. Fax: +1 (404) 410-1773 Cserve: 70373,2676
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: +1 (416) 925-6096; Fax: +1 (416) 488-7559
JAD Software	David Crossley	580 Eyer Drive, #81 Pickering, Ontario, Canada L1W 3B7 Tel: +1 (905) 837-1895 Fax: +1 (905) 831-5172
Japan APL Association		23-2-302 Hiromichi, Adachi-ku, Tokyo 120, Japan
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessmer Drive, Stevenage, Herts SG1 2DX UK. Tel: 01438-310155 Fax: 01438-310131
Lingo Allegro USA Inc.	Victoria H. Fil	113 McHenry Road, Suite 181, Buffalo Grove, IL 60089 USA Tel: +1 (708) 459-7529 Fax: +1 (708) 459-8501 Email: info@lingo.com
MasterWork Software Ltd	Fraser Jackson	PO Box 56-036, Tawa, Wellington, New Zealand. Tel and Fax: +64 (4) 232-4440 Email: 100242.2535@compuserve.com
Melbourne APL Group	Harvey Davies	CSIRO Div Atm Res, Private Bag No.1, Mordialloc, Victoria 3195, Australia Tel: +61 3 588 7574 Fax: +61 3 588 7600 Email: hld@dar.csiro.au
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Heneage Street West, Aston Science Park, Birmingham B7 4AX UK. Tel: 0121-359 5096. Fax: 0121-359 0375
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN UK Tel: 0171-922 8866 Fax: 0171-928 1006 Email: MicroAPL@microapl.demon.co.uk

Ellis Morgan	Ellis Morgan	Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants UK. Tel: 01730-263843
New York SIG APL	Nestor Nelson	PO Box 138, NY 10185-0002, USA
Optima Systems Ltd	Paul Grosvenor	Airport House, Purlay Way, Croydon, Surrey CR0 0XY UK. Tel: 0181-781 1812 Fax: 0181-781 1999
Potomac APL SIG	John Martin	Computer Sciences Corp, 1100 West St, Laurel, Maryland 20707-3587 US Tel: +1 (301) 497-2998 Fax: +1 (301) 498-8260 Email:jam@acm.org
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB UK. Tel: 01276-855860 Fax: 01276-855301
Renaissance Data Systems	Ed Shaw	PO Box 421, Georgetown, CT 06982, USA. Tel: +1 (212) 864-3078
RE Time Tracker Oy	Richard Eller	PO Box 363, FIN-00101 Helsinki, Finland. Tel: +358-0-400 2777
Rochester APL	Gary Dennis	Solliton Associates, 1100 University Avenue, Rochester, NY 14607 USA Email: gsd@lpsalab.tor.solliton.com
The Rochester Group Inc.	Robert Pullman	50 S.Union St., Rochester NY 14607-1828, USA. Tel: +1 (716) 454-4360. Fax: +1 (716) 454-5430
Rome/Italy SIG	Mario Sacco	Casella Postale 14343, 00100-Roma Trullo, Italy Email: marsac@vnet.ibm.com
SE APL Users Group	John Manges	991 Creekdale Drive, Clarkston, GA30021 USA
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St. Giles, Bucks HP8 4QH UK.
Snake Island Research Inc	Bob Bernecky	18 Fifth Street, Ward's Island, Toronto, Ontario M5J 2B9 Canada Tel: +1 (416) 203-0854 Fax: +1 (416) 203-6999 Email: bernecky@eocg.toronto.edu
SOCAL (South California)	Roy Sykes Jr	Sykes Systems Inc, 4649 Willens Ave, Woodland Hills, CA 91364-3812 USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Solliton Associates	Laurie Howard	Solliton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 646 4475 Fax: +31 20 644 1206 Email:ljh@solliton.com
SovAPL	Alexander Skomorokhov	PO Box 5061, Obninsk-5, Kaluga Region, Russia Email: askom@ap2.obninsk.su
Strand Software Inc	Anne Faust	19235 Covington Court, Shorewood MN 55331 USA Tel: +1 (612) 470-7345 Email: amfaust@aol.com
Rex Swain	Rex Swain	8 South Street, Washington, CT 06793 USA. Tel: +1 (203) 868-0131 Fax: +1 (203) 868-9970 Email: rswain@ix.netcom.com Compuserve: 70062,2303
SWAPL	Stuart Yarus	PO Box 210367, Bedford, Texas 76095, USA Tel: +1 (817) 577-0165 Compuserve: 73700,2545
SwedAPL	Christer Uthielim	Novator Consulting Group AB, Svärdvägen 11C, S-182 33 Danderyd Sweden Tel: +46 8 622 63 50 Fax: +46 8 622 63 51 CServe: 100341,404
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland Email: si@ifi.unizh.ch
Sydney APLUG	Rob Hodgkinson	PO Box 1511, Macquarie Centre, NSW 2113, Australia Tel:+61 2 257 5313
Sykes Systems Inc	Roy Sykes Jr	4649 Willens Ave., Woodland Hills, CA 91364, USA Tel: +1 (818) 222-2759 Fax: +1 (818) 222-9250
Toronto SIG	Marc Griffiths	PO Box 384, Adelaide St Post Office, Toronto Ontario M5C 2J5, Canada Tel:+1(416) 532-0843 Email: marcg@utoronto.ca CServe: 76260,3314
Uniware	Eric Lescasse	Tour Neptune, Codex 20, 92086 Paris la Defense 1, France. Tel: +33 (1) 47-78-78-00. Fax: +33 (1) 40-90-04-11
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd, Whitehaven, Cumbria, CA28 6JD UK. Tel: 01946-692568
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL UK Tel: 01203-523369
Zark Incorporated	Gary A. Bergquist	23 Ketchbrook Lane, Ellington CT 06029, USA. Tel: +1 (203) 872-7806

Making APL Readable

(PhD thesis of Christoph von Basum)

reviewed by Anthony Camacho

This book proposes the adaptation of an idea of Knuth to APL2.

The purpose is to produce a method for writing APL2 which will ensure it will be readable.

The suggestion is that the programmer writes a file which is combined documentation and specification which can be processed to produce prettily printed program documentation and/or processed to extract just the code and produce a workspace.

Since processing has to be done before the code can be executed this loses the interactive nature of APL2: it introduces a step as obstructive as compilation.

Are the results worth it? One cannot tell from the book because the only fully worked example is of the function *DISPLAY* — the utility included with APL2 which he criticises (quite unreasonably in my opinion) for being hard to follow. The criteria for judging a utility which needs to be fast, self-contained, compact and, once issued, never to be changed or maintained are not the same as the criteria for application code which probably will have to be maintained by programmers who didn't write it. The book is really aimed at writers of application code.

His example of documentation — *DISPLAY* stretched to many pages — did not strike me as the perfect way to do it although, at the end, even an expert would probably understand it better. No alternative way was offered: the obvious suggestion that clear comments could be added was not made or considered or rejected. Maybe he did not know that writers of APL2 utilities commonly remove comments when the code is fully tested so that what is issued is as compact as possible.

The author actually wants to do more than allow programmers to write readable APL2. I think there are several of these things.

He wants people to read APL2 programs with ease and he wants to force APL2 programmers to write in a way that makes this feasible. But of course there is a snag which he shows he is well aware of: the level of documentation required

depends on the skill of the reader. Some people can read quite complex APL2 expressions without explanatory comment. So his standard is pitched at his own level of understanding yet this is not explicitly recognised. Implicitly it is because he suggests that *S++/X* a *Sum the elements of X* isn't helpful.

He wants to force programmers to write clear English and to force them to write good code divided into well-structured modules. To this end he has, in section 7.3, assembled a series of injunctions "Know the audience", "Get the reader involved", "Revise and rewrite", "Do not overwrite" and "Do not overstate" each of which is supported by a paragraph of explanation.

We wonder how many of these were used on him by his supervisor. There are whole shelves of books on how to write good clear English; maybe he had to include something about it; maybe readers will agree with his choice, maybe not.

And to follow his advice one needs to learn TEX as well as a whole lot of arbitrary rules for his APL2web product before one could start to write APL2 as he wants us to.

What he wants us to write is a mixture of text, the TEX commands to format it, the APL2 code and the specialised syntax of the APL2web system. As he admits this stuff is hard to read. Only after processing it does one get to see or print 'Prettyprinted' documentation. This mixture is hard to write:

Any APL2 programmer who wants to document his or her programs in the literate style must not [only] learn the APL2web syntax but must know TEX. And TEX offers quite a few pitfalls of its own. It was initially "intended for the creation of beautiful books — and especially for books that contain a lot of mathematics".

What a lot one would have to learn before writing a line of code! Could anyone who had not devised the system do it? In a PhD thesis it is understandable that the decisive test — getting someone to learn and use the system from scratch was not tried. I am not convinced and will stick to documenting code with comments and writing the manual in WinWord, importing code from tested workspaces and using Adrian Smith's Vector APL Typewriter for the odd APL characters in a text paragraph. My prejudice is for a method he considers on page 206:

"It might be worth considering to develop a user handbook before even the first line of code has been written. A rough sketch of the handbook may be the introduction of the program documentation. Every programmer knows what the final program is to look like. The handbook does not describe the final product, but serves as an obligation to every programmer to meet the goal. A handbook is written in a different style since it communicates primarily with

the newcomer. (It must permit the expert user quick access to a wanted topic!) Besides the program itself, the handbook must transmit the commitment created by the new software package to the user. If the handbook is written in advance, every programmer must feel compelled to meet the requirements of the commitment created therein and to develop an excellent product."

My method is to write the user manual first. This ensures I can state the user's purposes in language the user can understand and asks him to agree the specification before any work is done. Once this is agreed it is usually possible to agree a good data structure. Then, if there are many staff involved, the manual can be presented to them to see if they are happy with the control mechanisms they are to operate. Changes are made to accommodate their criticism. Then the function headers and the top level comments in each function are written before entering a line of code. Glenford J. Myers' book *Reliable Software through Composite Design* is a good guide to refining the design to get the best structure. Restructuring at this stage if it seems desirable wastes little work. After this the coding and testing are easy.

Christoph von Basum's own task was even harder than the one he set his programmers: he had to explain and justify all his procedures in English, which to him is a foreign language. He should have asked a native English speaker to correct the text for him as there are many non-idiomatic phrases and even once or twice sentences where it is not clear what he intended to say. He has cords in his thesis where English arguments have threads; he has a complex software where English would insert piece of and many others. Lastly there is no index.

Book Announcement

We have just received a first announcement of "APL2 in Depth" by Norman Thomson and Ray Polivka. The publisher is Springer-Verlag, and the US price is \$39.95.

"Emphasis is placed on covering those APL2 language features which extend user versatility in describing data structures and communicating algorithms in ways which mirror current array-oriented thinking in computer science and software engineering"

We hope to have a copy in time to review it in the October Vector.

RECENT MEETINGS - APL95

This section of Vector documents all British APL Association meetings, and any other events of interest to the APL world. If you have recently attended any gathering which you feel would be interesting to Vector readers, please let the Editor have a brief note, and we will include it here.

APL95 has come and gone, and the Vector team have done their best to recover from the heat and the jet-lag in time to get a conference report together in double-quick time. Even so, we are running about one week behind our normal schedule, so apologies in advance if Vector has failed to reach you in July!

Thanks to Ray Cannon, Dieter Latterman and Anthony Camacho for the various photographs.

The conference challenge: Adrian's name-badge was mis-spelled as "Andrian Smith"; place the minimum number of APL symbols to the left of the text vector '*Andrian*' to remove the unwanted 'n'. In Dyalog APL you can do it with 3 characters!

APL95

APL 95 at San Antonio, Texas

notes by Anthony Camacho

The conference began on Monday 5 June. Before the opening plenary there was an excellent explanation of why APL is so valuable to actuaries, by Rhonda Aikens of United Services Automobile Association — a really large insurance company.



The Audience at the Opening Plenary

The opening plenary was an analysis, by John Feo of the Institute for Scientific Computing Research at Lawrence Livermore, of the reasons for the comparative failure of SISAL (a functional scientific computing language) to achieve the level of usage and acceptance its developers aimed at. The analysis could also be applied to APL.

The lunch breaks were occupied by Vendor Forums: Dyadic on Monday, IBM on Tuesday and Soliton on Wednesday. The conference ended before lunch on Thursday with a closing plenary by Charles Brenner who explained the work of the paternity laboratory, which uses DNA analysis to provide evidence to

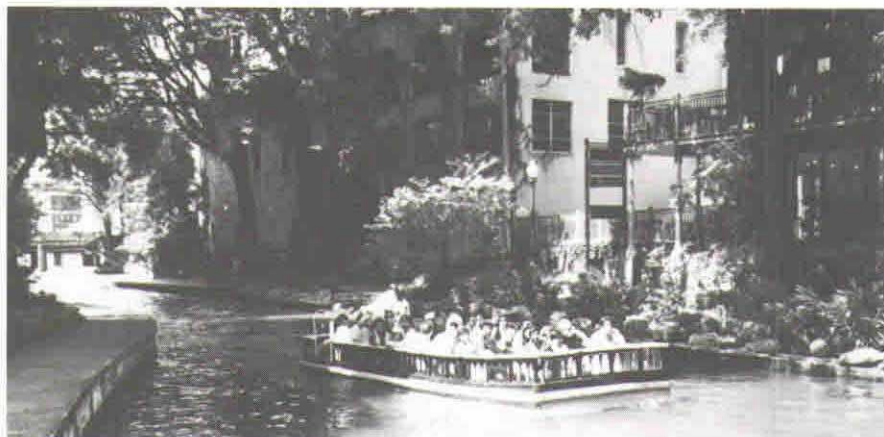
exonerate a man from the accusation that he has fathered a child and occasionally can establish parentage beyond reasonable doubt. He didn't show any APL because his portable and the display were incompatible.

The rest of the conference most of the time ran four streams in parallel — a seminar, a workshop and two refereed papers.

This meant a wider choice than usual and the opportunity for much fuller treatment. Seminars ran for 1.5 to 6 hours and many workshops for 3. Unfortunately there were no hands-on workshops. To provide a roomful of computers was too difficult or expensive.

There were under 140 delegates plus 60 or 70 people involved with the exhibition and conference organisation so splitting them into four streams sometimes meant a speaker had a very small audience.

Sometimes the organisers guessed wrong which sessions would be most popular and late room changes interrupted some presentations, but on the whole the conference ran very smoothly, with little trouble.



90 degrees in the shade!

Having almost everyone stay in the same hotel where all the sessions and meals were did help to make it possible to hold ad hoc meetings or to go down to the bar for company if one needed it with a good chance of finding it. When the time in the lift is the major part of one's travelling time one can do more, see more and end the day less tired.

On the other hand the negotiated special rate for the hotel, though good value for that hotel, was not cheap and for two people sharing a room the cost of registration and hotel was around \$1500 plus meals. In our case the fare from England and back was almost as much and so the total cost for two people was over \$3000.

At this rate very few people who have to pay their own way choose to attend. Most delegates were paid for by their employers and it is now the accepted wisdom that a conference full of seminars and workshops is more likely to motivate employers to send their staff than a conference that looks like a holiday.

The St Anthony hotel occupied a whole side of Travis Park, which is square, and the banquet was held in the open air on the paved area around the memorial to 'Our Confederate Dead' in its centre. The food was Mexican-style and there was a guitar playing trio, a cowboy skilled with whip and juggling and a (very loud) Mexican 8-piece band. There were Margheritas and beer to drink; the beer didn't run out and the promised cash bar did not materialise. Wine drinkers were not catered for. The San Antonio police and hotel security staff kept out interlopers.



Peter Donnelly receiving his shared plaque from Mike Kent, (with Melissa Dalton of C/M Planners in the background)

At the banquet the Kenneth E. Iverson award for outstanding achievement was presented by Mike Kent to John Scholes and Peter Donnelly. In response John and Peter gave handsome credit to their colleagues in England. Stuart Yarus

announced that APL 96 would be held in Europe with himself and David Eastwood (of the British APL Association and MicroAPL Limited) as joint chairmen and with Adrian Smith and Phil Benkard as joint programme chairmen. The location is not yet fixed.



John Scholes shares the Award

six-hour seminar by Donald McIntyre, a three hour seminar by Ken Iverson aided by Roger Hui and a 1.5-hour seminar by Chris Burke. There were even occasions when two J sessions clashed. I am very glad to see how far the APL community has come to recognise that J is APL and of great interest even to an audience predominantly consisting of commercial APL programmers. As recently as St Petersburg many people were taking the view that 'J is not APL unless written by a Russian'.

There were some marvellous APL papers. Bob Bernecky awarded prizes to Kevin Harzfield (for his paper on modelling the effects of acid leaching on concrete), to Clare Townsend (for her APL implementation of the Club of Rome's world model of population) and to (Bill) Koko (for the presentation of his six-hour seminar on Nonlinear Constrained Optimisation).

The hotel was less than two blocks from San Antonio's attractive Riverside Walk which has paths along both sides of a narrow loop in the San Antonio river with many restaurants, bars and shops. There is even a theatre with the seats on one side of the water and the stage on the other. It was also an easy walk from the hotel to the Alamo, the Mexican ethnic market, the craftsmen's colony 'La Villeta' and many good shops.

For me the outstanding feature of the conference, which was so well designed and run to be attractive and valuable to corporations and their employees, was the amount of J.

I spent the majority of my time in J sessions and nearly all were well attended. There were eight J papers, a

As John Searle was unable to attend we cannot bring you his usual collection of quotations. Here is the best I could glean:

Roger Hui, explaining his programming style: 'I adopted it in self-defence so I could keep up with Ken'.

Donald McIntyre: 'Everywhere you look you see almost nothing but forks'.

Ken Iverson: '... engineers — characterised not so much by their ignorance as their pride in it.' 'A mathematician, no matter what problem you give him, reduces it to the previous case.' Of Graham, Knuth and Patashnik's book on Concrete Mathematics: 'The way they managed to make do with conventional mathematical notation is an absolute marvel.'

Harvey Davies: 'Even the driver's blind dog knows that zero divided by zero is bugger-all.'

A rumour heard at the banquet: 'Dyalog's migration level $\square ML$ 99 will give Dyalog users 100% compatibility with J.'

Phil Benkard: 'Scalars conform easily, because they have no axes to grind' and (later in the same session) '... and they are not afraid of getting in over their depth.'

Phil Benkard: 'With Vector notation, you can't create a 1-item vector.'



Roger Hui with Chris Burke



Phil Benkard wearing two hats!

My Conference

by Sylvia Camacho

Everyone attends a different Conference as a consequence of their choices between sessions. This year there were papers, panels, workshops and seminars in addition to invited speakers and plenary sessions. So here I am describing my conference.

This year was also unusual in having an invited speaker before the opening plenary. Rhonda Aikens of United Services Automobile Association struck an upbeat note by her enthusiasm for the use of APL for a core set of tools used by her Company. She was accompanied by some of her staff and the suggestion that they would be around the Conference looking to see what they could learn or teach was encouraging. I wonder, did it happen?



John Feo with Bob Bernecky at the Opening Plenary

The opening plenary was a presentation of the flip-side. Dr John Feo of the Institute for Scientific Computing Research spoke about the SISAL language project which has been under development for the past twelve years but has made little headway against FORTRAN, COBOL and LISP. He is now of the opinion that it would be preferable to work to develop new facilities within

established languages such as FORTRAN and points to the success of C++ as an example. I felt that the vendors who presented their latest changes at APL95 had taken most of this message on board and are now moving to a much more favourable position in the market.

The Dyadic presentation by John Scholes was a superb piece of laid-back theatre which gave the impression of a confident and competent group who like what they are doing and enjoy each others' company. For those who know the duck joke, this time there were multiple ducks on multiple ponds and I predict that next year they will lay eggs.

The IBM vendor presentation by Jim Brown was workmanlike and carefully targeted to support those many programmers who have to support old-fashioned, largely mainframe, systems. I believe they will be stimulated to investigate the many ways IBM now makes it possible to give perfectly serviceable code a modern look and feel without the expense of a major rewrite.



Team Causeway at '95

The confident forward-looking stance was also very noticeable in Adrian Smith's workshop based on his *Rain* workspace which is shipped as shareware with Dyalog APL/W. The graphics are superb and potential users should have come away confident that they will be given good support over a wide range of platforms. This impression was reinforced by the presentation of the Causeway GUI software, which Adrian and Duncan Pearson's new company is now supporting. Thanks to both for taking the Mayor's Suite and for the beer they provided in it.

There is a convergence of ideas within the APL community that tools are now available to enable the complex algorithms and fast development times, which have always characterised APL, to be offered to users in any guise which makes them feel comfortable. If they have a favourite spreadsheet, you can now hide APL behind it to do the number-crunching. If they are Visual Basic experts wrap their APL up that way. On the other hand, if they are APL experts, the vendors are fast learning to match and outmatch the GUI functions of all the popular packages. In this respect APL is moving towards the position in the market

currently occupied by C++ which can only be coded by experts or very enthusiastic amateurs, but naive users need never be aware that APL is the engine at the back of their screen. The closing plenary was a good example of this; Charles Brenner needs APL to make his heavy analysis change rapidly with his research but he directed the interest of his audience entirely towards his fascinating investigation of comparisons of DNA which has been made possible by the Genome Project.

The bridge for me between these commercial considerations and the rest of my Conference was exemplified by the fairly intense reaction of Alan Sykes towards Walter Spunde's championship of a concrete numerical approach to the teaching of Calculus. Alan is concerned that the abstract methods traditional to mathematics will fall victim to the power of the computer. As an interested onlooker I think they are both right.



Donald McIntyre and Anthony Camacho showing his J side

The remainder of my conference was taken up by J. First with Donald McIntyre's seminar which made me believe at last that I could come to terms with J and that there is no better way to spend my time. I know that it is hard for professionals still needing to make a living, whether as Vendors or programmers, to accept that Ken Iverson's second thoughts are a great advance on the APL notation which has had no rival over so many years. As the editorial to Vector 11.1 admitted, APL is a language for the elite and practitioners have a large investment in it. J is a dialect of APL and the relationship is obvious but there is no upward compatibility and the learning curve is steep.

The Exhibition



Dave Liebttag and Jim Brown on the IBM Stand



Jaci and Bill Rutiser of Manugistics



Peter Donnelly of Dyadic

The seminar on Ken Iverson's new book '*Concrete Math Companion*' was given by Ken with Roger Hui at the keyboard. It was a delight for the way it demonstrated how well their minds are matched. It is always a pleasure to see a highly intellectual enterprise pursued with total commitment. However it was

Chris Burke who added the icing to the cake for me by showing that the version of the J software now being sold by Strand Software has the facility to present a GUI face to the world. What is more, as J is based entirely on ASCII files, it will look to naive users like a conventional motor even if they lift the bonnet (Brit for 'hood'). Consequently I believe that this language for the elite could come into widespread use. Why should J libraries not come to rival FORTRAN or C libraries? It has none of the unfamiliar look imposed by the special environment and character set of APL and thus J developers have the enormous advantage of moving onto a greenfield site, but they can build using the computer infrastructure established over many years by the major software vendors and accepted without question by computer users.

Two Machine-Learning Seminars

notes by Jon Sandles

At the conference there were two three-hour seminars on two of the most popular current methodologies for machine learning — genetic algorithms and neural networks.

Genetic Machine Learning — Dr Andreas Geyer-Schulz

This seminar was well structured and well presented. It began with an introduction to a problem that Andreas gives his students (at the University of Vienna) to try to solve. The problem is the MIT beer distribution game. For those unfamiliar with the concept, it is made up of four decision makers/planners at four different levels in a supply chain — namely Retailer, Wholesaler, Distributor and Factory. Each level works independently and only sees the orders coming from the level above.

Only the retailer sees the true order pattern. The object of the game is to minimise stock holding. When four humans play the game they exhibit an inability to control the stock the further they get down the chain; once you get to the Factory level, the lag effects on the orders causes the factory production/stocks to be very uneven. It was shown that a very simple genetic algorithm could outperform the human effort by orders of magnitude.

Genetic algorithms were then introduced along with some simple APL code. It was noted that they rely heavily on the quality of the random number generator underpinning the system, and that poor random number generators will

considerably slow the time it takes for the algorithm to converge on a solution. (It was noted that most APL random number generators exhibit cyclical effects which would be undesirable — Andreas mentioned he had some good random number generators — presumably in C.)

The **schema theorem** which underpins much of the GA theory was explained. This was done very well and I nearly understood it this time! Different variants of GAs were explained and a classification of these was introduced, including an indication of how you might choose between them, depending on how you choose to represent the problem.

The seminar then progressed further into some of the more explorative areas of the topic. Overall this was a well-structured and well-delivered seminar, covering both elementary and advanced areas of the subject. For further reading I recommend Andreas's paper in the conference proceedings, and especially his supplementary slides that came with the supplementary proceedings.

Neural Networks in APL and J — Ben Best

This seminar was split into two halves — the first half was an introduction to the topic requiring no prior knowledge, the second half a more advanced tutorial building on the first half. A historical perspective was put on the topic and some of the biological inspiration behind the early research was presented (modelling the brain, the neurone etc.) A set of exercises was used to demonstrate how a single-layer network can solve simple linearly separable problems.

The last exercise required us to solve the EX-OR problem with a single-layer network — after five minutes of trying we still had not solved this simple problem. *Of course, this is the simplest example of a non-linearly separable problem and can not be solved using a single-layer network!* It was precisely this point that halted much of the research into Neural Networks through the 70s. It was not until the 80s when a method of teaching multi-layer Neural Networks how to learn was developed, that research restarted. This first part of the seminar was a very good hands-on introduction to the topic.

Unfortunately the rest was not so good. Lots of different variants of Neural Networks were introduced — but these just seemed to confuse the issue — there was very little why and how from this point onwards. I would have preferred it if the hands-on approach had been exploited further and we had stuck to 'simple' multi-layer networks. The end deliverable of a three-hour seminar like this should be that you would be able to build a Neural Network (in APL) and use it to solve a problem.

I came away with a good historical introduction and a lot of confusing / conflicting ideas on the more advanced stuff. It was explained why APL/J would be good for programming Neural Networks — but one felt a real opportunity was missed — we could have all tried to implement a simple Neural Network in APL if we had all had access to PCs. The back-propagation algorithm for teaching Neural Networks was explained, but was not put into the context of a real problem, hence I came away not really knowing how to tackle a problem from start to finish using Neural Network technology.

Business Graphics Workshop

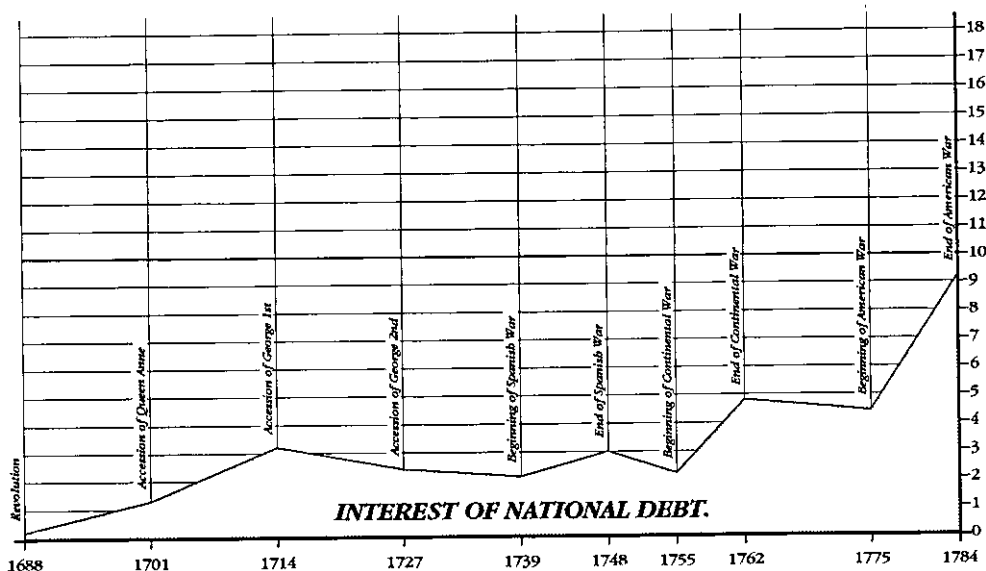
notes by Adrian Smith

From the presenter's point of view, a 3-hour workshop is a little tougher to prepare than a 45-minute paper, but infinitely more rewarding. I think we all got good value from this — I used it in part as a teaching opportunity and in part to explore what people really wanted from an APL graphics package. I was particularly keen to get a consensus (see Eke van Batenburg's paper in a recent QuoteQuad) on the requirements for 3D data-presentation and basically to prove that my *Rain* utilities were heading in the right direction to become an accepted standard for business graphics in APL.

The first hour and a half was mostly examples and philosophy. I quickly realised that many of the audience were already Tufte converts ("*The Visual Display of Quantitative Information*" — Graphics Press ... buy it!) and were aiming to go above and beyond the facilities in Excel or GRAPH.VBX. The afternoon was punctuated by questions beginning "I tried to in Excel and found there was no way I could do it!". I worked hard on the message that a good business graphic requires the classic "concept — design — program" cycle and that a package with a bunch of options (however sophisticated) will never do exactly what you want. A good example is the "Playfair" classic from Tufte's book where the x-axis tick marks (and grid lines) are computed from the data, and vertically-written notes are also placed to harmonise nicely with the rise and fall of the British National Debt over time.

I think I also got the message across that typography matters, and that you must take great care when scaling a large graph down in size (say to print 4-up on the page). *Rain* automatically thins out the tick-marks, for example if there was space on the full page to mark 0 2 4 6 8 10 12 ... it is likely that a half-size version will only have space for 0 5 10 15 ... and a 'Thumbnail' may need to come down to 0

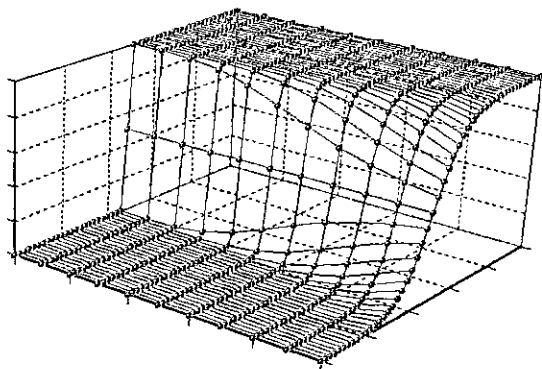
10 20. Surprisingly (but see any manual on typography) the type-sizes scale down roughly in square-root proportion, so that in halving the size of the chart you probably come down from 18-point to 12-point for your main heading! Line-weights adjust in the same way, and of course it is important to get the right relationship between the main axes, the major and minor ticks, and grid lines.



In the second part of the afternoon, we opened up a much more general discussion on "where next" with business graphics and APL. Two topics were of particular interest:

- how should a graphic report user-events back to a Windows APL application? Clearly there is a need for interactive graphs — users want to click on the heading and be offered an edit box to change the text/placement/style. They want to select data points and hide or annotate them. What is needed is a general set of hooks for the application to step in and do something specific.
- 3D presentation of data. I was amused to see the wonderful old *SKYSCRAPER* plot in an IBM handout on the latest graphics tools for APL2! However, I appreciate that there is a real need for tools which help you to visualise 3-dimensional data. Experimental results where *Yield* is some function of *Pressure* and *Temperature* are always a cloud of points in 3-space. What the experimenter needs is some way of visualising the shape of the cloud, and 2D cuts are never enough to do this adequately.

As it happens, all the code you need to actually draw the stuff was published in Norman Thomson's excellent "*APL for the Mathematics Classroom*" (Springer-Verlag 1989); the tricky part is specifying the viewpoint and the degree of perspective distortion in a way that users could control with a bunch of Windows tools (slider-bars and the like). There was universal agreement that the Excel "grab the corner and pull" method was just plain confusing, and that something more like the CorelDRAW! 'extrude' tool was appropriate. To help the discussion along, I showed some simple demo stuff (all totally hard-coded) like this ...



One completely off-the-wall idea was to use Flight-Simulator as a model and 'fly' your viewpoint around and through the data. I feel a Pentium may be barely adequate for the job!

We closed with some thoughts on the integration of graphics with text, in the form of management reports and technical papers. I have always assumed that it was adequate to push out the completed chart as a .EPS file, and that some high-end publishing tool would take care of the rest. However several people have tried this route and found that they really wanted the whole process under APL control, as the text (which sections to include, what special conditions to apply) was also very data-dependent. This seems to be particularly true of insurance companies where just about every policy is different!

All in all, a useful 3 hours, both for me as presenter and for the majority of the audience, who kept it going well past the official finishing time. Thanks to all concerned, and watch this space next year!

Report on Workshop on PORTABILITY

moderated by Eke van Batenburg & Adrian Smith

General Information

The workshop on portability was held at the APL conference on Wednesday June 7th. There were about 15 attendees, and most of them had experience with portability issues (if not portability problems). This made for an interesting discussion where everybody was very much involved.

The discussions mainly covered the following four topics. Is portability an issue? What can we demand from vendors? What can we do ourselves? Plan of action.

Is Portability an Issue?

As said before, all attendees had some experience with portability problems, but some had more horrible stories to relate to us than others. For example starting with APL on a micro-computer like TRS80, PET or other brands, going to a mainframe and subsequently to another mainframe, and finally to a mini-computer, expecting quite probably to have to migrate to a micro-computer again in the next few years.



Eke van Batenburg

In general, philosophical issues (floating vs grounded arrays) were not a concern, and missing functionality (such as `□DAYS` in APL68000 or the APL*PLUS `FASTFNS` library) was felt to be a nuisance, but one which could be easily covered by global variables or utilities. The real problems were with syntactic incompatibilities (the use of the up-arrow) and subtle variations in behaviour such as the point where integers turn to floats.

Some people were on the move because of the changes in APL support. For example lack of development in STSC's MacAPL or discontinuing support of APL by Dec which forced, or at least

motivated people to switch to another computer. Others had to move because of upsizing (going from 8-bit micro to mini or mainframe) or downsizing (the recent trend to quit the mainframe and go to PCs). One participant had been through 2 enforced migrations (from the MCM micro in 1984 and then from Wicat) when APL vendors went out of business. Another was working on Honeywell APL 66 (one of the first nested APLs) and had only 3 more weeks until the plug was to be pulled on the system!

Another group of people experienced daily portability problems because they have to support an application on different computers. Although the initial start focussed on portability of programs, several participants stressed the importance of porting data, in particular porting data in component files.

What Can we Demand from Vendors?

Of course the easiest solution to portability problems is to demand of vendors that they make their APL identical to that of the others. However, most participants agreed that the variability that gives the problems in porting has a very good side too. APL evolved into the more powerful product that it is today, because vendors introduced new things. On the other hand, it is in every programmer's interest to have a common set of APL functions that is available on every platform. However, it was recognised that vendors have to make a living and one cannot ask them to make many essential changes if this is not, at least to a certain extent, reflected in more revenue..Nevertheless the participants agreed that some things could be asked of vendors:

Consistency within their own product line. People felt that the products of one vendor on different platforms should not be different. For example differences in Dyalog APL on the Unix platform and Dyalog APL on the PC platform, or differences in Manugistics APL on the Macintosh and Manugistics APL on the PC should be remedied.

Consistency with the ISO standard for Extended APL. Many of the raised topics of portability problems were differences that are unequivocally defined in the standard for Extended APL. Discussions about those topics were cut short because we agreed that it is reasonable to ask vendors to be at least compatible with the APL extended standard. Unfortunately, some of the hardest problems to work around (such as the behaviour of Reshape on empties) are "consistent extensions"; it would be helpful if the standard included some guidance on these.

Some Goodies. Although everybody has a long, long wish list of things that he wants his vendor to include in that particular APL, it seemed that some wishes are more universally shared than others. So, apart from the wishes that

were already defined in the APL extended standard, some others that remained and were shared more or less are:

Standardisation of partitioned enclose (=APL2-like) or partition (=APL*PLUS-like). Some people liked the partitioned enclose using cumulative integers to separate, others were afraid that separation of big arrays by integers would result in *WS FULL* and preferred separation using bits. Which alternative is chosen is of less importance than that there is one standard way only.

Take and drop with axis to avoid having to specify the size of axes that are not changed.

Ravel with axis to make a 1-column or a 1-row matrix and for ravel over several dimensions.

Scalar functions with axis.

`Monitor` Function to measure CPU time of each function line. It is not only useful to pinpoint hot spots of CPU time devourers, but also to check if a test set covered all lines.

`IN` and `OUT` for porting. Many vendors like MicroAPL, IBM and Manugistics have already implemented effective `IN/OUT` commands, others like Dyalog only have an obsolete and buggy alternative.

Negative index. Useful for various constructions (the question was raised if it not implemented because of illogical behaviour with `IO=0?`).

Standard position of characters in `AV`.

Standard behaviour of `"A B C"` and `"A B C[...]"` (binding to C or to ABC). Some APLs require parenthesis to assign to `"A B C"`, others do not. Again (as with partitioned enclose) which alternative is chosen is of less importance than that there is one standard way only.

Standard storage (or at least publication of formats) of component files.

Availability of `MONTH` for names of months and `DAY` for days of the week.

Namespaces

What Can we Do Ourselves?

Of course if you think that you will never have a portability problem your need to protect yourself against future problems is small. Nevertheless the effort to protect yourself is sometimes small.

Be wary of non-portable constructions. Sometimes it is very easy to use an alternative programming construction if you know that some features are not universally supported. For example one can get a single value from a non-nested vector using index, using pick or using squeezed quad. If you know that the latter is not implemented in many APL systems, taking one of the

alternatives is no problem at all.

Some participants were rather surprised to learn that some constructions that had easy alternatives were not widely supported, and expressed their intention to change their programming immediately. Apparently it could be useful to publish a list of differences.

Use cover functions. Some actions, especially those regarding input and output, are always different in various APL systems, for example file I/O and graphics, and others will hopefully grow together but are still rather different (like GUI functions). A sensible way to diminish future portability efforts is to hide that machine-specific code in cover functions. In porting, only those cover functions need to be rewritten.

Develop public-domain cover functions. It will be useful if the APL community has some public-domain cover functions. The packages CAUSEWAY for GUI and RAIN for graphics are good examples of this.

Work at the most APL2-like level offered by Manugistics and Dyadic respectively. The bulk of the code in RAIN is now identical in both these environments, having been migrated to *ML 2* in Dyalog and then copied via the Windows clipboard to PLUS III.

Plan of Action

Publish this in QuoteQuad and Vector. Adrian will publish the discussions of this workshop in Vector and Eke will submit a report to QuoteQuad.

Report this to ISO. Eke will report the main topics of the workshop in the ISO meeting.

Develop more public domain software. No action proposed.

Publish a list of deviations. Eke will start a publication of differences between the mainstream vendors and ask Joseph De Kerf to join him if he wants to. Several participants offered to help, so it was proposed to set up preliminary versions on the Internet and ask for help. Eke will do so. The comp.lang.APL forum was suggested as a good repository for this information.

Eke van Batenburg
Leiden, 28th June 1995

Voyages in Dragon Country

by Sylvia Camacho

Marriage to the editor of Vector confers some privileges, one of which is to see the copy before it goes to print. Thus I am able to refer to Gérard Langlet's letter, included in this issue. This continues the attack that Gérard made in 11.2 (*'Chaos Revisited'*) on the uncritical use of computers to run iterative algorithms. Gérard illustrates the problems found with IEEE floating point arithmetic by reference to different formulations of the logistic equation, this being quoted in most works on Chaos as a paradigm of the effects being discussed. However, as Donald McIntyre points out in 11.4 (*'The Perils of Subtraction'*), older and much-loved algorithms such as that of Archimedes for calculating π are also sabotaged by standard computer number representation.

A Storm Brewing

In the current issue Gérard suggests that error checking, as supported by *Mathematica*, should be used and quoted whenever propagation of truncation errors is possible and that failure to do so may invalidate many respectable academic papers. Not unexpectedly, he reports that this suggestion is raising a storm among francophones. While I love a good fight and shall do my best to get to the battleground of this one, my own interest is less technical and mathematical and more philosophical. So in this article I am going to tilt at my own choice of dragon, albeit one from the same lair.

Look who Whistled up the Wind

ZENO of Elea (c. 490–c. 435 B.C.). Greek philosopher and mathematician. Propounded paradoxes to point out the problem of the relation of the discrete to the continuous.....

Zeno's paradox of Achilles and the tortoise.

A tortoise has a head start on Achilles equal to the distance from a to b and both start running, Achilles after the tortoise. Although Achilles runs faster than the tortoise, he would never catch up with the tortoise, since, while Achilles goes from a to b , the tortoise goes from b to c , and while Achilles goes from b to c , the tortoise goes from c to d , etc., this process never ending. The explanation of the fallacy is that motion is measured by space intervals per unit of time, not by numbers of points. If Achilles takes time $t_1, t_2, t_3, \dots$ to go from a to b , b to c , c to d ,

... , then Achilles will catch the tortoise in time equal to $\sum_1^{\infty} t_i$, if this sum is finite.

If the tortoise travels 10 feet per second and Achilles 20 feet and the tortoise starts 10 feet in advance, Achilles will catch him at the end of the first second, since then

$$t_1 = \frac{1}{2}, t_2 = \frac{1}{4}, \dots, t_n = \frac{1}{2^n}, \dots \text{ and } \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = 1.$$

James & James "Mathematics Dictionary" Van Nostrand Reinhold NY ed 5 1992

The Dragon at the Eye of the Storm

Now whenever in a mathematical expression I see ' $\dots = n$ ', the hairs on the back of my neck start to prickle. What, I ask myself, is finite about this procedure? I do not of course quarrel with the suggestion that in the example quoted the limit is 1 but how, in a finite time, could I **compute** this result using this algorithm? Also, in the real world, what counts as Achilles drawing level with the tortoise? Let us imagine the tortoise followed by Achilles striding along an (imaginary) time-line. Has Achilles drawn level when his leading edge touches the trailing edge of the tortoise? We know that in the real world they both have very fuzzy edges, the fuzziness depending on the resolution of our observing equipment.

To many readers all this will read like a philosophic quibble but remember that Chaos gurus aspire to model phenomena which are not amenable to Calculus. They are experts in discontinuity and the destabilising effects of small but precisely expressed differences. They ought therefore to be acutely aware of the minimum difference which is appropriate to the model they are investigating. In response to similar remarks, Frank Evans (long time APL user) said, 'one must always be aware of the fineness of one's grid'. So before accepting that an algorithm is a plausible model of any real-world system one must ask the crucial questions: 'What is the resolution of your lens?', 'What is the wavelength of your light?', 'What is the grain size of your film?', 'What is the pixel count of your screen?', 'What are the limits of accuracy of your computer in relation to the algorithm you are computing?' A statistician would say, 'what is your population?'. So I ask myself 'what do we populate our computers with?' and the answer is 'bits'. So let us declare a maximum population, say p . How might we organise p bits to emulate the logistic equation, as cited by Langlet, $x_{n+1} = 4x_n(1-x_n)$?

We have to set a starting value between zero and p . Let us call this v . So the equivalent of $1-x_n$ is $p-v$. We can imagine the starting value as a set of bits $v \rho 1$. Multiplication is then equivalent to replication $4/(p-v)/v \rho 1$ but this number of bits exceeds the maximum population so we must emulate division in order to

restore the original 'fineness of the grid'. The resolution of the grid is p and so we use this to set up a template to fit the result of the replication:

$$template \leftarrow (+/4 / (p-v) / v \rho 1) \rho \phi p + 1$$

This gives a boolean vector of the same length as the replicated bits, all zeros except for one bit at each interval of p . The number of bits not equal to zero in this template will be the value of the next generation of the iterated equation. The result is always integer and is equivalent to taking the floor of a division by p . To take the ceiling one would reshape $p+1$. To round the result the position of the bit set to 1 would be at the midpoint of $p+0$. The other possible positions for the 1 do not have an obvious interpretation but might be interesting to speculate about.

To many readers this will seem like a convoluted way of doing a bit of simple integer arithmetic but I hope that it serves to illuminate the problem that arises when the logistic equation is iterated using floating point. This is that the 'fineness of the grid' is made finer with every iteration, in a doomed pursuit of precision. This cannot be consonant with the use of $1-x_n$ as an expression intended to impose a limit to population growth. This criticism applies wherever the phenomena for which such an equation might be a model are not continuous and those are the very systems with which Chaos theorists are most concerned.

APL: the Dragon Tamer

What are the characteristics of the logistic equation if it is constrained to work at given resolutions? I have used some APL functions to calculate the results for various maximum populations, p being set to some power of 2. Thus we can compare and contrast the effects of doubling the 'fineness of the grid' on the sets of iterated results. A consequence of limiting the population to p is that there can only be $1/p$ starting values and a consequence of taking the floor of the division is that every result must also be taken from the maximum set of integers $1/p$. The set of all possible result vectors can thus be shown as a $p \times p$ matrix. Three examples are given below.

```

      vtry[[]]v
    v r←try p;[]io
[1]  []io+1
[2]  r←(p,1)ρ1p
[3]  l1:→(p=¯1+ρr)†0
[4]  r←r,[(÷p)×4×r[;¯1+ρr]×p-r[;¯1+ρr]
[5]  →l1
    v

```

try 8

```

1 3 7 3 7 3 7 3
2 6 6 6 6 6 6 6
3 7 3 7 3 7 3 7
4 8 0 0 0 0 0 0
5 7 3 7 3 7 3 7
6 6 6 6 6 6 6 6
7 3 7 3 7 3 7 3
8 0 0 0 0 0 0 0

```

try 16

```

1 3 9 15 3 9 15 3 9 15 3 9 15 3 9 15
2 7 15 3 9 15 3 9 15 3 9 15 3 9 15 3
3 9 15 3 9 15 3 9 15 3 9 15 3 9 15 3
4 12 12 12 12 12 12 12 12 12 12 12 12 12 12
5 13 9 15 3 9 15 3 9 15 3 9 15 3 9 15
6 15 3 9 15 3 9 15 3 9 15 3 9 15 3 9
7 15 3 9 15 3 9 15 3 9 15 3 9 15 3 9
8 16 0 0 0 0 0 0 0 0 0 0 0 0 0 0
9 15 3 9 15 3 9 15 3 9 15 3 9 15 3 9
10 15 3 9 15 3 9 15 3 9 15 3 9 15 3 9
11 13 9 15 3 9 15 3 9 15 3 9 15 3 9 15
12 12 12 12 12 12 12 12 12 12 12 12 12 12 12
13 9 15 3 9 15 3 9 15 3 9 15 3 9 15 3
14 7 15 3 9 15 3 9 15 3 9 15 3 9 15 3
15 3 9 15 3 9 15 3 9 15 3 9 15 3 9 15
16 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

```

32 16+try 32

```

1 3 10 27 16 32 0 0 0 0 0 0 0 0 0 0
2 7 21 28 14 31 3 10 27 16 32 0 0 0 0 0
3 10 27 16 32 0 0 0 0 0 0 0 0 0 0
4 14 31 3 10 27 16 32 0 0 0 0 0 0 0
5 16 32 0 0 0 0 0 0 0 0 0 0 0
6 19 30 7 21 28 14 31 3 10 27 16 32 0 0
7 21 28 14 31 3 10 27 16 32 0 0 0 0
8 24 24 24 24 24 24 24 24 24 24 24 24 24
9 25 21 28 14 31 3 10 27 16 32 0 0 0 0
10 27 16 32 0 0 0 0 0 0 0 0 0 0
11 28 14 31 3 10 27 16 32 0 0 0 0 0
12 30 7 21 28 14 31 3 10 27 16 32 0 0 0
13 30 7 21 28 14 31 3 10 27 16 32 0 0 0
14 31 3 10 27 16 32 0 0 0 0 0 0 0
15 31 3 10 27 16 32 0 0 0 0 0 0 0
16 32 0 0 0 0 0 0 0 0 0 0 0 0
17 31 3 10 27 16 32 0 0 0 0 0 0 0
18 31 3 10 27 16 32 0 0 0 0 0 0 0
19 30 7 21 28 14 31 3 10 27 16 32 0 0 0
20 30 7 21 28 14 31 3 10 27 16 32 0 0 0
21 28 14 31 3 10 27 16 32 0 0 0 0 0
22 27 16 32 0 0 0 0 0 0 0 0 0 0
23 25 21 28 14 31 3 10 27 16 32 0 0 0
24 24 24 24 24 24 24 24 24 24 24 24 24
25 21 28 14 31 3 10 27 16 32 0 0 0 0
26 19 30 7 21 28 14 31 3 10 27 16 32 0 0
27 16 32 0 0 0 0 0 0 0 0 0 0 0
28 14 31 3 10 27 16 32 0 0 0 0 0 0
29 10 27 16 32 0 0 0 0 0 0 0 0 0
30 7 21 28 14 31 3 10 27 16 32 0 0 0
31 3 10 27 16 32 0 0 0 0 0 0 0 0
32 0 0 0 0 0 0 0 0 0 0 0 0 0 0

```

As seen the set of result vectors for $1p$ starting values falls into three categories:

1. The iteration reaches the value p . All values after that are zero.
2. The iteration reaches some other value which stays the same for every subsequent iteration.
3. The iteration reaches one of a small subset of values and subsequently cycles through that set. Different starting values may enter the same cycle but at different points within it. This could be expressed as 'having the same periodicity but a different phase'. Other starting values may enter a different cycle constituting a different periodicity, again with phase variation. Different cycles, of course, have no values in common.

I have explored populations up to 2 power 12 and am intrigued to find that the mix of categories 1,2,3 above is very variable and that there are many more values which appear only once in an iteration than the small subset which participate in the cycles and that the same cycle is common to many starting values.

As larger tables are unwieldy and take too long to calculate in full, I have provided a function to analyse the distribution of values, eliminating rows which lead only to zero or to a steady state and terminating the calculation when all results are merely permutations of the same small subset of numbers. To facilitate printing I show only the first column and the last 10 or 16 columns of the larger matrices.

```

vlimited[[]]v
v r+limited p;[]io;v1;v2
[1]  analyse results of each iteration &
[2]  stop when only recycling a subset of 1p
[3]  []io+1
[4]  a set r to all possible initial values
[5]  r+(p,1)p v1+1p
[6]  a v2 is next generation
[7]  l1:v2+[(4xv1xp-v1)÷p
[8]  a exclude rows where values will never again change
[9]  v2+(b+~(v1=v2)v0=v2)/v2
[10] r+(b/r),v2
[11] a keep only first occurrence of each value
[12] v1+(b+(v2,v2)=1pv2)/v2
[13] r+b/r
[14] +((pv1)=pv2)/0
[15] +l1
v

```

limited 64

```
1 3 11 36 63 3 11 36 63 3
3 11 36 63 3 11 36 63 3 11
4 15 45 53 36 63 3 11 36 63
8 28 63 3 11 36 63 3 11 36
```

limited 128

```
1 3 11 40 110 61 127 3 11 40 110
2 7 26 82 117 40 110 61 127 3 11
3 11 40 110 61 127 3 11 40 110 61
7 26 82 117 40 110 61 127 3 11 40
8 30 91 105 75 124 15 52 123 19 64
9 33 97 93 101 85 114 49 120 30 91
10 36 103 80 120 30 91 105 75 124 15
11 40 110 61 127 3 11 40 110 61 127
14 49 120 30 91 105 75 124 15 52 123
24 78 121 26 82 117 40 110 61 127 3
30 91 105 75 124 15 52 123 19 64 128
31 93 101 85 114 49 120 30 91 105 75
33 97 93 101 85 114 49 120 30 91 105
35 101 85 114 49 120 30 91 105 75 124
36 103 80 120 30 91 105 75 124 15 52
48 120 30 91 105 75 124 15 52 123 19
```

matrix+limited 256

pmatrix

4 28

matrix[:,1,12+16]

```
1 140 253 11 42 140 253 11 42 140 253 11 42 140 253 11 42
2 241 56 175 221 120 255 3 11 42 140 253 11 42 140 253 11
3 253 11 42 140 253 11 42 140 253 11 42 140 253 11 42 140
4 11 42 140 253 11 42 140 253 11 42 140 253 11 42 140 253
```

matrix+limited 512

pmatrix

14 29

matrix[:,1,13+16]

```
1 261 511 3 11 43 157 435 261 511 3 11 43 157 435 261 511
2 211 496 62 217 500 46 167 450 217 500 46 167 450 217 500 46
3 511 3 11 43 157 435 261 511 3 11 43 157 435 261 511 3
4 144 414 316 483 109 343 452 211 496 62 217 500 46 167 450 217
5 11 43 157 435 261 511 3 11 43 157 435 261 511 3 11 43
6 157 435 261 511 3 11 43 157 435 261 511 3 11 43 157 435
7 496 62 217 500 46 167 450 217 500 46 167 450 217 500 46 167
8 3 11 43 157 435 261 511 3 11 43 157 435 261 511 3 11
10 500 46 167 450 217 500 46 167 450 217 500 46 167 450 217 500
12 450 217 500 46 167 450 217 500 46 167 450 217 500 46 167 450
17 435 261 511 3 11 43 157 435 261 511 3 11 43 157 435 261
19 43 157 435 261 511 3 11 43 157 435 261 511 3 11 43 157
49 177 463 177 463 177 463 177 463 177 463 177 463 177 463
177 463 177 463 177 463 177 463 177 463 177 463 177 463 177
```

```

matrix+limited 4096
pmatrix
20 117
      matrix[:,101:116]
1 3301 2562 3837 970 2961 3281 2611 3786 1146 3301 2562 3837 970 2961 3281 2611
2 3786 1146 3301 2562 3837 970 2961 3281 2611 3786 1146 3301 2562 3837 970 2961
3 2562 3837 970 2961 3281 2611 3786 1146 3301 2562 3837 970 2961 3281 2611 3786
4 1146 3301 2562 3837 970 2961 3281 2611 3786 1146 3301 2562 3837 970 2961 3281
5 2961 3281 2611 3786 1146 3301 2562 3837 970 2961 3281 2611 3786 1146 3301 2562
9 3837 970 2961 3281 2611 3786 1146 3301 2562 3837 970 2961 3281 2611 3786 1146
14 3767 1210 3410 2284 4041 217 822 2628 3767 1210 3410 2284 4041 217 822 2628
19 3281 2611 3786 1146 3301 2562 3837 970 2961 3281 2611 3786 1146 3301 2562 3837
25 970 2961 3281 2611 3786 1146 3301 2562 3837 970 2961 3281 2611 3786 1146 3301
29 2611 3786 1146 3301 2562 3837 970 2961 3281 2611 3786 1146 3301 2562 3837 970
55 1210 3410 2284 4041 217 822 2628 3767 1210 3410 2284 4041 217 822 2628 3767
84 4041 217 822 2628 3767 1210 3410 2284 4041 217 822 2628 3767 1210 3410 2284
145 217 822 2628 3767 1210 3410 2284 4041 217 822 2628 3767 1210 3410 2284 4041
203 2503 3893 771 2503 3893 771 2503 3893 771 2503 3893 771 2503 3893 771 2503
217 3410 2284 4041 217 822 2628 3767 1210 3410 2284 4041 217 822 2628 3767 1210
519 2628 3767 1210 3410 2284 4041 217 822 2628 3767 1210 3410 2284 4041 217 822
559 822 2628 3767 1210 3410 2284 4041 217 822 2628 3767 1210 3410 2284 4041 217
771 3893 771 2503 3893 771 2503 3893 771 2503 3893 771 2503 3893 771 2503 3893
822 2284 4041 217 822 2628 3767 1210 3410 2284 4041 217 822 2628 3767 1210 3410
1593 771 2503 3893 771 2503 3893 771 2503 3893 771 2503 3893 771 2503 3893 771

```

I hope that this analysis demonstrates why an iteration using floating point calculations will seem chaotic because the equation is applied to successively finer breakdowns of the grid but that where the grid is fixed (however large p may be) most starting values will lead in not too many iterations to a few cycles with periods short by comparison with ρ/p . From these experiments we might come to expect that different populations will have different propensities to reach a small subset of values and that any assumption of equiprobability of outcomes would be grossly mistaken. Some values can only be a starting value as they can never be reached from any other value.

Teaching with Executable Notation

Chapter 2: Linear Functions

by Kenneth E. Iverson

A. Introduction

A *linear* function is a monad that distributes over addition. Every *scalar* linear function is equivalent to multiplication by a constant, but *vector* and higher rank linear functions prove fruitful and interesting. We begin with examples of rank 1 linear functions; before reading them, the reader might furnish examples of his own:

```

a=. 2 3 5 7 [ b=. 0 1 2 3
f=. |.
(f(a+b));(f a);(f b);((f a)+(f b))


|    |   |   |   |   |   |   |   |   |   |   |   |    |   |   |   |
|----|---|---|---|---|---|---|---|---|---|---|---|----|---|---|---|
| 10 | 7 | 4 | 2 | 7 | 5 | 3 | 2 | 3 | 2 | 1 | 0 | 10 | 7 | 4 | 2 |
|----|---|---|---|---|---|---|---|---|---|---|---|----|---|---|---|


f=. +/
(f(a+b));(f a);(f b);((f a)+(f b))


|    |    |   |    |
|----|----|---|----|
| 23 | 17 | 6 | 23 |
|----|----|---|----|


f=. +/\ NB. Prefix scan
(f(a+b));(f a);(f b);((f a)+(f b))


|   |   |    |    |   |   |    |    |   |   |   |   |   |   |    |    |
|---|---|----|----|---|---|----|----|---|---|---|---|---|---|----|----|
| 2 | 6 | 13 | 23 | 2 | 5 | 10 | 17 | 0 | 1 | 3 | 6 | 2 | 6 | 13 | 23 |
|---|---|----|----|---|---|----|----|---|---|---|---|---|---|----|----|


f=. 1 2 3 4%*
(f(a+b));(f a);(f b);((f a)+(f b))


|   |   |    |    |   |   |    |    |   |   |   |    |   |   |    |    |
|---|---|----|----|---|---|----|----|---|---|---|----|---|---|----|----|
| 2 | 8 | 21 | 40 | 2 | 6 | 15 | 28 | 0 | 2 | 6 | 12 | 2 | 8 | 21 | 40 |
|---|---|----|----|---|---|----|----|---|---|---|----|---|---|----|----|


f=. +/@(1 2 3 4%*)
(f(a+b));(f a);(f b);((f a)+(f b))


|    |    |    |    |
|----|----|----|----|
| 71 | 51 | 20 | 71 |
|----|----|----|----|


f=. +/\@(1 2 3 4%*)
(f(a+b));(f a);(f b);((f a)+(f b))


|   |    |    |    |   |   |    |    |   |   |   |    |   |    |    |    |
|---|----|----|----|---|---|----|----|---|---|---|----|---|----|----|----|
| 2 | 10 | 31 | 71 | 2 | 8 | 23 | 51 | 0 | 2 | 8 | 20 | 2 | 10 | 31 | 71 |
|---|----|----|----|---|---|----|----|---|---|---|----|---|----|----|----|


```

- A.1 Comment on the foregoing sentences after experimenting with arguments (such as $a = 2 \ 3 \ 5 \ 7 \ 11$ and $b = 0 \ 1 \ 2 \ 3 \ 4$) of a different dimension (number of elements) from the arguments used above,
- A.2 Define the matrix $m = 1 \ 3 \ 4$ and determine the dimensions of the argument a and result r of the following expression:

```
+/ "1 m * "1 a
```

A.3 Experiment with the function `mp = . +/ . *` (matrix product) in expressions such as:

```
m mp a=. 2 3 5 7
f=. m&mp
f a
f a+b=. 4 3 2 1
(f a)+(f b)
```

Comment on the results.

As remarked in the answer to exercise A.1, some linear vector functions are limited to arguments of a fixed dimension, and we will concentrate on such functions. More particularly, we will emphasize uniform functions that produce *results* of the same dimension as the argument, and are "closed" in the sense that the result of one is a suitable argument to another.

As illustrated in Exercises A.2 and A.3, the matrix product `mp = . +/ . *` may be used with a matrix `m` in the expression `mp&m` to yield a linear function. Conversely, *any* linear function of fixed dimension can be represented in this form. The matrix required to represent a specific linear function `f` is obtained by applying it to an identity matrix as illustrated below:

```
f=. |. @ (+/\) @ |. "1
b=. 1 2 3 4 5
I=. = 1. #b
1 0 0 0 0
0 1 0 0 0
0 0 1 0 0
0 0 0 1 0
0 0 0 0 1
f b
15 14 12 9 5
] m=. f I
1 0 0 0 0
1 1 0 0 0
1 1 1 0 0
1 1 1 1 0
1 1 1 1 1
mp&m b
15 14 12 9 5
```

- A.4 What restriction on `m` ensures that the function `mp&m` is uniform?
- A.5 Prove that the function `mp&m` is linear, that is, that it distributes over addition.
- A.6 Why is a matrix such as `I = . = 1. n` called an *identity* matrix?
- A.7 Prove that `m = . f I` yields a matrix such that `mp&m` is equivalent to a linear function `f`.
- A.8 If the functions `mp&m` and `n&mp` are equivalent, what is the relation between the matrices `m` and `n`? Does the relation hold for non-uniform functions?

B. Representations and Compositions

Since 2 $\#$. 1 1 0 0 is twelve, the list 1 1 0 0 is said to be its *base-2 representation*. More generally, a list of integers $\mathbf{l}$ is said to be the base- b representation of the number $b \#$. $\mathbf{l}$.

Similarly, the coefficients $\mathbf{c}$ are said to be the *polynomial* representation of the function $\mathbf{c\&p.}$, and $\mathbf{m}$ is said to be the (matrix) representation of the linear function $\mathbf{mp\&m.}$ More specifically, we may call $\mathbf{m}$ the *post*-representation to distinguish it from the *pre*-representation in the form $\mathbf{n\&mp.}$

B.1 Use various integers b and integer lists $\mathbf{c}$ to experiment with the functions $\mathbf{b\&\#.}$, and $\mathbf{c\&p.}$

B.2 Test the assertion that the functions $\mathbf{+/\}$ and $\mathbf{+/\}.}$ are linear, and write and enter expressions to yield their dimension-4 representations.

In maths, notation such as $\mathbf{f \circ g}$ is used to denote the *composition* of functions $\mathbf{f}$ and $\mathbf{g}$, with the definition that $\mathbf{f \circ g\ x}$ is $\mathbf{f\ g\ x}$. We will use the term more generally to refer to the use of several conjunctions (including $\mathbf{@}$ and $\mathbf{\&}$ and $\mathbf{\&.}$) and of forks. For example, if $\mathbf{f}$ and $\mathbf{g}$ are linear functions, then $\mathbf{f + g}$ is also a linear function, and its representation is the sum of the representations of $\mathbf{f}$ and $\mathbf{g}$.

B.3 Enter $\mathbf{m=. ? 4 4 \$9 [n=. ? 4 4 \$9}$ and $\mathbf{f=. mp\&m}$ and $\mathbf{g=. mp\&n}$ to generate two random matrices of dimension 4, and experiment with the uniform linear functions $\mathbf{f}$ and $\mathbf{g}$ represented by them.

B.4 Assuming $\mathbf{f}$ and $\mathbf{g}$ linear, state which of the following compositions are linear, and write expressions for their representations in terms of the representations of $\mathbf{f}$ and $\mathbf{g}$:

$\mathbf{f@g \quad f+g \quad f-g \quad f\&g \quad f\%g}$

B.5 Experiment with the verbs and pronouns of Ex B.3 to test your conclusions in Ex B.4.

B.6 Is the alternating sum $\mathbf{-/}$ linear?

C. Inverses

The functions $\mathbf{+ :}$ and $\mathbf{- :}$ are said to be (mutually) inverse because $\mathbf{+ :@- :}$ and $\mathbf{- :@+ :}$ are identity functions. For example:

```

a=. i. 4
(+:a);(-:a);(+:-:a);(-:+:a)


|         |             |         |         |
|---------|-------------|---------|---------|
| 0 2 4 6 | 0 0.5 1 1.5 | 0 1 2 3 | 0 1 2 3 |
|---------|-------------|---------|---------|


(*:a);(%:a);(*:%:a);(%:~:a)


|         |                     |         |         |
|---------|---------------------|---------|---------|
| 0 1 4 9 | 0 1 1.41421 1.73205 | 0 1 2 3 | 0 1 2 3 |
|---------|---------------------|---------|---------|


(^a);(^.a)


|                           |                        |
|---------------------------|------------------------|
| 1 2.71828 7.38906 20.0855 | ___ 0 0.693147 1.09861 |
|---------------------------|------------------------|


(^.^a);(^.^a)


|          |         |
|----------|---------|
| _. 1 2 3 | 0 1 2 3 |
|----------|---------|


```

- C.1 Experiment with the foregoing sentences using $a = -.1.4$ and comment on the results.
- C.2 Experiment with the expressions $f^{\wedge} _1$ and $f^{\wedge} _1 a$ for each of the functions used in the foregoing sentences, and state the effect of the power conjunction with the right argument $_1$ (or, equivalently, the effect of the adverb $inv = .^{\wedge} _1$).

A function that yields the same result for two different arguments cannot be invertible. For example:

```

0: 3          0: 4
0              0
]m=. 1 2 3,0,:4 5 6
1 2 3
0 0 0
4 5 6
mp=. +/ . *
mp&m 2 3 5
22 29 36
mp&m 2 7 5
22 29 36

```

Subsequent remarks are intended to apply only to invertible functions. The inverse of a linear function is also linear. For example:

```

f=. +/\ "1
g=. f^:_1 "1
a=. 4 3 2 1
b=. 2 3 5 7
(f a);(g a);(f g a);(g a+b);((g a)+(g b))


|          |            |         |         |         |
|----------|------------|---------|---------|---------|
| 4 7 9 10 | 4 _1 _1 _1 | 4 3 2 1 | 6 0 1 1 | 6 0 1 1 |
|----------|------------|---------|---------|---------|


I=. =i. 5
I;(f I);(g I);(% f I)


|           |           |            |            |
|-----------|-----------|------------|------------|
| 1 0 0 0 0 | 1 1 1 1 1 | 1 _1 0 0 0 | 1 _1 0 0 0 |
| 0 1 0 0 0 | 0 1 1 1 1 | 0 1 _1 0 0 | 0 1 _1 0 0 |
| 0 0 1 0 0 | 0 0 1 1 1 | 0 0 1 _1 0 | 0 0 1 _1 0 |
| 0 0 0 1 0 | 0 0 0 1 1 | 0 0 0 1 _1 | 0 0 0 1 _1 |
| 0 0 0 0 1 | 0 0 0 0 1 | 0 0 0 0 1  | 0 0 0 0 1  |


```

The representations of f and g given in the final result show the relation between "integration" (f) and "differencing" (g), and the result of the function $\%$ suggests why it is called *matrix inverse*.

C.3 Prove that the inverse of a linear function is a linear function.

D. Scans

In Section A it was seen that the prefix scan adverb $\backslash$ applied to the linear function $+/$ yielded a linear function. This suggests a further examination of it and other scan adverbs listed in the Vocabulary of the dictionary of J, including the *suffix scan* ($\backslash.$), the *infix* and *outfix* (dyadic cases of $\backslash$ and $\backslash.$), and the *oblique* ($/.$) which is of interest for linear functions of rank 2.

Make your own examples to test the linearity of the various scans, and then consider the following:

```
a=. 2 3 5 7 11 [ b=. 0 1 2 3 4
+/\a
<\a NB. Structure of prefix scan
2 2 3 2 3 5 2 3 5 7 2 3 5 7 11
<\.a NB. Structure of suffix scan
2 3 5 7 11 3 5 7 11 5 7 11 7 11 11
(+/\.a);(+/\.b)
28 26 23 18 11 10 10 9 7 4
((+/\.a)+( +/\.b));(+/\.a+b)
38 36 32 25 15 38 36 32 25 15
2 <\a NB. Structure of infix
2 3 3 5 5 7 7 11
((2 +/\a)+(2 +/\b));(2 +/\a+b)
6 11 17 25 6 11 17 25
1 <\.a NB. Structure of outfix
3 5 7 11 2 5 7 11 2 3 7 11 2 3 5 11 2 3 5 7
((1 +/\.a)+(1 +/\.b));(1 +/\.a+b)
36 34 31 28 23 36 34 31 28 23
m;n=.8-2*m=.1. 3 3
0 1 2 8 6 4
3 4 5 2 0 _2
6 7 8 _4 _6 _8
```

```
</.m NB. Structure of oblique
```

0	1	3	2	4	6	5	7	8
---	---	---	---	---	---	---	---	---

```
((+//.m)+(+//.n));(+//.m+n)
```

8	12	12	4	0	8	12	12	4	0
---	----	----	---	---	---	----	----	---	---

The outfix can be used to produce the minors of a matrix argument as shown below:

```
$ 1 [\ . m
3 2 3
```

```
box=. <"2 NB. Box tables (2-cells)
```

```
box 1 [\ . m
```

3	4	5	0	1	2	0	1	2
6	7	8	6	7	8	3	4	5

```
f=. 1&{[:\.) NB. Transposed outfixes
```

```
box f m
```

3	6	0	6	0	3
4	7	1	7	1	4
5	8	2	8	2	5

```
minors=.f"2^:2 NB. f0n tables (twice)
```

```
$ minors m
```

```
3 3 2 2
```

```
box minors m NB. All minors of m
```

4	5	3	5	3	4
7	8	6	8	6	7
1	2	0	2	0	1
7	8	6	8	6	7
1	2	0	2	0	1
4	5	3	5	3	4

These minors of a matrix can be used to determine its *adjoint*. Moreover, the function minors is linear, as will be illustrated:

```
min=. minors
```

```
(box(min m)+(min n));<box min m+n
```

4	3	5	3	5	4
1	0	2	0	2	1
7	6	8	6	8	7
1	0	2	0	2	1
7	6	8	6	8	7
4	3	5	3	5	4

4	3	5	3	5	4
1	0	2	0	2	1
7	6	8	6	8	7
1	0	2	0	2	1
7	6	8	6	8	7
4	3	5	3	5	4

References

- [1] K. E. Iverson, *J Introduction and Dictionary*
 [2] K. E. Iverson, *Arithmetic*

Answers to Chapter 2

A. Introduction

- A.1 The first three examples of linear functions apply to vector arguments of any dimension. The rest are restricted to a fixed dimension (4). We are commonly concerned only with a fixed dimension, as in plane geometry (2), solid geometry (3), and space-time (4).

The dimension of the result of a function may differ from the dimension of the argument (as in $+/@(1\ 2\ 3\ 4&*)$ "1 and $+/"\ 1$).

- A.2 The shape of m (that is, $\$m$) is $3\ 4$; the dimension of the result is 3, and the dimension of the argument is 4.

- A.3 Note that the space before the dot in the expression $mp = . +/ . *$ is essential (since $/ .$ would otherwise denote an adverb different from $/$). Also note that the final expressions are tests of linearity, and that the dot conjunction provides for other functions that are similar to the matrix product, but are based upon functions other than summation and multiplication.

- A.4 The matrix m must be square, that is, $=/\$ m$.

- A.6 Because the function $mp\&m$ is an identity.

- A.5 One is the transpose of the other ($n - : | : m$). This is true even for non-uniform functions.

- B.4 If $f m = . f\ I\ [gm = . g\ I = . = 1 .\ 4$, then $f m$ and $g m$ are the representations of f and g . The following are the representations of $f@g$ and $f+g$ and $f-g$, respectively:

$f m\ mp\ gm$
 $f m + gm$
 $f m - gm$

Multiprecision Arithmetic: Part I

by John Sullivan

Introduction

When I was a lot younger I became interested in prime numbers. Not, you will understand, run-of-the-mill primes like 2, 3, 5, 7 or even 2357, but decent primes that you can get your teeth into, such as $391581 \times 2^{216193} - 1$, which was discovered in August 1989[1]. Later, my interest expanded to the repunits (numbers consisting entirely of 1s) and their factors. These numbers are of the form $(10^n - 1)/9$, which is similar to the form of the Mersenne numbers, $(2^n - 1)/1$, which can be considered to be repunits in radix 2. It was but a short step from these to what I have the temerity to call the *generalized repunit* which has the form $(a^n - b^n)/(a - b)$. In order to study the factors of these numbers I needed some way to perform arithmetic operations on large numbers quickly, as pencil and paper long division soon became tiresome, and my pocket calculator could only handle small numbers.

I hate programming. I do it because I have to (it provides the daily crust), but like most lazy people I do as little as necessary to do the job in hand. The wonderful thing about APL is that you do not have to do all those nasty things that are necessary with Fortran or C, like writing nice tidy routines to store your data in a file where you can get at it again for future use. In APL you can store all of your junk in the workspace. You can key in a long line of instructions at the six-blank prompt and let APL do the parsing for you, leaving you to write the underlying functions. What I am trying to say is I don't like writing things in Fortran or C, where you have to finish the whole shebang before you can start testing it, whereas with APL you can write bits now and add more later, with very little extra effort at the start, because the APL workspace concept handles the fiddly bits for you.

So, when I came to start writing my multiprecision suite, the obvious choice of language was APL. To be sure, I'll recode it in C when it's eventually finished, except that by that time I'll probably be too busy with other things, like playing the harp and singing in the choir.

Laziness has also meant that I have drawn on several source materials in writing this suite. Firstly, I suppose, I got the idea from IBM, who published their own version of a multiprecision suite in 1977[2]. Pretty nice it looked, until you tried to do anything with it, and then you discovered the bugs! There is a whole section

on multiple-precision arithmetic in Donald Knuth's *Art of Computer Programming*[3], which explains the algorithms very well. Hans Riesel[4] has "A Complete Package for Multiple-Precision Arithmetic" written in Pascal, which unfortunately contains a number of errors, but gives a lot of good algorithms for primality testing and factorization, as does Paulo Ribenboim[5]. Finally, a good source of information on repunits is Samuel Yates[6].

In this document the following abbreviations and terms are used:

Maxint.....1+the maximum integer value the computer will handle. For 4-byte integers this is 2147483648.

scalarA number which is represented by an ordinary integer, as opposed to a multiprecision number. This is allowed by all the functions as long as it is not too big.

Preliminaries

Multiprecision numbers can be considered to be objects in a class, so it seemed natural to make use of the Namespace facility in Dyalog APL, where the gory details of some of the functions could be hidden from immediate view. This also allows me to set the index-origin in the namespace to zero, and allow the user to have any other index origin in the root namespace. The multiprecision namespace is called *mp* and is set up as follows:

```

)ns mp
#.mp
)cs mp
#.mp
⎕io←0
```

Of course, you don't have to do this if you don't want to. Although these functions are written in Dyalog APL, I have tried to write them in such a way that they will work with any APL, although you may need to make some minor changes, especially with `⎕SIGNAL`.

Internal Representation of a Number

Each number is represented as a vector of numbers, preceded by a number, called the *exponent*, which defines the position of the radix-point with respect to the right-hand end of the number. A zero exponent indicates an integer, a positive exponent implies that number of zeros catenated at the end of the number, and a negative exponent implies that the radix point is moved to the left that number of places. It's a bit like E-notation (e.g. 1E4 is really 10000).

All non-exponent items of the number have the same sign. Leading zeros are eliminated.

The Radix

Normal arithmetic performed mentally, or with a pencil and paper, is done in radix 10. In a computer environment, however, radix-10 arithmetic is somewhat inefficient, and it is possible to do much better. What are the considerations for determining the optimal radix for the multiprecision suite?

The radix must be a positive integer. It is too complicated to use negative, imaginary or complex radices.

For addition the radix must be less than or equal to half of *Maxint* and for multiplication it must be less than or equal to the square root of *Maxint*, in order to avoid conversions to floating point numbers in the calculations.

The radix for input and output of numbers must be a power of 10. However, the exponentiation algorithm requires the binary representation of the exponent, and so a radix change to a power of 2 has to be made at some time during exponentiation. In all other cases the radix is immaterial, since the functions will work with any radix, but a power of 10 allows easy inspection of intermediate results when debugging.

When deciding which radix to use a balance is struck between the extra processing required and the benefits. For example, with a large radix (greater than the square root of *Maxint*) a radix-change to a smaller one is made when multiplying numbers, but the numbers will be stored in smaller-sized arrays, requiring less storage and less processing in the addition and division functions. With a small radix there is no need to change radix when multiplying, but numbers require more storage and more processing for addition and division.

In this suite there are two radix variables, which are used as follows. The main radix variable is called *base* and it must be less than *Maxint* and it must be a square if it is not equal to the other variable. The other variable is called *bsqr*, and it is either the square root of *base* (in which case a radix change is made in the multiplication routine) or it is equal to *base* (so a radix change is not made in the multiplication function). In all cases *bsqr* must be less than the square root of *Maxint*. It is an error if *base* and *bsqr* are not as described here, and the results of your processing will be unpredictable.

In the examples provided in this series of articles, unless stated to the contrary the variable *base* is set to 10^8 and *bsqr* is 10^4 .

The function to change the radix of a number is as follows. The left argument is a 2-element vector containing the new radix and the existing radix in that order. The right argument is the number in the existing radix and the result is the number in the new radix. Obviously (line 2) we do nothing if the radix does not change.

```

      v z+b chbase z;base;bsqr
[1]   'Invalid radix specification'[] SIGNAL(2≠b)/8
[2]   +(-/b)/0

```

If the new base is less than 32768 then we can make *base* and *bsqr* the same

```

[3]   +(b[0]≥32768)/Δ1
[4]   bsqr←base+⌊b[0]
[5]   →Δ2

```

Otherwise the new radix must be a square. In the process we generate the square root of the radix, and make absolutely sure that this square root is stored as an integer

```

[6]   Δ1: 'Radix not a square'[] SIGNAL(0≠1|bsqr←(base+b[0]) *0.5)/8
[7]   bsqr←⌊bsqr

```

Having made certain our radix variables are correct, we now change the radix of the number.

```

[8]   Δ2: z+b[1]frombase z
      v

```

Radix conversion is done by a separate function. If the radix is unchanged, we exit with no change to the number.

```

      v z+b frombase y;a;f;q
[1]   +(b≠base)/Δ1
[2]   z←y 0 →0

```

Set the result variable to zero, then express the old radix in terms of the new radix

```

[3]   Δ1: z+0 0
[4]   +(base≥b)/Δ2
[5]   b←(0,base)↑b
[6]   Δ2: b+0,b

```

If we have a floating-point number we will not always be able to give an exact representation in the new radix. However, we can always represent integers exactly in the new radix. Therefore we multiply appropriately in order to make our number an integer, carefully noting the amount we multiplied by (stored in f as a power of the old radix):

```
[7]      f←0{ -y[0] ◊ y[0]+0[y[0]
```

Pad the number with trailing blanks if necessary, and drop the radix-point indicator:

```
[8]      y←1+fullint y
```

If the next digit is greater than the new radix then split it (equivalent to $45_{100} = 4_{10}5_{10}$):

```
[9]      Δ4:→(basezq+y[0])/Δ5
```

```
[10]     q←,(0,base)τq
```

Multiply the existing result by the radix and add the new digit. If we have any more left then repeat the process:

```
[11]     Δ5:z←(0,q)Fadd b Fmul z
```

```
[12]     →(0≠py←1+y)/Δ4
```

If we started with a fraction then we must return a fraction, so divide by the appropriate power of the original radix. Notice that we may lose data here.

```
[13]     →(f=0)/0
```

```
[14]     z←z Fdiv b Fspow f
```

▽

The function *fullint* converts integers which have had their trailing zeros removed to "full" integers including their trailing zeros. It is also used in other functions, such as the division function.

```
▽ z←fullint z
```

```
[1]      →(0>1+z)/0
```

```
[3]      z←0,(1+z),(1+z)ρ0
```

▽

The Basic Functions

The basic functions are Add, Subtract, Multiply, Divide and Scalar power. There are two functions for Divide: one for an integer result with remainder, and one for a floating-point result which is rounded to the appropriate precision. All function names start with F if they can be used for floating-point operations, and I if they are for integers only.

Before describing the functions I should say that everywhere a multiprecision number is acceptable so also is a scalar. However, we must first convert these scalars to multiprecision numbers, and the following function does just that.

```

      v a+scalar a
[1]   a+((1=pa)/0),a+,a
      v

```

Add

The Add function is simple:

```

      v z+a Fadd b;d;m;s
[1]   a+scalar a o b+scalar b

```

Make the two numbers align on the right by adding trailing zeros if necessary. Save the position of the radix point, which is the lower of the two original positions. We can then drop the radix point position, and add leading zeros to make both numbers the same length (implied only, by variable *m*):

```

[2]   a+a,(0[a[0]-b[0]]p0
[3]   b+b,(0[b[0]-a[0]]p0
[4]   d+a[0]b[0]
[5]   a+1+a o b+1+b o m+-(pa)[pb

```

Add the two numbers together, making a note of the sign of the most significant part of the result. If the result is 0 then exit here.

```

[6]   + (0+s+*1+(z≠0)/z+(m+a)+m+b)/Δ1
[7]   z+0 0 o +0

```

Tidy up the result. Carrying is dealt with in lines 9 and 10, leading zeros are dropped in 11, and all is put together in 12.

```

[8]  Δ1:Z+S×Z
[9]  Δ2:→(0^.= (Z+(0,base)rz)[0;])/Δ3
[10] Z←(Z[0;],0)+0,Z[1;] ◊ →Δ2
[11] Δ3:Z+S×Z[1;] ◊ Z+((Z≠0)1)+Z
[12] Z←(d+m),(-m+(0≠φZ)1)+Z
      v

```

Subtract

This is even easier. We just multiply the right argument by -1 and call *Fadd*.

```

      v z+a Fsub b
[1]  z+a Fadd(1+b),-1+b+scalar b
      v

```

There are two boolean comparison functions that make use of *Fsub*. *Fequal* is the multiprecision equivalent of =

```

      v z+x Fequal y
[1]  z+0^.=x Fsub y
      v

```

Fgt is the multiprecision ordering function. We only need one function, because, given variables X and Y:

```

For X>Y use X Fgt Y
For X<Y use Y Fgt X
For X≤Y use ~X Fgt Y
For X≥Y use ~Y Fgt X

```

```

      v z+x Fgt y
[1]  z+0^.>1+y Fsub x
      v

```

Multiply

Start by storing the number of radix places and the signs of the arguments, then calculate the sign of the result, exiting quickly if the result is zero.

```

      v z+a Fmul b;da;db;noconv;s;sa;sb
[1]  a←scalar a ◊ b←scalar b
[2]  da←a[0] ◊ db←b[0]
[3]  sa←1+a+((a≠0)1)+a+1+a
[4]  sb←1+b+((b≠0)1)+b+1+b
[5]  →(0≠s+sa×sb)/Δ1
[6]  z←0 0 ◊ →0

```

We work on positive numbers only. Because of the way the algorithm is written it is better that *a* should be the larger of the two numbers, so swap them if necessary.

```
[7]   Δ1: a ← |a| a > b ← |b|
[8]   → ((ρb) ≥ ρa) / Δ2
[9]   a b ← b a
```

Because of the danger of overflowing and converting into floating-point numbers the arguments are converted to a smaller radix here (unless, of course, we have specifically requested the function not to do so by making the two radix variables equal). We use variable *noconv* later when we want to convert back again.

```
[10]  Δ2: → (noconv + base = bsqr) / Δ3
[11]  a ←, &(0, bsqr) τ a
[12]  b ←, &(0, bsqr) τ b
```

Calculate the raw result, and refine it by carrying. We cannot avoid a loop here: in radix 10, if we add 1 to 499999 we have 5 carries, one on each iteration of the loop. The same applies here.

```
[13]  Δ3: z ← + / (-1 + 1 + ρz) φ z, (2ρ1 + ρz + a * b) ρ0 a Raw result
[14]  Δ4: → ((a + (0, bsqr) τ z) [0;] ^ . = 0) / Δ5
[15]  z ← (a [0;], 0) + 0, a [1;]
[16]  → Δ4
```

Convert back from the smaller to the larger radix if necessary:

```
[17]  Δ5: → noconv / Δ6
[18]  z ← ((2 | ρz) ρ0), z
[19]  z ← (0, bsqr) ⊥ & ((0.5 * ρz), 2) ρz
```

Apply the sign to the result, drop leading zeros, set the number of radix places, drop trailing zeros and that's it.

```
[20]  Δ6: z ← s * ((z ≠ 0) | 1) + z
[21]  da ← da + db
[22]  z ← (-db + (0 ≠ φz) | 1) + z
[23]  z ← (da + db), z
```

▽

Divide

This function is complicated because of the way long division is performed. The result of the integer division function is a nested vector of two numbers, the

quotient and the remainder. The floating-point version rounds the result to the appropriate precision. The integer division function gives a domain error if floating-point input is supplied.

First the integer division function. It is a domain error if either the divisor or dividend is non-integer, or if the divisor is zero. If the dividend is zero we can quit early and avoid some processing.

```

v c+a Idiv b;af;bf;j;q;qf;q1;r;r1;s;t
[1]  a+scalar a o b+scalar b
[2]  [] SIGNAL(0v.>a[0],b[0])/11
[3]  [] SIGNAL(0^.=1+b)/11
[4]  +(0v.^1+a)/Δ1
[5]  c+(0 0)(0 0) o →0

```

Calculate the sign of the result, and work from now on in positive numbers. Make sure we have trailing zeros if necessary:

```

[6]  Δ1:s+1 ^1[(av.<0)*bv.<0]
[7]  a+fullint|a o b+fullint|b
[8]  +(2*pb)/Δ3

```

There is no point in using the full algorithm if the divisor is small (i.e. less than the radix), so we divide into two parts here. This is the small divisor part:

```

[9]  →c+(2*(b+1+b)=1+r+0,1+a)pb
[10] Δ2:c+c,q+[(t+base|r,1+a)÷b
[11]  r+0,t-q*b o +(0<pa+1+a)/Δ2
[12]  c+c Fadd 0 o →Δ99

```

And this is the path for multiprecision divisors. Start by creating the remainder, and make a floating-point divisor from the first two digits of the divisor.

```

[13] Δ3:r+a o c+0 0
[14]  bf+b[1]+b[2]÷base

```

Loop here. If $b > r$ then we are done, otherwise we make a trial division to find the next part of the result. qf is the provisional quotient (line 19), which is refined in line 21 if it equals 1. The algorithm here is quite complicated, see Riesel[4], p.338 for more details.

```

[15]  $\Delta 4: \rightarrow (b \text{ Fgt } r) / \Delta 99$ 
[16]  $\Delta 5: af \leftarrow r[1]$ 
[17]  $\rightarrow (2 \geq \rho r) / \Delta 6$ 
[18]  $af \leftarrow af + r[2] \div base$ 
[19]  $\Delta 6: q \leftarrow \lfloor qf + af \div bf \circ j + 0$ 
[20]  $\rightarrow (1 \neq qf) / \Delta 7$ 
[21]  $q \leftarrow r \text{ Fgt } (\rho r) + b$ 
[22]  $\Delta 7: \rightarrow (0 \neq q) / \Delta 8$ 
[23]  $q \leftarrow \lfloor qf + qf \times base \circ j + 1$ 
[24]  $\Delta 8: r1 \leftarrow r \text{ Fsub } b \text{ Fmul } q1 \leftarrow (2 + (\rho r) - j + \rho b) + q1 + 0, \lfloor q$ 
[25]  $\rightarrow (\sim 0 \text{ Fgt } r1) / \Delta 9$ 
[26]  $q \leftarrow q - 1 \circ \rightarrow \Delta 8$ 
[27]  $\Delta 9: c \leftarrow c \text{ Fadd } q1 \circ r + 0, (1 + r1), r1[0] \rho 0$ 

```

It may be that our truncation of the provisional quotient has resulted in $r=b$, so

```

[28]  $\rightarrow (\sim b \text{ Fequal } r) / \Delta 10$ 
[29]  $r \leftarrow 0 \circ c \leftarrow c \text{ Fadd } 1$ 
[30]  $\Delta 10: \rightarrow \Delta 4$ 

```

And here we tidy up, and return the quotient and remainder:

```

[31]  $\Delta 99: c \leftarrow (c \times 1, (\sim 1 + \rho c) \rho s) (r \times 1, (\sim 1 + \rho r) \rho s)$ 
       $\vee$ 

```

The floating-point division function plays around with our numbers to make integers of them, calls the integer version, then tidies up:

```

       $\vee a \leftarrow a \text{ Fdiv } b; d; r$ 
[1]  $a \leftarrow scalar \ a \circ b \leftarrow scalar \ b$ 
[2]  $\square \text{ SIGNAL}(0 \wedge . = 1 + b) / 11$ 
[3]  $\rightarrow (0 \wedge . = 1 + a) / 0$ 

```

Increase the precision of the dividend by the number of digits in the divisor:

```

[4]  $a \leftarrow (a[0] + 1 - \rho b), 1 + a, (\sim 1 + \rho b) \rho 0$ 

```

Pretend that our numbers are integers, and perform the integer division:

```

[5]  $d \leftarrow a[0], b[0]$ 
[6]  $a \leftarrow 0, 1 + a \circ b \leftarrow 0, 1 + b$ 
[7]  $a \leftarrow r \leftarrow a \text{ Idiv } b$ 

```

If the remainder is greater than or equal to half the divisor add 1 to the quotient. Maintain the precision of the quotient by padding with unnecessary trailing zeros, finally positioning the radix point in the correct place.

```
[8]      →(b Fgt r Fmul 0 2)/Δ1
[9]      a←a Fadd 0 1
[10]     Δ1:a←fullint a
[11]     a[0]←-/d
      ▽
```

Raising to a scalar power

In this section, I will call the number b in the expression a^b the power, rather than the exponent, to avoid confusion with the exponent of the multiprecision number.

The algorithm for this little gem comes from Ribenboim[5], p.38. The function raises a multiprecision number to the power of another number which must not be too big (for workspace considerations). We allow a small multiprecision number for the power, as well as a scalar. Anything larger than the radix will trigger a domain error, as will a negative power.

```
      ▽ z←m Fspow x;i;n;q;rem
[1]      →(2≠ρ,x)/Δ1
[2]      →(0≠1+x)/Δ1
[3]      x←x[1]
[4]      Δ1:⌈SIGNAL((^/x∈⌈AV)∨1≠ρ,x)/11
[5]      ⌈SIGNAL(x≤0)/11
```

Write the power in radix 2. Start with $z=1$, and at the left of the binary representation of the power. Square z . If the current bit in the power is 1 we also multiply z by the number m . Proceed this way until the right of the power is reached. Of course, the first bit in the power is always 1, so we can cut some processing, and we get the following:

```
[6]      z←m
[7]      →(1=ρn+((⌈2≠1+x)ρ2)⌈x)/0
[8]      i←1
[9]      Δ2:z←z Fmul z
[10]     →(~n[i])/Δ3
[11]     z←z Fmul m
[12]     Δ3:→((ρn)>i+i+1)/Δ2
      ▽
```

And there's more ...

If you have got this far you might like to try the following, assuming that your APL will let you do it. It is based on the well-known matrix multiplication way of generating the Fibonacci sequence, and it illustrates the use of the inner product operator with two user-defined functions.

```
Fadd.Fmul\4p=2 2p1 1 1 2
```

Of course, it doesn't run very fast (this can run faster if you write your own scan operator), and the output looks a bit strange, with numbers all over the place, but I will show how to speed and tidy this up in future articles.

In Part 2 I shall show the input and output routines that will enable you to make some sense of the numbers you get out of these functions. I shall also show you how to extract the square root of a random 100-digit number in less time than it takes to key the number in.

References

- [1] Geoffrey V Wood, *Large Primes* (Dept. of Mathematics & Computer Science, University College of Swansea, 1988, later amended).
- [2] *The APL Handbook of Techniques* (IBM DP Scientific Marketing, 1988. Order No. S320-5996-0).
- [3] Donald E Knuth, *The Art of Computer Programming*, vol. 2: seminumerical algorithms (Addison-Wesley, 1989), Section 4.3.
- [4] Hans Riesel, *Prime numbers and computer methods for factorization* (Birkhäuser, 1985).
- [5] Paulo Ribenboim, *The Book of Prime Number Records* (Springer, 1988).
- [6] Samuel Yates, *Repunits and Repetends* (Star Publishing, Florida, 1982).

Calculating Probabilities for Elementary Distributions

by Alan M Sykes
A.M.Sykes@Swansea.ac.uk

Introduction

Recently I wanted to calculate a probability from a Poisson distribution having a large mean, preferably without approximating it using a Normal distribution. It prompted me to revise my ideas on how to calculate simple combinatorial quantities and associated probabilities, where use of standard APL primitives such as ! and * soon let you down. For example the APL syntax $r!n$ for the number of ways of choosing r objects from n objects is given by the formula:

$$\binom{n}{r} = \frac{n(n-1) \dots (n-r+1)}{1 \cdot 2 \dots r} \quad (1)$$

and it is obvious that if $r=1$, no matter how large the value n is, the required value is n itself. However if you try $1!n$ for values of $n > 32768$, domain error results. Similarly calculating a Poisson probability p_n given by the formula

$$p_n = e^{-\mu} \frac{\mu^n}{n!} \quad (2)$$

say with $n = \mu$ will fail if $\mu > 143$ if you use the conventional APL approach. Try for example:

$$(*-144) \times (144 * 144) \div !144$$

Just how should you go about calculating such probabilities, if you wish your routines to be as widely applicable as possible? Answering this question exploits the recursive nature of some of these objects.

Calculating $\binom{n}{r}$

The first thing to note is the recursive nature of the formula for $\binom{n}{r}$. For instance,

$$\binom{10}{4} = \frac{10}{4} \binom{9}{3}$$

and so the APL expression

```

      ×\ (10-0,18)÷19
10 45 120 210 252 210 120 45 10

```

gives all the values required except those for $r=0$ and $r=10$. This is wasteful however, in that, for example, values for $r>5$ equal their symmetric counterparts for $r<5$. Nevertheless you can get the idea that if you need to calculate the combinatorial coefficients for $n=1000$ and $r=50, 100, 200$, you may as well generate them for all values of r up to 200 using $\times\backslash$ and then pick out those you particularly wanted!

The second thing to note is that very often combinatorial coefficients occur as a multiplicative factor in a larger formula and so if you really want to strive for wide applicability, it is best to be able to calculate the logarithm of $\binom{n}{r}$ and exponentiate as appropriate. (For example, formula (2) for the Poisson distribution, where for large μ , the exponential power is small, the power of μ term is large and the factorial term is large. Similar problems can occur with the hypergeometric distribution to be considered later which involves the three combinatorial terms

$$\frac{\binom{n}{r} \binom{N-m}{m-r}}{\binom{N}{m}} \quad (3)$$

and similarly for the better known Binomial distribution whose formula is given by:

$$p_r = \binom{n}{r} p^r (1-p)^{n-r} \quad (4)$$

In such situations you have to contend with the possibility that individual terms will be beyond the range of the computer, whilst the overall result is not — hence the need for taking logarithms.)

The first function considered therefore calculates $\log_r \binom{n}{r}$ with n a fixed positive integer, and r a general array of integers in the range 0 to n . The basic idea is illustrated by calculating $\binom{n}{r}$ for $n = 10$ and $r = 1, 2, 3, 4, 5$ using the code:

```
*+\(10-0,14)-15
10 45 120 210 252
```

The function listing is as follows:

```

v RES←N LNNCR R;TR;T;TEMP;TL;TU;IT;D
[1]  a calculates log of N Comb R, with N a scalar,
    R an array
[2]  D←ρR ρ R←,R
[3]  RES←(ρR)ρ0 ρ *(0=ρTR+((R≠0)∧R≠N)/R) / '→END'
[4]  T←[TR\N-TR
[5]  TEMP←+\(10-IT-1)-1IT+1T
[6]  TL←(TR≤N-TR)/TR ρ RES[(R≤TL)/1ρR]+TEMP[IT;TL]
[7]  TU←(TR>N-TR)/TR ρ RES[(R≤TU)/1ρR]+TEMP[IT;N-TU]
[8]  END:RES+DρRES
v
```

Interestingly enough, despite the use of logarithms and exponentials, this is a factor four times faster than $r!n$ for integer r and n as can be seen from the following piece of code (Dyalog APL/W Version 7.0 on a 486DX):

```

Q←[AI[2]ρZ+*1000 LNNCR 0,11000ρQ+Q,[AI[2]ρ
Z+(0,11000)!1000ρQ+Q,[AI[2]
Q[2]-Q[1]
110
Q[3]-Q[2]
440
```

so it seems that we have only lost out on generality ($r!n$ works for non-integer r, n and also for more than one n value) and temporary use of memory. (The program can be extended to cope with more than one n value but it is tricky — any volunteers?)

The Hypergeometric Distribution

Understanding this distribution is of course now obligatory for anybody who wishes to work out the odds on winning the jackpot in the National Lottery! If n numbers are chosen from the numbers 1 to N without replacement, then a further

m numbers are chosen from the same numbers 1 to N without replacement, what is the probability that r of the m numbers were also in the sample of size n ? The answer is given by formula 3 above. This is easily programmed using *LNNCR*, and the code is as follows:

```

      V RES←r HYPERGEOM N_n_m;N;m;n
[1]  A calculates hypergeometric probs for single integers N n m
[2]  A and an array of integers r
[3]  N n m←N_n_m
[4]  RES←*(n LNNCR r)+((N-n)LNNCR m-r)-N LNNCR m
      V

```

This copes quite happily with some very large integer arguments. The probability of winning a 'tenner' (the probability of having 3 numbers of the 6 chosen) is then calculated by:

```

      3 HYPERGEOM 49 6 6
.01765040387

```

Calculating Poisson Probabilities

Similar tricks can be played to calculate Poisson probabilities given by formula (2). Again, if you assume the mean parameter is fixed but you may require more than one probability, then you might as well (memory space willing) calculate the lot up to the maximum n value required. Again the formula is recursive, and again, taking logarithms is important to achieve wide applicability. Taking logarithms of formula (2) gives a term of the form $n \log_e \mu$: a nice trick is to calculate this by just adding a $\log_e \mu$ term to the $\log_e n!$ vector which has n terms and letting the $+$ do the rest (which if you think about it means that the calculation of the powers has been subsumed into the calculation of the factorial term!)

So, for example, if $\mu = 5$ and we require all the probabilities for $n = 0, 1, 2, \dots, 10$, then this is achieved by:

```

      8 4*+*(-MU)+0,+ \ (eMU)-e 1 10
0.0067  0.0337  0.0842  0.1404  0.1755  0.1755  0.1462
0.1044  0.0653  0.0363  0.0181

```

which you can check by comparison with a more direct interpretation of (2) in APL:

```

      8 4*( *-MU)*(MU*0,110)/!0,110
0.0067  0.0337  0.0842  0.1404  0.1755  0.1755  0.1462
0.1044  0.0653  0.0363  0.0181

```

The complete function is as follows:

```

▽ R←N POIS MU;DN;DM;D;M;I
[1]  a calculates Poisson probabilities for array of integers N
[2]  a and array of mean values MU
[3]  DN←ρN ◊ DM←ρMU
[4]  N←,N ◊ D←ρMU←,MU
[5]  ⚡((0=1 N)∧1=ρN)/'R←*-MU◊→0'
[6]  ⚡(1=ρN)/'R←*(-MU)++/(⊙MU)◊.-⊙1N◊→END'
[7]  I←0,1M←I/N
[8]  R←*((M+1),D)ρ-MU)+⊙0,+\\(⊙MU)◊.-⊙1M
[9]  R←R[I;N;]
[10] END:R←(DN,DM)ρR
▽

```

Here (because it is easy to), we have allowed the arrays *N* and *MU* to be anything you want. Here are a few examples:

```

      8 3*0 1 2 3 4 5 POIS 1
0.368   0.368   0.184   0.061   0.015   0.003

      8 3*0 1 2 3 4 5 POIS 1 2
0.368   0.135
0.368   0.271
0.184   0.271
0.061   0.180
0.015   0.090
0.003   0.036

      1000 POIS 1000
0.01261461135

```

It is worth modifying the code of *POIS* ever so slightly to get a *POISCDF* function that accumulates the probabilities — this too works well for very large examples:

```

▽ R←N POISCDF MU;DN;DM;D;M;I
[1]  a calculates the Poisson probabilities ≤ N
[2]  a for integer array values N and mean array MU
[3]  DN←ρN ◊ DM←ρMU
[4]  N←,N ◊ D←ρMU←,MU
[5]  ⚡((0=1 N)∧1=ρN)/'R←*-MU◊→END'
[6]  I←0,1M←I/N
[7]  R←+\\1)*((M+1),D)ρ-MU)+⊙0,+\\(⊙MU)◊.-⊙1M

```

```
[8]      R←R[I∖N;]
[9]      END:R←(DN,DM)ρR
      v
```

```
      8 4#100 120 140 POISCDF 100 120
0.5266 0.0347
0.9773 0.5243
0.9999 0.9669
      1000 POISCDF 1000
0.5084093672
```

Calculating Binomial Probabilities

For the final discrete distribution dealt with in this paper, we turn to the Binomial distribution where the probabilities of r successes out of n trials (where each trial has a success probability p) is given by formula (4) for values of $r=0,1,2,...n$.

If all these probabilities are required, then the same kind of technique as used for the Poisson distribution exploiting the recurrent nature of the formula can be employed. To see how, consider the $n=2$ case, and think of the vector of probabilities required as a product of a scalar ($n!$), a vector of three values involving powers of p divided by factorials and a vector of three values of powers of $(1-p)$ divided by factorials:

$$n! \left\{ 1 \quad \frac{p}{1} \quad \frac{p \cdot p}{1 \cdot 2} \right\} \left\{ \frac{(1-p)(1-p)}{2!} \quad \frac{(1-p)}{1} \quad 1 \right\}$$

If in addition you play safe and use logarithms to convert all the products to sums you end up with an APL expression like:

```
P←0.2
R←0 1 2
N←2
*(+/⊙∖N)+(0,+\(⊙P)-⊙∖N)+(⊙0,+\(⊙1-P)-⊙∖N)
0.64 0.32 0.04
```

which we can check by translating the formula (4) directly into APL:

```
(R!N)×(P*R)×(1-P)*N-R
0.64 0.32 0.04
```

This is worth having as an *ALLBINPROB* function:

```

      ▽ R←ALLBINPROB N_P;N;P;T
[1]  a calculates all binomial probabilities for scalar N P
[2]  N_P←N_P ◊ T+⊙N
[3]  R←*(+/T)+(⊙0,+\\(⊙1-P)-T)+0,+\\(⊙P)-T
      ▽

      ALLBINPROB 2 .2
0.64 0.32 0.04
      +/ALLBINPROB 1000 .3
1

```

If you don't want all the probabilities, and you do want to cope with very large n values and/or very small p values, then *LNNCR* can be used in the obvious way:

```

      ▽ RES←R BIN N_P;D;N;P
[1]  a calculates specified Binomial probabilities for scalar
[2]  a N and P
[3]  D←ρR ◊ R←,R ◊ N_P+N_P
[4]  RES←Dρ*(N LNNCR R)+(R×⊙P)+(N-R)×⊙1-P
      ▽

      0 1 2 BIN 2 .5
0.25 0.5 0.25
      +/(0,11000) BIN 1000 .5
1

```

A brief examination of speed for these routines calculating all the Binomial probabilities for $n=1000$ seems to suggest that *ALLBINPROB* is **seven** times faster than *BIN*, which in turn is about **seven** times faster than the conventional APL calculation!

6. Conclusion

Basically, avoid the use of shriek (!) if its arguments are integers! Perhaps this is a message for the *ISOAPL* standards people as well as APL programmers?

Alan M. Sykes
European Business Management School
University of Wales
Swansea SA2 8PP

A Quite Different New Primitive

by Gérard Langlet

APL, especially extended APL, now offers a large panoply of primitives. However, one is missing for a useful extension to binary algebra. In order to avoid the design of a new symbol, it seemed interesting to present a rationale for the use of an already-existing symbol, which, for the moment, has no monadic meaning. This symbol, known by all students even outside the APL world, is the unequal $\neq$ symbol which, in many other programming languages, corresponds to the logical function XOR with a couple of Boolean arguments. It will be shown here why the same symbol could be adopted as the one of matrix inverse when the argument is a modulo2 integer matrix. The essential reason comes from complex arithmetics, revisited with the help of the biternion theory.

From quaternions to biternions

Sir Rowan Hamilton, once, invented quaternions, i.e. hypercomplex numbers with four components, one real, and three imaginary ones on 3 orthogonal imaginary axes, which are also orthogonal with the real axis.

Very useful in geometry, especially for 3D transformations, quaternions have never been implemented as a functionality of APL.

But a distinction should be made between APL as a mathematical notation - what it was, when invented by Iverson prior to its first implementation by IBM as APL\360 (without complex arithmetics) - and APL as a programming language, which has always been - exactly like all other programming languages - a slave of the hardware.

Even if we would like to use the quaternion theory with a computer, and, again, not only under APL, the present-time limitations which come from the fuzz of floating-point arithmetic would lead to unefficient implementations : although FORTRAN allows - and even obliges - the user to declare REAL constants, no FORTRAN program has ever processed any real number during the 40 years of existence of that language ! Computers only know rational numbers. So, when a complex constant is used for any classical operation such as a product or a division (and even for add and subtract in the general case), the result is a complex constant containing two real numbers, each with a 50% probability of truncation on the least-significant bit. For quaternions, the probability of truncation would be the same... on each component, i.e. 4 times what it is on a single rational floating-point constant.

All microprocessors truncate information at each arithmetic operation in the general case, by rounding, always either in excess (ceiling) or in default (floor); their behaviour depends on the way bit-complements are managed in their internal cuisine (according to a recipe of bit complementing either to 1 or to 2).

Such effects, already very sensitive when a function is iterated, would become catastrophic with quaternions, although the theory is one of the most beautiful of the history of mathematics. A computer is not a mathematical processor; it is, in all cases, a bit processor first.

So, it seemed adequate to look for a computer-oriented way of representing, in general, complex constants.

Binary algebra - as exposed in the workshop presented at APL94 - may be considered as the union of two different algebras : logical algebra and modulo2 integer algebra. All operations which are necessary in modulo2 integer algebra may be performed thanks to the use of the logical Boolean primitives, because both algebras are isomorphous; only the meaning of 0 and 1 differs.

If matrix inversion a priori does not make sense in logic, it does make sense in modulo2 integer algebra; hence the need for some notation.

But binary matrices (understood as modulo2 ones) have fantastic properties, in particular the ability of representing - exactly - some complex or hypercomplex constants which, otherwise, cannot be represented in computers without fuzz.

Already, a variety of papers has shown that a simple modulo2-invertible matrix with order 2 (in APL with size 2 2) - a biternion - is able to represent - exactly - with 4 bits only, the main symmetry operator of spacetime (for me the main constant of physics), the 3-fold rotation operator around an axis, i.e. constant j , expressed in Euler's notation as $e^{2i\pi/3}$.

The reader may easily check that for any modulo2 matrix with order 2, matrix inversion is performed - without any operation - exchanging the contents of the bits which form its main diagonal; (the proof is immediate, when one knows how a regular matrix with order 2 is inverted, due to the fact that modulo 2 integer algebra is an unsigned algebra, so that the modulo2 determinant of any modulo2 integer matrix is 1 when the matrix is invertible, and 0 otherwise).

If matrix $\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}$ is chosen as j , then, matrix $\begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$ is $\pm j$ and also j^2 .

The hereabove-given symmetry rule for such matrix inversion also shows that matrices :

$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ $\begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}$ as well as $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ are self-inverses.

The reader may also check that, in coherent modulo2 algebra, the algebraic sum of the three cubic roots of 1, represented by the unit matrix and the j and j^2 matrices, then itemwise-modulo2-summed, is a null matrix, as it has to be.

The algebraic modulo2 sum is given either by $+/\$ taken modulo 2 on a vector of matrices, or, directly, by its logical counterpart $\neq/\$ since scalar function *different* (*unequal*) is isomorphous with either *plus modulo2* or *minus modulo2*.

Also, the result of $\neq/\$ on a vector of matrices with the same size, is the last item of $\neq\backslash$ applied to the same vector of matrices, independently from the order (indices) of the matrices within the vector.

Now, why can we generalise the meaning of $\neq$ and propose its monadic syntax as an equivalent of the one of the monadic domino for the modulo2 matrix inversion of binary matrices (i.e. rank-2 cells according to Iverson's terminology) ?

For several reasons. One comes from extended APL, the second one from quaternions.

In extended APL, monadic $+$ means "conjugate". Already, in both versions of the unextended ISO8485 standard (e for English, and f for French with word "conjugué"), the future complex extension was foreseen.

For "real" constants, $+Z$ is just Z ; for Z a complex (a,b) , then $+Z$ is $(a,-b)$.

So, the modulo2 counterpart of $+$, which, when $+$ symbolises addition, is indeed $\neq$, could also mean modulo2 conjugate for the new binary complex and hypercomplex numbers, based on biternions.

For all modulo2 invertible matrices with order 2 (the 6 matrices listed hereabove), each matrix is the complex conjugate of its modulo2 matrix inverse. j is indeed the conjugate of $j2$ and conversely. And self-inverse matrices are also their own conjugates. Complex conjugacy is, geometrically, a line-symmetry, the real axis being the symmetry element of the operation, seen in the complex plane.

In modulo2 matrices with order 2, the corresponding symmetry element simply becomes the 2nd diagonal of the matrices; *matrix inversion* and *complex conjugacy* have become, modulo 2, at least for all these matrices, the same operation.

Now, we have to show that the same coalescence of meanings for two (apparently) *different* expressions of conventional mathematics, also holds for any order i.e. for modulo2 invertible square binary matrices with any size.

Let us do it slowly - just because, at the present time, modulo2 integer matrix algebra is taught at school nowhere in the world (so that no book can help yet).

Think that you stay where you are on Earth, but that - perhaps - somebody you do not know is standing, not quite upside down - probably in a boat - not at the antipodes (this word, in Greek, does mean "upside down"), but in a point, on the Earth's surface, with the same longitude (E or W of Greenwich - or Paris), but with opposed latitude i.e. S instead of N or conversely. This guy would be your conjugate - and

conversely - on Earth. (The reference to Paris meridian, in fact not much in use anymore, shows that conjugacy is independent from the location of the reference axis chosen for zero-longitude). The location of the conjugate guy, however, will depend from the choice of the Equator, or, which is equivalent, from the choice of the polar axis, orthogonal with the equatorial plane.

Now, binary matrices with order 3 (size 3 3), have the same properties as the ones of the couple formed by you and your hypothetical conjugate guy.

If their modulo2 determinant is zero, they are not invertible. The square root of the absolute value of the determinant of any regular invertible matrix does play the role of the distance to the center of the sphere (or of the hypersphere for orders larger than 3), i.e. of the module of a 3-D hypercomplex number - a *ternion*, of a 4-D hypercomplex number - a quaternion - or a multi-D hypercomplex number - a *multi(n)ion* ? with n for euphony. Mathematical properties are more general than what one may think.

So, you could not live (except if you follow Jules Verne) at the center of the Earth with a null determinant.

Longitudes and latitudes have no meaning for a hot point which lies at the center of any such system, exactly as, in the (*module, argument*)-representation of a regular complex number, the number is 0, for any value of the argument when the module is already 0.

This may help to show that the hyperconjugate of any hypercomplex number is its symmetric on the same hypermeridian, the symmetry element being still the unique real axis, which is the polar axis.

On Earth (if supposed spherical, as it was for the first definition of the length unit named metre) the circumference is 40,000 km. Suppose a guy is sentenced to occupy on Earth, quantised locations only, each of them being separated by one meter, on the same meridian - including the opposed part, i.e. e.g. Greenwich meridian (0°) as well as the ideal "date-change" one '180° either E or W, or π). The guy may jump on the big circle, from one location to the next one; after 40,000,000 nice entrechats - think of Nureev - he will be back on the Covent Garden stage after some time. If two guys (conjugate-sentenced) are to jump synchronously in an endless ballet, they may meet one another - what a collision - on the poles only. Now, if the number of allowed locations were odd, they could meet only at one pole, let us say the North pole.

This is exactly what happens if you try to solve equation $z^{999} = 1$ which has 999 different solutions, one being - obviously- 1, the unique real solution, and the 998 remaining complex solutions being pairwise-conjugated on the trigonometric circle.

The North pole of the trigonometric representation of the complex plane is number 1 (or, as a matrix, a unit matrix). The South pole is number -1 (in matrix representation any self-inverse matrix : for regular matrices, the either vertical or horizontal mirror of the unit matrix - cfr. Pauli or Dirac matrices). In complex geometry, the polar axis is taken horizontally.

In 3-D complex geometry, this is still true : Imagine, this time Charles Chaplin, in the Great Dictator, playing with the globe : spinning around the poles - as the Earth does "spontaneously", - makes every possible (discrete) meridian (i.e. drawn on the globe) occupy every day, all along the day, e.g. the location Greenwich meridian had at midnight. Successively, all discrete meridians become, for a short while, a trigonometric circle. All points on the surface of an ideal planet are defined by their module (which is 1 by definition as the radius of the planet), and by two arguments (a latitude and a longitude, measured not from 0 to 90° N or S or from 0 to 180° E or W, but from 0 to 360° or 2π for the latitude and from 0 to 180° for the longitude).

A guy and his conjugate guy are on the same meridian, with module 1 and the same longitude. Their latitudes are indeed opposed in a plus and minus system, or, if one is Alpha, the other one is $2\pi - \text{Alpha}$ (or $2k\pi - \text{Alpha}$, with k any integer).

What is true for the complex roots of 1 also holds for binary matrices.

If any binary matrix (with module 1) is invertible, it will correspond to an allowed (discrete) location on some meridian of its hypersphere with latitude L.

Squaring this matrix will double the latitude (exactly as squaring a rotation matrix doubles the rotation angle in the complex plane, without making any change to the module which is 1, and with no possibility of escaping from the complex plane); similarly the matrix will stay on the same meridian. Raising the same matrix to any (integral) power does sentence the result to stay on the same meridian, so, to keep - always - the same longitude.

But, for a given order, the number of bit configurations within a matrix is finite. If one raises (modulo 2) some binary matrix M with modulo2 determinant 1 to its successive powers, one will obtain immediately a unit matrix if the matrix is self-inverse (lay at the S pole), and a different matrix, as its square, if the matrix is invertible but not self-inverse. For small matrices (there are only 6 invertible matrices with order 2), the number of different configurations is very limited. Even for huge matrices, one of these days, one of the matrix powers - let us say power Q, with Q an integer - will have to reproduce an already-known configuration of bits (e.g. already found for power P, another integer).

Then, matrix M raised to power Q+1 will reproduce M raised to power P+1, etc...

Power Q+K reproduces power P+K for any integer K, negative, positive, or 0.

When K equals -P, this proves :

- a) that power Q+K or Q-P of M is also power 0 of M then a unit matrix,
- b) that powers $n(Q-P)$ with n any integer will be unit matrices,
- c) that, if p is Q-P, then power p+1 of M is M.

d) that power $p-1$ of M is such that if this matrix, multiplied by M has to produce a unit matrix, power $p-1$ of M is also the modulo2 unique matrix inverse of M (by definition of an inverse matrix).

p is the power of the cyclic groups of all the different matrices which are matrix powers of M , including M^{-1} the inverse matrix of M , as well as M^0 the unit matrix.

Of course, M^{-2} is the square of M^{-1} and also the matrix inverse of M^2 the square of M ; and this property will hold for any power : the matrix inverse in modulo 2 integer algebra of any invertible matrix M , is, for any order, up to infinity, the *conjugate* of M in the geometric interpretation with hyperspheres, of hypercomplex quantities.

In other words, if M is equivalent to a rotation matrix on any meridian with angle α , its inverse matrix lies on the same meridian and corresponds to angle $-\alpha$.

And M is some exact equivalent of $e^{2k\pi i/p}$ with p the order of its cyclic group i.e. the power to which M raised equals a unit matrix, and with k "some" integer.

The difference with classical rotation matrices in 2D comes from the fact that k is not defined. Just word "some" is used. It is a matter of taste to choose a value for k , (modulo p).

If we fix k , then the value of the arguments for all the other powers of M is defined, in particular for the matrix inverse. The fact that a matrix and its inverse are conjugate in independent from the choice of k .

(A similar case arises with logarithms of complex numbers which are defined modulo $2\pi i$, and for the choice of the definition domain for inverse trigonometric functions).

Among the pair of matrices the cube of them is 1 for order 2, we may choose either of both as the complex constant j (the complex cubic root of 1 with positive imaginary part) ; then the other one corresponds to j^2 or j^{-1} .

Conclusion : If monadic + produces the complex conjugate of any complex number in APL, then its modulo 2 equivalent, monadic $\neq$ could produce the modulo2 generalised conjugate of any hypercomplex number which may be defined with modulo 2 integer matrices; then, indeed, monadic $\neq$ also corresponds to a perfect monadic domino (which will, this time, always return exact results, independently from the computer and from its floating-point arithmetic; even a bugged Pentium would then compute with no error).

Algorithm

The explanations given so far also lead to a general algorithm for binary matrix inversion, coming both from (cyclic) group theory and from graph theory.

However, there is no immediate need for studying group theory and graph theory for a reader who just knows some trigonometry :

The path (or ballerine jump) which leads, on a meridian from the North pole to any permitted location on the meridian, e.g. exactly where matrix M (supposed invertible) lies, is exactly the same, in length and direction¹ on the meridian, than the path which leads from the conjugate of M to the North pole.

Both paths, reversed, are the same in length and direction on the meridian.

So if we take matrix M and try to transform it, using a convenient algorithm (a recipe) into a unit matrix, the same algorithm will be able to transform, automatically, a unit matrix into the conjugate of M , i.e. the matrix inverse of M . Simple, isn't it ?

Now, any path may be divided into two sub-paths; this holds for algorithms : instead of a single function or program, we may chose to write two successive sub-functions or subroutines which will be invoked in a cover-function or in the main program, successively. I may fly from Paris to London with a connection in Brussels. I surely will choose this path if any ticket I may buy in Brussels costs nothing since the president of the Belgian company is my brother-in-law (unfortunately a stupid and wrong hypothesis, because I have no sister).

We shall not transform matrix M directly into a unit matrix - this costs too much thinking, looping or checking - , but into some other matrix P which will be easily invertible in all cases or, at least, which may be, in turn, easily transformed into a unit matrix.

If we apply, let us say to the first row of M , a transformation which will preserve the first 1 in this row, and reset the other 1s to 0, we will obtain a new row with only one 1 (of course).

Of course, again, there is always a first 1 in the first row of M ; otherwise, this row would contain only 0s, so that M would have a 0 determinant and would not be invertible (which does not fit the initial hypothesis).

If there is only one 1 in the first row - this will happen -, there is nothing to do (the algorithm becomes simple).

Let N be the order of the matrix.

If the number of 1s is more than one in row R_1 , we have to set to 0 the extra 1-bits from R_1 , using (as usual now) $\neq$. We shall apply the same recipe to the first row of a unit matrix. In fact we simply XOR the first row of the unit matrix with R_1 after having reset the first 1-bit of R_1 to 0 (XORing it with 1).

¹ direction on the meridian means « angular direction ».

And we shall perform the same operations to all other rows which contain a 1 with the same index as in R1, both in M and in the unit matrix.

Then, no other row in M may contain 1 in the same column as R1 does.

If we repeat the same operation on row R2, keeping this time the first 1-bit of R2 only (which, again, must exist, otherwise M would not have been invertible), and on rows which also have a 1 in the same column as R2, the new R2 will have just one 1-bit left in a different column than R1. The same rows are XORed with the same vector also in the former unit matrix.

TheXORing elementary algorithm is now applied to row R3 in order to keep the first 1-bit, and to the rows which contain the same first 1-bit, and to the same rows, with the same XORing mask in the former unit matrix,... etc... until the last row is reached and processed.

In the end, matrix M has become a matrix in which there is only one 1 in each row, all the N 1s being located in different columns. Such matrices have a name in mathematics : they are *permutation* matrices.

Then, if one knows that the matrix inverse of any permutation matrix is its transposed matrix, the inverting task is almost already over (without any second sub-program).

In fact we just have to reindex the columns of the former unit matrix according to the permutation defined by the permutation matrix : this makes transposition useless (it could take time to transpose a huge matrix), so we simply reindex the columns because we have worked with the rows; of course, transposing the dimensions from the beginning would lead to the same result : keeping only the first 1 successively in all columns of M, and transforming each time with the same mask the columns of a unit matrix, produces a permutation matrix which reindexes the rows of the former unit-matrix, thus transformed, producing the desired modulo2 matrix-inverse or modulo hypercomplex conjugate...

As arithmetic operations, the algorithmic definition of matrix inversion or conjugation, corresponding to the monadic $\neq$ symbol, will have used only... XORs corresponding to classical $\neq$.

It is hoped that this paper may bring some new some ideas to implementers, because APL could become an uncomparable tool also for exact hypercomplex geometry in modulo2/logic algebra with plenty of applications (even if the reader cannot imagine them still now).

MAGIC was solved using modulo2 cyclic groups - already almost 3 years ago - in half a line of APL. Many problems may be in fact solved by modulo2 matrix inversion or matrix powers, once one knows how to formulate them in this fantastic algebra (it is not still obvious in most cases...); this sentence is not just a conjecture, but a theorem, which may make, in the future, the usage of computers more fiable and accurate than ever before. The consequence of a theorem may be a new conjecture. If this conjecture is true, this will have happened thanks to APL.

Here is a possible version of the modulo2 matrix-inversion algorithm which results from the « hypersphere meridian » method :

```

  ▽ M←INV M;D;I;J;K;L;N;V;T;e
[1] I←010≠1 ∘ L←1D+1'S←PM ∘ K←pI+V-D+1 ∘ T←SpV,N←0
[2] e←'J+V≠M[;I1] ∘ J←pI+J/L ∘ M[I;]←M[I;]≠M[J+J/N;] ∘ T[I;]←T[I;]≠T[J;]'
[3] 'J+1∈I←,M[N+V/L;] ∘ J/e ∘ V←0,~1+V' wh 'K≠N' ∘ V←VM ∘ M←T[V;]
  ▽

```

(Other versions appeared e.g. in «The Power of Boolean Computing», by G. A. Langlet, SEAS94 Spring Meeting, La Hulpe, in APL94 Companion, in *Les Nouvelles d'APL*, in APL-CAM J.).

Note that this version (like the other ones) also « general-inverts » non-invertible square matrices with a modulo2-determinant equalling 0; (such a general inverse of a square null matrix is a conforming unit matrix, by definition). Beware of the fact that different versions may produce different general inverses; however, all versions produce the same unique inverse for a modulo2 invertible matrix.

So, one never gets DOMAIN ERROR.

For the direct solution of a modulo2 linear system, see also function *SEG* in the special issue for APL95 of *Les Nouvelles d'APL* N° 15 (May 1995), paper « In XOR we trust », by Ilya Mironov.

≠

The Complete Elliptic Integrals and APL

by Joseph De Kerf

In a previous note [1], a definition was given of the so-called *common mean* of two numbers, and it was shown how easy it is to program the algorithm in APL. In this note it is shown how the concept may be used to calculate the complete elliptic integrals of the first and second kind [2] [3]. User-defined functions in APL are given. Incomplete integrals will be treated in a following and final note.

The complete elliptic integrals of the first kind $K(p)$ and of the second kind $E(p)$ may be defined as:

$$K(p) = \int_0^{\pi/2} \frac{d\theta}{\sqrt{1-p^2 \sin^2 \theta}}$$

$$E(p) = \int_0^{\pi/2} \sqrt{1-p^2 \sin^2 \theta} \, d\theta$$

with $p^2 \leq 1$ or $-1 \leq p \leq 1$. Both complete elliptic integrals are even:

$$K(-p) = K(p) \text{ and } E(-p) = E(p)$$

Complete elliptic integrals of the complementary modulus q are defined as $K'(p) = K(q)$ and $E'(p) = E(q)$, with $q^2 = 1 - p^2$. The two auxiliary functions are also both even:

$$K'(-p) = K'(p) \text{ and } E'(-p) = E'(p)$$

The complete elliptic integrals may be calculated for instance by numerical integration or by series expansion [2] [3]. Much easier to program, and this is especially true in APL, is the algorithm based on the common mean concept [2] [3]. Indeed, the complete elliptic integral of the first kind $K(p)$ may be calculated by application of the relation:

$$K(p) = \frac{\Pi}{2c(1,q)}$$

where $c(1,q)$ is the common mean of 1 and $q = \sqrt{1-p^2}$

A user-defined function *CEI1*, based on this procedure, may be:

```

      VCEI1[[]]V
      VR←CEI1 P
[1]   R←1, (1-P*2)*0.5
[2]   LAB:→(π/R+(0.5×+/R), (×/R)*0.5)/LAB
[3]   R←(o1)÷+/R
      V

```

which for $p = 0.0(0.1)0.9$ and comparison tolerance $1E^{-15}$ gives:

```

R←CEI1 P←0.0.1×19
3 1 15 10 20 15 P,R,[1.5] R
0.0 1.5707963268 1.570796326794897
0.1 1.5747455615 1.574745561517356
0.2 1.5868678475 1.586867847454166
0.3 1.6080486199 1.608048619930513
0.4 1.6399998659 1.639999865864511
0.5 1.6857503548 1.685750354812596
0.6 1.7507538029 1.750753802915752
0.7 1.8456939984 1.845693998374723
0.8 1.9953027777 1.995302777664729
0.9 2.2805491384 2.280549138422770

```

Results are displayed with 10 and 15 digits after the decimal point respective should be noticed that for $p = \pm 1$, the complementary modulus q is 0, and complete elliptic integral of the first kind is infinite (∞). A domain error reported and the user-defined function is suspended. Eventually, the error may be trapped and an appropriate reaction may be programmed.

The complete elliptic integral of the second kind $E(p)$ may be calculate application of the relation:

$$E(p) = \frac{K(p)}{2} \left(2 - p^2 - \sum_{i=1}^{\infty} 2^i (x_i^2 - y_i^2) \right)$$

$$E(p) = \frac{K(p)}{2} \left(2 - \sum_{i=0}^{\infty} 2^i (x_i^2 - y_i^2) \right)$$

$$\text{or } E(p) = \frac{\Pi}{4c(1,q)}(2-S) \quad \text{and} \quad S = \sum_{i=0}^{\infty} 2^i (x_i^2 - y_i^2)$$

where $c = (1, q)$ is the common mean of 1 and $q = \sqrt{1-p^2}$, and the x_i s and y_i s are the successive arithmetic and geometric means while calculating this common mean:

$$\begin{aligned} x_0 &= 1 & \text{and} & & y_0 &= q \\ x_{i+1} &= (x_i + y_i)/2 & & & y_{i+1} &= \sqrt{x_i y_i} \end{aligned}$$

A user-defined function *CEI2*, based on this procedure, may be:

```

VCEI2[[]]V
VR←CEI2 P;S;I
[1]  →(1=|P)/0,R←1
[2]  R←(1+S+1+I←1),(1-P*2)*0.5
[3]  LAB:S←S+(-/R*2)*2*I+I+1
[4]  →(←/R+(0.5×+/R),(×/R)*0.5)/LAB
[5]  R←((01)÷2×+/R)*2-S
V

```

which for $p = 0.0(0.1)1.0$ and comparison tolerance $1E^{-15}$ gives:

```

R←CEI2'' P←0,0.1×10
3 1 15 10 20 15#P,R,[1.5] R
0.0 1.5707963268 1.570796326794897
0.1 1.5668619420 1.566861942021668
0.2 1.5549685462 1.554968546242529
0.3 1.5348334649 1.534833464923249
0.4 1.5059416124 1.505941612360040
0.5 1.4674622093 1.467462209339427
0.6 1.4180833944 1.418083394448724
0.7 1.3556611356 1.355661135571956
0.8 1.2763499432 1.276349943169906
0.9 1.1716970528 1.171697052781614
1.0 1.0000000000 1.000000000000000

```

It should be noticed that for $p = \pm 1$ the complementary modulus q is 0, the common mean $c = (1, q)$ is 0, while the complete elliptic integral of the second kind is $E(\pm 1) = 1$. As such, in the first line of the function, for $p = \pm 1$ the result R is set to 1 and flow of execution returns to the invoking expression. In the second line, the partial sum S and the counter I are initialised to 0 and -1 respectively. The loop, lines 3 and 4, may be written in one line of course, but the iteration has been split to increase readability, while the overhead is negligible. In the last line $K(p)/2$ is multiplied by $2-S$, giving finally $R = E(p)$.

For both user-defined functions, the relative accuracy of the common means calculated is at least the current value of the comparison tolerance, for the results shown $1E^{-15}$. In practice, however, this relative accuracy is slightly better, as the exact value of the common mean lies between the two iterations calculated and the result is set to the arithmetic mean of the final values of those iterations.

Note: The number of iterations needed and the cpu time depend on the current value of the comparison tolerance and on p . Results of a benchmark for $1E^{-15}$ and $p = 0.1(0.2)0.9$ are shown below. The cpu times are in milliseconds.

p	0.1	0.3	0.5	0.7	0.9
Number of iterations	3	3	4	4	5
CEI1	112	112	137	137	163
CEI2	208	208	255	255	303

The cpu times are rather high, but are considerably better than for other methods such as for instance numerical integration and series expansion.

The calculations and the benchmark have been done on a MicroVAX 2000, with VAX APL Version 3.1, under VMS Version 4.7 [4]. The default value for the comparison tolerance in VAX APL is $1E^{-15}$. The number of iterations has been measured by globalising the counter I . The cpu times have been measured with the system function `MONITOR`.

References

- [1] J. De Kerf: *The Common Mean and APL*. Vector Vol.11 No.3, pp21-23. 1995
- [2] M. Abramowitz and I.A. Stegun (Eds): *Handbook of Mathematical Functions*. Applied Mathematics Series No. 55, National Bureau of Standards, Washington D. C., 1964 and subsequent revisions.
- [3] J. Spanier and K.B. Oldham: *An Atlas of Functions*. Hemisphere Publishing Corporation, New York, New York 1987.
- [4] *VAX APL Reference Manual – Vols. 1 and 2 (Software Version V.3.0)*. Publication Numbers: AI-P142D-TE and AI-GV09B-TE. Digital Equipment Corporation, Maynard, Massachusetts, June 1987.

Production Manager's Observation

When run in Dyalog/W, the sample evaluations produce very slightly different results! Both are run in the same comparison tolerance!

First Example ...

```

[]CT+1E-15
R+CEI1'' P+0,0.1x19
3 1 15 10 20 15*P,R,[1.5] R
0.0 1.5707963268 1.570796326794897
0.1 1.5747455615 1.574745561517356
0.2 1.5868678475 1.586867847454166
0.3 1.6080486199 1.608048619930513
0.4 1.6399998659 1.639999865864511
0.5 1.6857503548 1.685750354812596
0.6 1.7507538029 1.750753802915753
0.7 1.8456939984 1.845693998374724
0.8 1.9953027777 1.995302777664730
0.9 2.2805491384 2.280549138422770

```

Second Example ...

```

[]CT+1E-15
R+CEI2'' P+0,.1x110
3 1 15 10 20 15*P,R,[1.5] R
0.0 1.5707963268 1.570796326794897
0.1 1.5668619420 1.566861942021668
0.2 1.5549685462 1.554968546242529
0.3 1.5348334649 1.534833464923250
0.4 1.5059416124 1.505941612360040
0.5 1.4674622093 1.467462209339424
0.6 1.4180833944 1.418083394448724
0.7 1.3556611356 1.355661135571955
0.8 1.2763499432 1.276349943169906
0.9 1.1716970528 1.171697052781616
1.0 1.0000000000 1.000000000000000

```

Perhaps someone with *Mathematica* could say which is correct!!

The Common Mean, Non-negative Definite Matrices, and APL

by Dietrich Trenkler (University of Osnabrück)
and Götz Trenkler (University of Dortmund)
Email: trenkler@rols1.oec.uni-osnabrueck.de

Introduction

In a recent contribution to *Education Vector* De Kerf [3] advocated the use of the common mean as an alternative to the geometric mean and the arithmetic mean. If x_0 is the smaller of two positive numbers x_0 and y_0 , the geometric mean of these numbers is $x_1 = \sqrt{x_0 y_0}$ while their arithmetic mean is $y_1 = (x_0 + y_0)/2$. If the procedure of forming geometric and arithmetic means is repeated indefinitely so that

$$x_{i+1} = \sqrt{x_i y_i} \quad \text{and} \quad y_{i+1} = (x_i + y_i)/2,$$

where $i = 1, 2, \dots$, one obtains

$$x_0 \leq x_1 \leq x_2 \leq \dots \leq x_i \leq \dots \leq y_i \leq \dots \leq y_2 \leq y_1 \leq y_0,$$

and there is convergence to a common mean $c(x_0, y_0)$ of x_0 and y_0 .

In the following it will be shown that this notion of a mean can be generalized to nonnegative definite (n.n.d.) matrices, provided some assumptions are met. Note that the arithmetic, harmonic and geometric mean of nonnegative definite (n.n.d.) matrices has already been discussed in the literature, e.g. Anderson and Duffin [1].

Theory

Consider two n.n.d. matrices X_0 and Y_0 of type $n \times n$ such that X_0 and Y_0 commute, i.e. $X_0 Y_0 = Y_0 X_0$. Since both matrices are also symmetric, there exists a simultaneous diagonalization of X_0 and Y_0 by an orthogonal matrix P , that is

$$X_0 = P \Lambda P', \quad Y_0 = P \Omega P', \quad (1)$$

where $\Lambda = \text{diag}(\lambda_j(X_0))$ and $\Omega = \text{diag}(\omega_j(Y_0))$ is the diagonal matrix of eigenvalues of X_0 and Y_0 , respectively. This will be demonstrated in the section "Algorithm and Programs". Furthermore, we assume that

$$\lambda_1(X_0) \leq \dots \leq \lambda_n(X_0) \quad \text{and} \quad \omega_1(Y_0) \leq \dots \leq \omega_n(Y_0).$$

Consider now the following sequence of matrices

$$\begin{aligned} X_1 &= (X_0 Y_0)^{\frac{1}{2}} & Y_1 &= (X_0 + Y_0)/2 \\ X_{i+1} &= (X_i Y_i)^{\frac{1}{2}} & Y_{i+1} &= (X_i + Y_i)/2 \end{aligned} \quad i = 1, 2, \dots$$

Observe that since X_0 and Y_0 commute, $X_0 Y_0$ is an n.n.d. matrix, and the square root X_1 of $X_0 Y_0$ is given by

$$X_1 = P(\Lambda\Omega)^{\frac{1}{2}}P' = P \text{diag}(\sqrt{\lambda_j(X_0)\omega_j(Y_0)})P'.$$

By induction it is readily shown that the X_i and Y_i are well defined and n.n.d. Furthermore X_i and Y_i commute, and

$$X_i = P\Lambda_i P', \quad Y_i = P\Omega_i P',$$

where $\Lambda_i = \text{diag}(\lambda_j(X_i))$ and $\Omega_i = \text{diag}(\omega_j(Y_i))$. If λ_{ij} and ω_{ij} denotes the j -th diagonal element of Λ_i and Ω_i , respectively, it can be readily seen that

$$\begin{aligned} \lambda_{1j} &= \sqrt{\lambda_j(X_0)\omega_j(Y_0)} & \omega_{1j} &= (\lambda_j(X_0) + \omega_j(Y_0))/2 \\ \lambda_{i+1j} &= \sqrt{\lambda_{ij}\omega_{ij}} & \omega_{i+1j} &= (\lambda_{ij} + \omega_{ij})/2 \end{aligned}$$

for $i = 1, 2, \dots$ and $j = 1, \dots, n$. Let now $c(\lambda_j(X_0), \omega_j(Y_0))$ denote the common mean of $\lambda_j(X_0)$ and $\omega_j(Y_0)$. Then obviously the sequence of matrices (X_i, Y_i) converges to a common matrix mean $C(X_0, Y_0)$ given by

$$C(X_0, Y_0) = P \text{diag}(c(\lambda_j(X_0), \omega_j(Y_0)))P',$$

which might be considered as a competitor to the geometric mean $X_1 = (X_0 Y_0)^{\frac{1}{2}}$ and the arithmetic mean $Y_1 = (X_0 + Y_0)/2$ of the matrices X_0 and Y_0 .

Algorithm and Programs

In the following (assuming $\Pi IO+1$ throughout) we give a program that yields the orthogonal P mentioned in (1). It is based on the following:

Let A and B be $k \times k$ symmetric matrices. A necessary and sufficient condition that an orthogonal matrix P exists such that $P'AP$ and $P'BP$ are each diagonal is that $AB = BA$ (or AB is symmetric).

cf. Graybill [4], pp. 406-7. The following algorithm is extracted from the proof of this theorem.

- (i) Compute an orthogonal matrix R such that $R'AR = D$ where D is diagonal.
 (ii) Assume that D can be written as

$$D = \begin{pmatrix} \lambda_1 I_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 I_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_m I_m \end{pmatrix}$$

- (iii) Set $C = RBR'$. It follows that C is of the form

$$C = \begin{pmatrix} C_{11} & 0 & \cdots & 0 \\ 0 & C_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & C_{mm} \end{pmatrix}$$

where C_{ii} is $k \times k$ whenever I_i is $k \times k$. If each C_{ii} consists of exactly one element C is diagonal and the algorithm stops by setting $P=R$. Otherwise, we have to obey (iv)-(vi).

- (iv) Find orthogonal matrices Q_j for $j = 1, 2, \dots, m$ such that $Q_j' C_{jj} Q_j$ is diagonal.
 (v) Set

$$Q = \begin{pmatrix} Q_1 & 0 & \cdots & 0 \\ 0 & Q_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & Q_m \end{pmatrix} \quad (2)$$

- (vi) Finally, $P = RQ$ is such that $P'AP$ and $P'BP$ are each diagonal.

It is readily seen that there are two instances (ii) and (iv)) where we need the spectral decomposition of an n.n.d. matrix. This calls for the computation of eigenvalues and eigenvectors and since most modern *APL2* systems offer workspaces with programs to find them we resort to them. For instance, *APL*PLUS II* provides a workspace *EIGENVAL* from which we took the program *SYMEIG* to compute the spectral decomposition of a matrix. This is done by *VALVEC+SPECDEC MAT*, where the first row of *VALVEC* are the eigenvalues *MAT* in ascending order and $1 \ 0 + VALVEC$ is an orthogonal matrix P of eigenvectors. It uses the function *STANDARDIZE* to convert the columns x of a matrix to

$$y = x/\sqrt{x'x} \quad \text{such that} \quad y'y = 1.$$

```

VR←SPECDEC M
[1] R←R[1;],[1]STANDARDIZE 1 0+R+1 SYMEIG M
[2] R←R[;AR[1;]]
V
VR←STANDARDIZE M
[1] R←M×((ρM)ρ+/[1]M×M)*-0.5
V

```

SIMDIAGMAT can be used to compute the orthogonal matrix P that diagonalizes X_0 and Y_0 simultaneously, cf. equation (1). It uses the function *FREQUENCIES* to find the frequencies of distinct elements in a vector and *DIAGMATFROM* to construct the block diagonal matrix Q given in (2).

```

VR←A SIMDIAGMAT B;R;C;Q;QI;D;E;I
[1] Q←10 0 I+1
[2] C←(QR)+.×B+.×R+1 0+SPECDEC A
[3] E←0,+D+,1 0+FREQUENCIES 1 1Q(QR)+.×A+.×R
[4] LOOP:Q←Q,C←1 0+SD (2ρD[I])+(2ρE[I])+C
[5] +((ρD)≥I+I+1)/LOOP
[6] P←R+.×DIAGMATFROM Q
V
VR←FREQUENCIES X;A
[1] A←A-1+0,A←(R+(-1+X×1φX+X[AX]),1)/1ρX+,X
[2] R←(R/X),[0.1]A
V
VR←DIAGMATFROM M;I;A;B;C
[1] I←1
[2] M←(-2+(-c1 1),-ρ~M)ρ~M
[3] R←(R,R+1/A+,1 0+1[1]ρ~M)ρ0
[4] B←0,+A
[5] LOOP:R[C;C+B[I]+1A[I]]+I=M
[6] +((ρM)≥I+I+1)/LOOP
V

```

X_0 *CMEANMAT* Y_0 computes the common matrix mean $C(X_0, Y_0)$ of matrices $X_0, Y_0 \leftrightarrow X_0 \ Y_0$. It is based on the function *CMEAN* taken from De Kerf [3].

```

VR+X CMEANMAT Y;P;L;O
[1] P+X SIMDIAGMAT Y
[2] L+1 1*(Q P)+.*X+.*P
[3] O+1 1*(Q P)+.*Y+.*P
[4] R+P+.*(DIAGMATFROM L CMEAN"O)+.*Q P
V

```

Finally, to compute the arithmetic matrix mean and the geometric matrix mean we have

```

VR+A AMEANMAT B
[1] R+0.5*A+B
V

VR+A GMEANMAT B;P;L;O
[1] P+A SIMDIAGMAT B
[2] L+(Q P)+.*A+.*P 0+(Q P)+.*B+.*P
[3] R+P+.*( (L*O)*0.5)+.*Q P
V

```

Example

What is the common matrix mean of a matrix X_0 and its inverse $Y_0 = X_0^{-1}$? For instance, let $X_0 \ Y_0 \leftrightarrow X_0 Y_0$ where

$$X_0 = \begin{pmatrix} 2 & 0 & 1 & -1 & -2 \\ 0 & 2 & -2 & -1 & -1 \\ 1 & -2 & 4 & -1 & -1 \\ -1 & -1 & -1 & 3 & 2 \\ -2 & -1 & -1 & 2 & 4 \end{pmatrix} \quad Y_0 = \begin{pmatrix} 2 & 5 & 3 & 2 & 2 \\ 5 & 27 & 17 & 11 & 8 \\ 3 & 17 & 11 & 7 & 5 \\ 2 & 11 & 7 & 5 & 3 \\ 2 & 8 & 5 & 3 & 3 \end{pmatrix}.$$

X_0 and hence Y_0 are p.d. and $X_0 Y_0 = I$ which is the geometric mean of X_0 and Y_0 . Accordingly they can be used to exemplify the new concept of a common matrix mean of two n.n.d. matrices. We get

```

4 * X0 CMEANMAT Y0
1.3767  0.7982  0.6784  0.1113 -0.0731
0.7982  5.3691  2.3040  1.5550  1.0720
0.6784  2.3040  3.2190  0.9224  0.5988
0.1113  1.5550  0.9224  2.0441  0.8833
-0.0731 1.0720  0.5988  0.8833  1.9346

```

One is inclined to speculate on a relation like $a_{ij} < c_{ij} < g_{ij}$ where $A = (a_{ij})$ is the arithmetic mean, $C = (c_{ij})$ is the common mean and $G = (g_{ij})$ is the geometric mean. However, for the example above we have

5	1	X0	AMEANMAT	Y0
2.0	2.5	2.0	0.5	0.0
2.5	14.5	7.5	5.0	3.5
2.0	7.5	7.5	3.0	2.0
0.5	5.0	3.0	4.0	2.5
0.0	3.5	2.0	2.5	3.5

so that $a_{15} = g_{15} = 0$ and $c_{15} = -0.0731$.

Concluding Remarks

In this paper we introduced a matrix generalization of the common mean, originally defined only for real numbers. Like the other means which already exist it might prove useful in statistics and matrix theory. Further research should focus on algebraic properties related to the common mean of n.n.d. matrices (cf. Anderson and Trapp [2]). From the construction of the sequences X_i and Y_i it has become clear that

$$X_0 \leq X_1 \leq X_2 \leq \dots \leq X_i \leq \dots \leq Y_i \leq \dots \leq Y_2 \leq Y_1 \leq Y_0$$

and $X_0 \leq C(X_1, Y_0) \leq Y_0$, where $A \leq B$ for matrices A and B means that $B - A$ is n.n.d. It has also been investigated under which conditions the order relation " $\leq$ " can be substituted by " $<$ ", i.e. the corresponding differences are positive definite.

Acknowledgements

We would like to thank Adrian Smith and Bernhard Strohmeier who helped in preparing this paper. Furthermore, thanks to Jon Sandles for several useful hints.

References

- [1] Anderson, W.N. and Duffin, R.J. (1969). "Series and Parallel Addition of Matrices", *Journal of Mathematical Analysis and Applications* 26, 576-594.
- [2] Anderson, W.N. and Trapp, G.E. (1986). "Inverse Problems for Means of Matrices", *SIAM Journal of Algebraic Discrete Methods*, 7, 188-192.
- [3] De Kerf, J. (1995). "The Common Mean and APL", *Vector* 11.3, 21-23.
- [4] Graybill, F.A. (1983). *Introduction to Matrices with Applications in Statistics*, Second Edition, Wadsworth Publishing Co., Belmont, California.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Driving the MS Daisy-wheel	Ray Cannon	114
Technical Correspondence	Gérard Langlet	121
At Work and Play with J:	Gene McDonnell	125
"Correct" Windows File Behaviour	Duncan Pearson	129
Native File Functions for Dyalog APL	Adrian Smith	137

Hackers' Corner: Driving MS Daisy-wheel (and the HP laserjet, Canon BJ10 et al.)

by Ray Cannon

This article contains the listing of all the Dyalog APL/W code needed to:

- Set the orientation of the default printer. (Global to all Windows applications)
Function `SetGlobalOrientation`;
- Change the orientation within a single print job. (Local to the current printer)
Function `SetLocalOrientation`;
- Examine the capabilities of the selected printer driver. Function
`ShowDeviceMode`.

All of these tasks are performed by calls (via `⌈NA`) to Windows DLLs. The main function called is `ExtDeviceMode`. Calling Dynamic Link Libraries from Dyalog APL/W has been well documented in these pages in the past [1][2][3][4][5], so I will not go into much detail.

To standardize the interface between Windows and any printer attached, each printer manufacturer has to provide a piece of software (called a printer driver) which converts the Windows device-independent printer controls into the specific commands needed for their printer. Because capabilities of printers vary, a secondary task of the driver is to (on request) supply Windows with information about the capabilities of the printer. The structure used to pass this "control" data (as against "print" data, e.g. text or bitmap) between Windows and the driver is termed a `DeviceMode`. The function in the printer driver which utilises the `DeviceMode` is called `ExtDeviceMode`. At the Windows end, the information is used by the Device Context created for the printer.

A device context (DC) is a data structure created and maintained by the windows GUI in support of device-independent drawing operations on displays, printers, metafiles, and bitmaps. A DC is not only a toolbox containing a set of drawing tools, it's also the permission slip to use a device and provides the connection over which "print" data (as against control data) is passed.

The Windows call to create a DC for a printer requires the `DeviceMode` supplied by the printer driver.

The three things `ExtDeviceMode` is called on to do in the APL function `SetGlobalOrientation` are:

1. Return the size of the `DeviceMode` structure required by the printer driver. This is then used to allocate memory for a `DeviceMode` structure.
2. Read `DeviceMode` information from the printer driver. The contents of `DeviceMode` are then modified as required to change the orientation.
3. Write `DeviceMode` to the printer driver. The printer is now set up for the amended orientation.

Since a change is made to the Printer Driver, the change can be "seen" by any and all applications using the driver. That is, the change is **Global to Windows**. Also Windows will remember the change even over a re-boot of the PC.

In `SetLocalOrientation`, steps 1 and 2 are the same, but in step 3 rather than writing the `DeviceMode` to the printer driver, it is used to replace the DC being used by the named printer object (via `ResetDC`). Thus the global environment is unchanged, but the orientation is changed for the named printer.

The function `ShowDeviceMode` displays the contents of the `DeviceMode` obtained from the printer driver. It shows which capabilities the printer driver supports (and which ones it does not).

For documentation on the (Windows) functions called, can I refer you to Microsoft's Windows 3.1 System Developers Kit [6] (available from many sources in many guises, e.g. books, on CD, via Compuserve, on disc with language packages such as Visual Basic or Borland C++). The documentation I needed to get these functions working ran to over 20 A4 pages so cannot be reproduced within the limited pages of *Vector*. A copy of the workspace has been lodged in the APL section of the Portable Programming Library of Compuserve, and an earlier form of the WS is being distributed with the latest version of Dyalog APL/W (version 7.1).

Disclaimer. Making calls to Windows DLLs from APL with incorrect arguments or under the wrong conditions could crash Dyalog APL or Windows (or other applications running under Windows) with a subsequent loss of data. Neither I, nor *Vector*, the BAA, or Dyadic Systems, can be held responsible for any such loss or damage however caused through the use of this code.

Notwithstanding the above disclaimer, during the development of this code I made many incorrect calls to DLLs and did not crash Windows (or APL) once. So, just don't test your implementation of this code while running other critical software, and don't forget to)*SAVE* often.

Appendix: The Functions

```

ShowDeviceMode;gmh;gps;str;etxt;name;device;port;s;edm;args;fm;hand1;hand2;
lpdmin;lpdmout;rc;sdm
afunction to Show the current (default) printer driver settings
afix fns to get other info
'gmh'[]NA'I4 kernel.exe.P16|GetModuleHandle <0T'
'gps'[]NA'I kernel.P16|GetProfileString <0T <0T <0T >0T I'
aget name of (printer) device from WIN.INI
str+',',>1+gps'windows' 'device' '(undefined)' 256 255
name device port+3+1+'(','=str)<str
etxt+'No default printer defined'
+(name='(undefined)')/err
aset up arg string for ExtDeviceMode
s+'
s,+' I2'a hwnd      +-2 byte integer
s,+' I4'a hdriver   +-4 byte integer
s,+' I4'a lpdout     +-4 byte integer (long pointer to output buffer)
s,+' <0T'a lpszdevice +-pointer to char string
s,+' <0T'a lpszport  +-pointer to char string
s,+' I4'a lpdmin     +-4 byte integer (long pointer to input buffer)
s,+' I4'a lpszprofile+-pointer to char string or NULL (win.ini)
s,+' I2'a fwmode     +-2 byte integer (word)
afix external function call to ExtDeviceMode
'edm'[]NA'I2 'device','drv.P16|ExtDeviceMode ',s
aset up arguments to ExtDeviceMode
args+8p0
args[1]+0 a cgethandle'fm' aShould be the handle of the parent form
args[2]+cgmh<device
args[3]+c0 a lpdout
args[4]+cname a device name
args[5]+cport a port name
args[6]+c0 a lpdmin
args[7]+c0 a 'win.ini'
args[8]+c0 a fwMode = read size of buffers

aRead size of buffer required by printer driver
sdm+edm args

allocate space for buffer
lpdmout+GlobalLock hand1+GlobalAlloc sdm

aset up output buffer and request to read data into it
args[3]+clpdout
args[8]+c2 a fwmode = DM_OUT_BUFFER #2
rc+edm args

ashow contents of buffers
'Current printer driver settings (Global)'showdevmode lpdout
GlobalUnlock hand1
GlobalFree hand1
+0
err:etxt

```

```

(R)←SetGlobalOrientation lorp;prname;putd;puti;alloc;free;lock;unlock;rdc;
hpr;gmh;gps;str;name;device;port;s;edm;args;fm;hand1;hand2;lpdmin;lpdmout;
sdm;rc;old;bin;geti
aset printer orientation (GLOBAL)
alorp←1=Portrait 2=Landscape
areturns old orientation

```

aFix external functions (calls to DLLs)

```

'gmh'□NA'I4 kernel.exe.P16|GetModuleHandle <0T'
'gps'□NA'I kernel.P16|GetProfileString <0T <0T <0T >0T I'
'rdc'□NA'I2 gdi.dll.P16|ResetDC I2 I4'
'alloc'□NA'U2 kernel.exe.P16|GlobalAlloc U2 U4'
'lock'□NA'I4 kernel.exe.P16|GlobalLock U2'
'free'□NA'U2 kernel.exe.P16|GlobalFree U2'
'unlock'□NA'U2 kernel.exe.P16|GlobalUnlock U2'
'puti'□NA'kernel.exe.P16|hmemcpy I4 <I2[] I4'
'putd'□NA'kernel.exe.P16|hmemcpy I4 <I4 I4'
'geti'□NA'kernel.exe.P16|hmemcpy >I2[] I4 I4 '

```

aget name of (printer) device from WIN.INI

```

str←','>1;gps'windows' 'device' '(undefined)' 256 255
name device port←3+1+'(','<str>cstr
+(name*(undefined))/found
'No print driver currently defined as default'
+0

```

found:hpr+0 a We dont need a handle to the parent form for this call

aset up arg string for ExtDeviceMode

```

s←''
s,←' I2'A hwnd ↔2 byte integer
s,←' I4'A hdriver ↔4 byte integer
s,←' I4'A lpdmount ↔4 byte integer (long pointer to output buffer)
s,←' <0T'A lpszdevice ↔pointer to char string
s,←' <0T'A lpszport ↔pointer to char string
s,←' I4'A lpdmin ↔4 byte integer (long pointer to input buffer)
s,←' I4'A lpszprofile↔pointer to char string or NULL (win.ini)
s,←' I2'A fwmode ↔2 byte integer (word)

```

aFix external function

```

'edm'□NA'I2 ',device','drv.P16|ExtDeviceMode ',s

```

aBuild argument string for ExtDeviceMode

```

args←8p0
args[1]←hpr a Handle to parent
args[2]←gmh<device a Handle to device driver
args[3]←+0 a NULL or Pointer to lpdmount (output buffer)
args[4]←cname a Device name
args[5]←cport a Port name
args[6]←+0 a NULL or Pointer to lpdmin (input buffer)
args[7]←+0 a NULL or Pointer to initialisation filepointer
args[8]←+0 a Request code fwMode←+0 = read size of buffers
aNote args[7] is NULL if initialisation file is WIN.INI

```

allocate and lock space for DeviceMode structure

```

lpdmout+lock hand1←alloc(0)(edm args)

```

```

a change args and read current values into DeviceMode structure
args[3]←lpdmout
args[8]←2 a fwmode = DM_OUT_BUFFER =2
rc←edm args

a remember old orientation
R←geti 1(lpdmout+44)2

a make changes in DeviceMode structure
rc←putd(lpdmout+40)(1)(4) a set dmFields for orientation
rc←puti(lpdmout+44)(lorp)(2) a set orientation

aset request to change DEFAULT settings
args[8]←9 a fwmode=9 DM_OUT_DEFAULT+DM_IN_BUFFER (1+8)
args[6]←c lpdmout
rc←edm args
→(rc=1)/ok
'error, update failed'
→end
ok:

a clean up
rc←unlock hand1
rc←free hand1

pname SetLocalOrientation lorp;putd;puti;alloc;free;lock;unlock;rdc;hpr;
gmh;gps;str;name;device;port;s;edm;args;fm;hand1;hand2;lpdmin;lpdmout;
sdm;rc;old
aset printer orientation (local to printer <pname>)
alorp←1=Portrait 2=Landscape
1 □NQ pname 100 a Send any output on the current DC to the printer

a Fix external functions
'gmh'□NA'I4 kernel.exe.P16|GetModuleHandle <0T'
'gps'□NA'I kernel.P16|GetProfileString <0T <0T <0T >0T I'
'rdc'□NA'I2 gdi.dll.F_6|ResetDC I2 I4'
'alloc'□NA'U2 kernel.exe.P16|GlobalAlloc U2 U4'
'lock'□NA'I4 kernel.exe.P16|GlobalLock U2'
'free'□NA'U2 kernel.exe.P16|GlobalFree U2'
'unlock'□NA'U2 kernel.exe.P16|GlobalUnlock U2'
'puti'□NA'kernel.exe.P16|hmemcpy I4 <I2[] I4'
'putd'□NA'kernel.exe.P16|hmemcpy I4 <I4 I4'

a get name of (printer) device from WIN.INI
str←',',1+gps'windows' 'device' '(undefined)' 256 255
name device port+3+1+'(','=str)<str
→(name#'(undefined)')/ok
'No print driver currently defined as default'
→0

a Get the handle of the current printer object
ok:hpr←pname □WG'handle'

aset up arg string for ExtDeviceMode
s←''

```

```

s,+ ' I2'A hwnid      ↔2 byte integer
s,+ ' I4'A hdriver    ↔4 byte integer
s,+ ' I4'A lpdmount   ↔4 byte integer (long pointer to output buffer)
s,+ ' <0T'A lpszdevice ↔pointer to char string
s,+ ' <0T'A lpszport   ↔pointer to char string
s,+ ' I4'A lpdmin      ↔4 byte integer (long pointer to input buffer)
s,+ ' I4'A lpszprofile ↔pointer to char string or NULL (win.ini)
s,+ ' I2'A fwmode      ↔2 byte integer (word)

```

*a*fix external function

```
'edm'␣NA'I2 ',device,'.drv.P16\ExtDeviceMode ',s
```

*a*Build argument string for ExtDeviceMode

```

args+8p0
args[1]+hpr a      Handle to parent
args[2]+gmh=device a  Handle to device driver
args[3]+0 a        NULL or Pointer to lpdmount (output buffer)
args[4]+cname a     Device name
args[5]+cport a     Port name
args[6]+0 a        NULL or Pointer to lpdmin (input buffer)
args[7]+0 a        NULL or Pointer to initialisation filepointer
args[8]+0 a        Request code fwMode↔0 = read size of buffers

```

*a*Note arg[7] is NULL if initialisation file is WIN.INI

*a*allocate and lock space for DeviceMode structure (output buffer)

```
lpdmout+lock hand1+alloc(0)(edm args)
```

*a*change args and read current values into DeviceMode structure

```

args[3]+lpdmout a    Pointer to the output buffer
args[8]+2 a          Request code fwmode↔2 = Read current settings
rc+edm args

```

a make changes in DeviceMode structure

```

putd(lpdmout+40)(1)(4) a set dmFields for orientation
puti(lpdmout+44)(lorp)(2) a set orientation to Landscape

```

a Reset the printers device context from the modified DeviceMode

```
old+rdc hpr lpdmount
```

*a*clean up

```

rc+unlock hand1
rc+free hand1

```

handle+GlobalAlloc bytes;alloc

*a*return a handle to global allocated memory of <bytes>.

```

'alloc'␣NA'U2 kernel.exe.P16\GlobalAlloc U2 U4'
handle+alloc 0 bytes

```

```
prr+GlobalLock handle;lock
```

*a*lock global memory

```

'lock'␣NA'I4 kernel.exe.P16\GlobalLock U2'
prr+lock handle

```

... and similarly for *GlobalFree* and *GlobalUnlock*.

References:

Some articles about calling DLLs from Dyalog APL/W:

- [1] Pearson, Duncan: *A Standard Font Dialogue Box from Dyalog APL/W using □NA*. Vector 10.1 pp 117-124
- [2] Pearson, Duncan: *A Windows Task-Killer for APL*. Vector 10.2 pp 126-127
- [3] Smith, Adrian: *Minding your .INIs*. Vector 10.3 pp 96-99
- [4] Smith, Adrian: *Native Files without Tears*. Vector 11.1 pp 126-129
- [5] Smith, Adrian: *Making Noises in APL*. Vector 11.2 pp 118-122
- [6] Windows SDK on-line help

A Virtual Classroom



**Makoto Kikkawa of Shokei Women's
Junior College**

Ms Makoto Kikkawa of the English department of Shokei Women's Junior College brought to APL95 a proposal for a 'virtual classroom' on Internet, where English-speakers learning Japanese can co-operate with Japanese speakers learning English.

If you are interested, please contact:

Ms Makoto Kikkawa
Shokei Women's Junior College
4-10-1 Yurigaoka, Natori-shi
981-12 JAPAN

Tel: +81 22 383 0272
Fax: +81 22 383 0260

Email:
100227.1100@compuserve.com
or BXD01017@niftyserve.or.jp

Technical Correspondence

More Thoughts on Dragons

From: Gérard Langlet

May 1995

Many thanks for the magnificent paper about dragons (*Here Be Dragons*, Vector 11.4 pp 69-75).

Apart from my article in *Vector*, one appeared in APL-CAM J. and another one in *Les Nouvelles d'APL* No 12-13. In the last issue (No 14) there was a paper by F. Hilaire who studied the logistic equation (also with other values for k) and Lyapunov coefficients, thanks to the user-defined precision (from 5 to 60 decimal digits) which is available in *Mathematica*. A controversy — in France — has started with some people who are considered as "authorities" in Chaos Theory (Pierre Bergé, Ch. Vidal, J. Laskar and G. Rumèbe *inter alia*). Laskar is famous for having predicted — thanks to Chaos Theory and "powerful computers" — that planet Mercury could quit its orbit in... 100,000 million years! His calculations involve the trajectories of all major planets (Newton revisited thanks to Laplace & Leverrier, making 800 PAGES of equations — of course non-linear) to be numerically iterated about 1,000,000 times, a real overdose of floating-point arithmetic!

As well as modulo2, integer algebra is unsigned, so are some of my writings in *Les Nouvelles d'APL*, e.g. in No 14 p 144. The case of Archimedes' algorithm — the classical method for computing π — is treated in No 14 p 67. In BASIC with single precision it is well known that PI converges towards 0 after 10 iterations. Convergence towards 0, however, is true on PC-like machines; PI converges towards infinity in APL.68000 for the Macintosh... *wh* is a "while" function.

```

v PI←ARCHI;C;I;L;S;P1
[1] P1←OP1←C+I+L+S+1
[2] 'I,PI,P1-PI+3×S×C*.5×C+4-C×C+2-C*.5×S+S+S×I+I+1×L+PI<9' wh 'L≠0#PI' v

ARCHI
1 3 0.141592653589793
2 3.10582854123025 0.0357641123595438
3 3.13262861328124 0.00896404030855136
4 3.13935020304687 0.00224245054292105
5 3.14103195089061 0.00056070269918207
6 3.14145247228567 0.000140181304123743
7 3.14155760791293 3.50456768680729E-5
8 3.14158389215415 8.7614356458765E-6
9 3.14159046325761 2.19033218717968E-6
10 3.14159210612642 5.4746336886069E-7
11 3.14159251692166 1.36668133254858E-7
12 3.14159261997481 3.36149832413923E-8
13 3.1415926506573 2.93249247107497E-9

```

```

14 3.141592666666556 -1.30757626948252E-8
15 3.14159273069858 -7.71087855788721E-8
16 3.14159298683066 -3.33240662460116E-7
17 3.14159366984943 -1.01625963289109E-6
18 3.14159776795893 -5.11436913663843E-6
19 3.1416086962248 -1.6042635010205E-5
20 3.14167426502176 -8.1611431963946E-5
21 3.14202394216292 -0.000431288573123911
22 3.14307274017004 -0.00148008658024557
23 3.1486604294525 -0.00706777586270624
24 3.15980616494113 -0.0182135113513406
25 3.18198051533946 -0.0403878617496702
26 3.35410196624968 -0.212509312659891
27 4.24264068711928 -1.10104803352949
28 6 -2.85840734641021
29 12 -8.85840734641021

```

I strongly disagree with Nicholas Small: Chaos effects are, in all cases, biased by the computer. Small emits the same argument as Laskar about the iterations of successive matrix powers of a matrix with $\det=1$, for which, indeed, a very small inaccuracy is detected after many iterations. Indeed, for $k < 1$, iterations of the logistic map do converge to 0.

But every time a squared term, which does not converge towards 0 if one develops analytically the successive formulae, is present within a nonlinear application, we shall be in the case for which the first truncation of the last significant bit will be amplified (then will not remain constant) at the next iteration. Truncation errors will exceed the range of the variable after some iteration, because of the doubling-angle effect. One cannot escape the fact that the derivative of ax^2 is $2ax$. So, when x does converge towards 0, then $2ax$ remains low in magnitude, and everything is OK. If x oscillates between two or more non-0 values, or, statistically between 0 and 1, even with a very small magnitude for a , truncation errors will propagate exponentially.

Small should try with Mathematica, which has a nice property: when the user-defined precision exceeds 16 decimal places, all classical options of arithmetic are performed with automatic error-checking, so that, even if you have asked for 100 digits, the result will be printed with its significant digits only. So, when truncation errors, cumulated or propagated, have reached 100%, then the first significant digit disappears and the result clings to 0. Visually, one has just to plot a graph of x as a function of the iteration numbers, for various (mathematically but not-numerically) equivalent formulae, and for various values of user-defined precision. All curves will collapse to 0. Just wait...

As a conclusion, let us re-state that no paper should be accepted for publication, especially in physics, if error-windows are absent in the figures or numeric tables (of either measured or computed results).

Non-Syllogistic Mathematical Proof

An Answer to *Vector* 11.4 p 126

From: Gérard Langlet

May 1995

Given any sophisticated function f of any number of variables with the result $r=f(x,y,z,...)$, we can experiment with as many runs as we wish on any available computer.

First, we shall read a series of values for $x, y, z, \dots$ as well as for initial r (e.g. set to 0), and store them into the linear memory of the computer (note: we do not impose that such variables — arguments as well as result(s) — be numerical scalars; they can be heterogeneous arrays of arrays).

Second, we take a snapshot of V , the occupied sequence of the computer memory for such a problem. Of course, this snapshot ignores the input-conversion format routines that were used, and ignores the naming of the variables as well as their respective lengths. V is just a binary sequence of 0s and 1s with length N matching $p V$.

Third, we let the function operate.

Fourth, we take a second snapshot of W , the binary sequence of the modified memory at the same locations (it has the same length N).

Now, we shall repeat the procedure with random values of the argument variables (a Monte Carlo technique), at least N times, let us say P times. So, we get two rectangular arrays AV and AW , each with P rows and N columns in which the columns are the successive V s and W s respectively.

Is it always possible for $N=P$, to find a matrix M so that AW is $M \cdot AV$ (if the period stands for the modulo2 matrix product, i.e. $\neq \cdot$ in APL)? If the answer to such a question is YES, we have a proof that function f may be replaced by a matrix product, then a proof that f is a linear function even if it is thought of as **nonlinear**! (Even if N had to be replaced by N^2 or by any still larger but finite number, this would hold!)

It has to be so, because in modulo2 algebra, nonlinearities do vanish. In order to simplify, let f be a secret nonlinear fn of just one variable, let us say a numeric variable x coded in floating-point arithmetic, e.g. with 64 bits. The result $r=f(x)$ is supposed to be another numeric variable of the same type; so, storing r into another location than the one used for x would waste memory; once the value of x is used by f , it becomes useless, so we shall store the result at the same location; our application has become $x=f(x)$.

The initial value for x can be chosen among 2^{64} values (supposing our arithmetic has no "nan" (not-a-number)).

What we say here for numbers coded in 64 bits also holds for numbers coded in n bits, with n small, e.g. 2. So, for pairs of bits, initial x has to be one of the 4 couples (00) (01) (10) (11). The result r or final x has to be also a member of the same 4 couples.

Function f may transform (00) into (00) or (01) or (11) or (11), i.e. 4 possibilities.

Function f may transform (01) into (00) or (01) or (11) or (11), i.e. 4 possibilities again.

Etc...

Altogether, one could think we have a restricted choice between $4 \times 4 \times 4 \times 4$ functions f , i.e. 256 functions. In fact, this is much too much...

Function f has to reduce to an algorithm which:

If the first bit of a couple of bits is 0, either swap it to 1 or don't (2 possibilities). Then either swap the second bit or don't (2 more possibilities).

If the first bit of a couple of bits is 1, either swap it to 0 or don't (2 possibilities). Then either swap the second bit or don't (2 more possibilities).

Obviously, only 2^4 i.e. 16 possibilities exist for f , not 256.

Only 2^4 i.e. 16 possibilities exist for matrix M : $\begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$ in 0-origin.

Among the 16 possible matrices, only 6 are invertible; only 4 of these are self-inverse (modulo 2) which corresponds to a reversible function (among which the do-nothing f_n or identity matrix, the mirror-function i.e. ϕ in APL or the anti-identity matrix, and 2-genitons G_h and G_v , responsible for the cognitive and the helix transform respectively).

The same reasoning holds for any finite value of n .

Without repeating the geniton story, the proof that — at least numerically, i.e. with any type of computer which would use any type of representation based on bits, in order to code any type of data — all functions coalesce into linear functions, implies that no function may be chaotic, except if one allows strange rules to alter precision, such as the ones which are used e.g. to subtract values in floating-point arithmetic and transform PI either into 0 on the PC and 360-like mainframes or into ∞ on the Macintosh.

At Play with J

by Eugene McDonnell

This column explores some ways of changing among different ways of representing a permutation.

Representations of a permutation:

standard	reduced	atomic
0 1 2	0 0 0	0
0 2 1	0 1 0	1
1 0 2	1 0 0	2
1 2 0	1 1 0	3
2 0 1	2 0 0	4
2 1 0	2 1 0	5

The tables above give three different forms of length-3 permutations. It is useful to be able to go between the standard and the atomic forms, and this conversion is facilitated by the reduced form. We develop the following verbs:

```
ra    reduced from atomic
ar    atomic from reduced
sr    standard from reduced
rs    reduced from standard
```

With these we can convert from each of the forms to any other. A factorial digits number base is used to convert between the atomic and reduced forms. The verb `fdb` gives the factorial digits base for permutations of the order of its argument.

```
fdb=. >:@1.-
```

For example,

```
fdb 3
3 2 1
```

With this base we can convert an atomic to a reduced form and vice-versa:

```
(fdb 3)#: 4
2 0 0
(fdb 3)#. 2 0 0
4
```

So the two verbs `ra` and `ar` are easily defined:

```
ra=. ([: fdb ]) #: ]
ar=. ([: fdb #) #. ]
```

For example:

```
  3 ra 4
2 0 0
  ar 2 0 0
4
```

To convert from a reduced to a standard form is somewhat more difficult. The trick is to begin at the right and add 1 to each atom which is equal to or greater than the atom at the left. This ensures that all atoms are kept distinct, and that at each step we have a permutation. For example, suppose we take the length 9 reduced form of the atomic form 288918:

```
]r=. (fdb 9) #: 288918
7 1 2 1 3 1 0 0 0
```

and then work from the right to develop the standard form:

```
      0
      0 1
      0 1 2
      1 0 2 3
      3 1 0 2 4
      1 4 2 0 3 5
      2 1 5 3 0 4 6
      1 3 2 6 4 0 5 7
7 1 3 2 6 4 0 5 8
```

and the last result is the desired standard form. We can encapsulate this in a verb as follows:

```
g=. [ , ] + ] >: [
7 g 1 g 2 g 1 g 3 g 1 g 0 g 0 g 0
7 1 3 2 6 4 0 5 8
```

This suggests that we define a verb

```
f=. g/
```

and use it on r directly:

```
f r
7 1 3 2 6 4 0 5 8
```

But we can do better than this, employing the identity, for integer k and permutation p ,

$k, p + p >: k \leftrightarrow /: /: k, p$

and arrive finally at the desired verb:

```
sr=. /:@/:@, /
]s=. sr r
7 1 3 2 6 4 0 5 8
```

The last verb we need, to translate from standard to reduced form, is arrived at by noting that, if r is the reduced form of a standard form s , then $1\{r$ is obtained from $1\{s$ by taking a count of how many atoms to the right of $1\{s$ are less than $1\{s$. For example:

```
7 1 3 2 6 4 0 5 8
7 > x x x x x x x x = 7
1 >           x       = 1
3 > x         x       = 2
2 >           x       = 1
6 > x x x x         = 3
4 > x             = 1
0 >               = 0
5 >               = 0
8 >               = 0
```

This sounds like a scan, but it isn't exactly. J provides the `\` adverb to fulfil the function of APL's scan adverb, but calls this `_prefix_`, to emphasize that the derived verb is applied to longer and longer prefixes. It calls the `\.` adverb `suffix_` to emphasize that the derived verb is applied to shorter and shorter suffixes. Thus the `rs` verb we need can be defined as:

```
rs=. ([: +/ ]. < {.)\.
```

```
rs s
7 1 2 1 3 1 0 0 0
```

With these four verbs, `ar`, `ra`, `rs`, `sr`, it is possible to obtain any of the three forms from any other. We don't need a verb to go directly from standard to atomic or vice-versa. However, J provides this as a primitive verb, denoted by `A.` and called 'Atomic Index' for its monad and 'Atomic Permute' for its dyad. For example,

```
A. s
288918
288918 A. 1. 9
7 1 3 2 6 4 0 5 8
```

So with A. a primitive, the four verbs we laboured over above are more interesting for pedagogical than for practical reasons. Atomic permute doesn't care what its right argument is; it will permute any object of sufficient length:

```
288918 A. 'netrilacy'
certainly
```

A useful verb to generate a table of all permutations of a given length is easy to write:

```
apn=. 1.@! A. 1.
apn 3
0 1 2
0 2 1
1 0 2
1 2 0
2 0 1
2 1 0
```

You would need a computer with a rather large amount of main store to generate apn 12 — about 46e9 bytes (8 bytes per element, 12 elements per row, 479,001,600 rows). Of course, the computer would also have to be able to address that large a store, too. Judging from the current state of affairs, it may well be almost the year 2000 before we routinely have these capabilities on our desktops.

“Correct” Windows File Management

by Duncan Pearson

Introduction

I spent a few days earlier this year writing an odd function to handle file open and close requests in a Windows application that I was writing. The behaviour I had to implement is not quite as straightforward as it seems and the function that I used to implement it is a little weird. It should raise a ‘Disgusted of Cheltenham’ letter if nothing else.

One of the things that can really irk a user of an application is odd behaviour in the area of the file management interface. By odd I mean non-standard. If you are using ten applications under Windows, and nine of them behave in one way but the tenth behaves in another, then in the tenth you will make mistakes. The area where these mistakes can be most costly is that of file handling. For instance, when you close most applications you are asked if you wish to save any work that you have done since last saving. When I am using such an application I tend not to save my work explicitly before leaving, as I know that the application will prompt me to do so if I might need to. Inevitably when I am using an application without this helpful behaviour I still just double-click the system icon and lose all my unsaved work — bummer.

In order not to saddle my users with the same problem I spent some time working out exactly what my application should do in this situation. As a model I took Windows Write. It is supplied with Windows and might be considered to a certain extent definitive.

The interface to the file management process consists of five menu items under the main ‘File’ menu. They are ‘New’, ‘Open’, ‘Save’, ‘Save As’ and ‘Exit’. ‘Exit’ is considered part of the file management process for the reason given above.

New

The current data is cleared and the name is changed to ‘(untitled)’ or ‘Documentn’ or some such. However if the current data is unsaved the user is prompted to save it. If he chooses to save then the ‘Save’ process (see below) is performed before clearing the existing data.

Open

If the user has unsaved data he is prompted to save it as above. He then selects a file from disk and the contents are loaded into the system. The application file name is set to the name of the loaded file.

Save

The current data is saved under the current file name. If the name is '(untitled)', or whatever the system uses to represent a new file, then the user is asked to choose a name as in 'Save As' below.

Save As

The user is prompted for a new file name. If the current file name was not generated by the system as a result of the 'New' option then it is given as the default. If the name chosen is that of an existing file then the user is warned that this is the case and is asked whether he wishes to overwrite this file or not. If he answers in the affirmative then the process continues as 'Save' above. If he answers 'No' then he is given the opportunity to select another file name.

Exit

If the current data is unsaved then the user is prompted as in 'New' and 'Open' above. The application then terminates although strictly this is not part of the file management process.

In each of the above processes, if the user cancels when asked a question or asked to select a file (for 'Open' or 'Save As') either by pressing the Cancel button or by pressing Escape, then the whole process is terminated.

These five options share a great deal of behaviour and in the case of 'Save' and 'Save As' each may be a subroutine of the other depending on circumstance. I found it easier to think about the problem by dividing the processes into the individual actions they perform and the questions that they ask. These are, in no particular order:

Questions

- Has the current data changed since the last save, is the current file 'dirty'?
- Does the user wish to save the changed data?
- Is the current data associated with a file name or is it '(untitled)'?
- Does the name chosen in the 'Save As' process represent an existing file?
- Does the user wish to overwrite the existing file?

Actions

- Clear the data
- Open a file
- Get a file name for 'Open'
- Get a file name for 'Save'
- Save the data to a named file

Each of these actions or questions can occur in two ways. Either it is specified initially by the user selecting a menu option, or it is made necessary by the answer to a question or result of an action. For example "Get a file name for 'Save'" can occur as a result of the user selecting "Save As" or because the "Is current data named?" returned "No".

The extra actions and questions that result from a particular condition do not depend on what actions need to be performed subsequently, so it is possible to think of the process as a form of an execution stack and this was how I implemented it. I decided that the behaviour was sufficiently horrible that I wanted to restrict it to one function. The function has a number of blocks corresponding to the actions and questions above. Each block consists of a label naming the block, some block specific code and some code to manage the transfer between blocks.

The way that the application developer specifies the set of blocks to execute in a call of the function, is in the argument to the function. This is a set of block names which when executed resolves to the **line numbers** of the **block labels**. They are stored in a variable *path*, to the end of which we append the line number of a special 'Ok' block, and we branch to *path* which takes us to the first block specified. At the beginning of each block the first element of *path* is removed (as it is no longer needed) and at the end of a block the new value of *path* is branched to. If, as a result of executing the block-specific code, it becomes necessary to execute more blocks, then the labels are simply added to the head of *path*. In this way when the extra blocks have been dealt with execution will continue as if nothing had happened. In the case of the user cancelling the operation then a special 'Cancel' block is executed which changes the result to 0 and adds 0 to the head of *path*, causing the function to terminate at the end of the block. If all specified blocks are executed without the user cancelling the operation then the 'Ok' block is executed which sets the result to 1 and adds 0 to the head of *path* in the same way as the 'Cancel' block.

As a consequence of each question, and depending on the success or failure of each action, the execution path is changed to include all necessary new behaviour. In this way the argument to the management function need only contain the minimum set of actions that the function should perform and the normal

Windows file management behaviour can be obtained by specifying the following simple values of *path*.

New	' <i>Dirty,NewFile</i> '
Open	' <i>Dirty,OpenName,OpenFile</i> '
Save	' <i>Named,SaveFile</i> '
Save As	' <i>SaveName,Exists,SaveFile</i> '
Exit	' <i>Dirty</i> '

However, if the user had selected the option to open a file when the current data was unsaved and the file untitled, and furthermore he had chosen to save his data under the name of an existing file and declined to overwrite it before selecting a different name, then the execution path would be:

```
Dirty,AskSave,Named,SaveName,Exists,
AskReplace,SaveName,Exists,SaveFile,
OpenName,OpenFile,Ok
```

Blocks

Dirty

If the file is dirty execute the block *AskSave*

AskSave

Use a "Yes, No, Cancel" dialogue box to ask whether the user wishes to save the current changes. If the answer is yes then execute the blocks *Named* and *SaveFile*. If the user cancelled the dialogue box execute the *Cancel* block.

Named

If the file name represents a new file then execute the blocks *SaveName* and *Exists*.

SaveName

Use the standard file dialogue box to get a file name under which to save the file. If the user cancels the dialogue box then execute the *Cancel* block.

Exists

If the file name corresponds to an existing file execute the *AskReplace* block.

AskReplace

Use the "Yes, No, Cancel" dialogue box to ask whether to overwrite the file or not. If the user specifies that the file should not be overwritten then execute the blocks *SaveName* and *Exists* to allow the user to choose another name. If the user cancels the dialogue then execute the *Cancel* block. Otherwise, continue execution at the next block in the stack (which will be *SaveFile*).

SaveFile

Save the current data to a file using the current name or the name specified by the *SaveName* block if it has been executed. If an error occurs in the operation then inform the user and execute the *Cancel* block.

OpenName

If the function was called with a left argument then use that as the new name. Else use the standard file dialogue to get the name of a file to open. If the dialogue is cancelled then execute the *Cancel* block.

OpenFile

Get the data from the named file into the application. If an error occurs in the operation then inform the user and execute the *Cancel* block.

NewFile

Clear the current data from the application then continue executing the next block in line.

Cancel

Set the result of the function to 0 then leave the function.

Ok

Set the result of the function to 1 then leave the function.

Listing

```

    v r+newname File path;name;res
[1]   name+Δfile
[2]   →path+(Δpath),0k
[3]
[4]   Dirty:path+1→path
[5]   path+(Is_File_Dirty/AskSave),path
[6]   →path
[7]
[8]   AskSave:path+1→path
[9]   res+Win_YNC name'This data has changed.' '' 'Save current changes?'
[10]  path+((!io+res)>0(Named SaveFile)Cancel),path
[11]  →path
[12]
[13]  Named:path+1→path
[14]  path+((~Is_File_Named name)/SaveName Exists),path
[15]  →path
[16]
[17]  SaveName:path+1→path
[18]  res name+Get_Save_File_Name
[19]  path+res+Cancel,path
[20]  →path
[21]
[22]  Exists:path+1→path
[23]  path+((Does_File_Exist name)/AskReplace),path
[24]  →path
[25]
[26]  AskReplace:path+1→path
[27]  res+Win_YNC name'File already exists.' '' 'Replace existing file?'
[28]  path+(res>(SaveName Exists)0 Cancel),path
[29]  →path
[30]
[31]  SaveFile:path+1→path
[32]  :if Save_Data_To_File name
[33]      Δfile+name
[34]  :else
[35]      path+Cancel,path
[36]  :endif
[37]  →path
[38]
[39]  OpenName:path+1→path
[40]  :if 2=ΔNC'newname'
[41]      res name+1 newname
[42]  :else
[43]      res name+Get_Open_File_Name
[44]  :endif
[45]  path+res+Cancel,path
[46]  →path
[47]
[48]  OpenFile:path+1→path
[49]  :if Get_Data_From_File name
[50]      Δfile+name
[51]  :else
[52]      path+Cancel,path

```

```

[53] :endif
[54] →path
[55]
[56] NewFile:path+1+path
[57] Clear_Data
[58] →path
[59]
[60] Cancel:path+1+path
[61] r←0
[62] path←0,path
[63] →path
[64]
[65] Ok:path+1+path
[66] r←1
[67] path←0,path
[68] →path

```

v

Arguments and Effects

path is a text vector indicating the tasks to perform. It is a set of label names separated by spaces or commas such that when executed within this function it will resolve to a vector of line numbers. During the execution of the function, and depending on circumstances, various line numbers may be catenated to the beginning of this list. It is in effect an execution stack.

Δfile is global and is the name of the current file. It may be changed by the function.

The result *r* is 0 if the user cancelled at any stage, otherwise 1.

If the new name is known (if for example it came from a list of most recently used files) then it is passed as the left argument *newname*.

Sub-Functions

The following functions are used in the function *File* with the described effects.

<i>Is_File_Dirty</i>	returns 1 if the application data has changed since the last save, else 0.
<i>Win_YNC</i>	shows the standard Yes, No, Cancel message box and returns 1 0 or 2 respectively.
<i>Is_File_Named</i>	returns 1 if its argument represents a file name, else 0.

<i>Get_Save_File_Name</i>	using the standard file dialogue box asks the user for a file name and returns the chosen name. The first element of the result is 0 if the user cancelled the transaction, else 1.
<i>Does_File_Name_Exist</i>	returns 1 if its argument refers to an existing file, else 0.
<i>Save_Data_To_File</i>	saves the application data to the file named by its argument and returns 1 for success or 0 for failure. It handles the error dialogue on failure.
<i>Get_Open_File_Name</i>	using the standard file dialogue box asks the user for a file name and returns the chosen name. The first element of the result is 0 if the user cancelled the transaction, else 1.
<i>Get_Data_From_File</i>	loads the file named by its argument into the application and returns 1 for success or 0 for failure. It handles the error dialogue on failure.
<i>Clear_Data</i>	clears the current data from the application and sets <i>Δfile</i> to represent a new file.

Many of these (such as *Does_File_Name_Exist*) are quite generic; the majority are completely application-dependent, and there is the occasional borderline case such as the code in the *AskSave* block where it might be necessary to have more specific wording in the message. The next challenge is to make a Causeway object out of this interesting collection of code so that it can safely and easily be used from any application that must guarantee to provide the accepted file management interface.

Emulating APL*PLUS/PC Native Files

by Adrian Smith

Background

I never took any great delight from coding for the old `□NREAD` functions in APL*PLUS/PC, but at least they were fast and totally reliable. I can also appreciate that there is a lot of application code out there which would really appreciate a 'no-brainer' conversion to Dyalog/W. This particular collection of functions sets out to emulate the old native file standard quite closely (even failing on `DOMAIN ERRORS` at the right places) so please let me know where you find inconsistencies.

How it all works

The basic technique is to 'overload' the various `□NA` calls so that the same code can be used to read characters or numbers (integer / float) without the overhead of data-conversion in APL. The speed on integers is quite frightening - this will read 4000 integers from a random offset into a 4Mbyte data file in less time than you can measure.

Just for the fun of it, I have also included a cover for `□FREAD` to pull in data from an APL*PLUS/PC component file. If anyone is up to the challenge of completing the set of these (and extending the set to cover PLUS II files) please feel free.

The APL Code

The first stage is to declare all those external calls - this one is called automatically by all the other top-level functions. Note that it keeps a global table of file-tie numbers and that positive tie-numbers are tolerated.

```

NInit
⍝ Set up □NA functions for native file I/O
□NA'I kernel.P16|_lcreat <0T I'
□NA'I kernel.P16|_lopen <0T I'
□NA'U4 kernel.P16|_llseek I U4 I'

⍝ Read and write characters as 1-byte unsigned ints
'_lread82'□NA'U4 kernel.P16|_lread I >U1[] U2'
'_lwrite82'□NA'U4 kernel.P16|_lwrite I <U1[] U2'

⍝ Repeat the above for two-byte integers

```

```

'_lread163'[]NA'U4 kernel.P16|_lread I >I2[] U2'
'_lwrite163'[]NA'U4 kernel.P16|_lwrite I <I2[] U2'

a Repeat the above for float
'_lread645'[]NA'U4 kernel.P16|_lread I >F8[] U2'
'_lwrite645'[]NA'U4 kernel.P16|_lwrite I <F8[] U2'

[]NA'I kernel.P16|_lclose I'

a Clear log for (Tie Handle Name) of each tied file
_nties+0 3p0 0 ''

-----

fi NCreate tn;fh
a Create new file (no result)
→(3=[]NC'_lcreat')+[]LC+1 o NInit
→(tn=_nties[];[]IO)+TieError
fh+_lcreat fi 0 o →(1=fh)+TiedError
_nties;+tn fh fi o →0
a ===== Error Handlers =====
TieError:'FILE TIE ERROR'[]SIGNAL 18 o →0
TiedError:'FILE TIED'[]SIGNAL 24

-----

r+Ascii;[]IO
a Make ASCII translate table
r+256p's' o []IO+1
r[32+132]+! '%$%&'()*+,-./0123456789:;<=>?'
r[64+132]+' @ABCDEFGHIJKLMNPOQRSTUVWXYZ[\]^_ '
r[96+132]+' `abcdefghijklmnopqrstuvwxyz{|}~ '
r[9 11 14]+[]TC

-----

bytes Nappend tn;fp;fh;ct;len
a Write to file starting at the end.
fp+_nties[];[]IO;tn o →(fp>p_nties)+TieError
fh+_nties[fp;1+[]IO]
a Jump to the end ...
ct+_llseek fh 0 2
a Check data type and use correct function ...
→(2≤|bytes)+DmnError
→(0≤1+0pbytes)+Char
→((32768^.>|bytes)^(bytes^.=|bytes))+PlusInt
+Float

Char:bytes+(-[]IO)+255|Ascii:bytes
len+pbytes o ct+_lwrite82 fh bytes len o →0
PlusInt:len+2+pbytes o ct+_lwrite163 fh bytes len o →0
Float:len+8+pbytes o ct+_lwrite645 fh bytes len o →0

a ===== Error Handlers =====

```

```
TieError:'FILE TIE ERROR'[]SIGNAL 18 0 +0
DmnError:[]SIGNAL 11 0 +0
```

```
size+NSize tn;fp;fh;sink
a File size of native file
fp+_nties[;[]IO];tn 0 +(fp>>p_nties)+Error
fh+_nties[fp;1+[]IO]
a Remember where we were, seek to the end, restore!
fp+_llseek fh 0 1
size+_llseek fh 0 2 0 sink+_llseek fh fp 0
+0
Error:'FILE TIE ERROR'[]SIGNAL 18
```

```
len NResize tn;fp;fh;fn;ct;bytes;size
a Resize file starting at given byte
a NB not more than 64K please!

fp+_nties[;[]IO];tn 0 +(fp>>p_nties)+TieError
fh fn+_nties[fp;1 2+[]IO]
size+_llseek fh 0 2 0 ct+_llseek fh 0 0
bytes+0 0 +size+Null
ct bytes+_lread82 fh size size
Null:bytes+len+bytes
a Now untie it and rewrite from a clean file ...
fh+_lclose fh
fh+_lcreat fn 0 0 +len+Zero
ct+_lwrite82 fh bytes len
Zero:_nties[fp;1+[]IO]+fh
+0
a ===== Error Handlers =====
TieError:'FILE TIE ERROR'[]SIGNAL 18 0 +0
```

```
NUntie tn;r;fh
a Untie native file(s)
fh+_nties[;[]IO];tn)+_nties[;1+[]IO]
+(0<pfh)+0
r+_lclose fh
a Remove from file ties
_nties+~+tne+_nties[;[]IO]
```

```
fi NTie tn;fh
a Tie another file (no result)
+(3=[]NC'_lcreat')+[]LC+1 0 NInit
+(tne+_nties[;[]IO])+TieError
fh+_lopen fi 2 0 +(~1=fh)+TieError
```

```

_nties;+tn fh fi 0 +0
^ ===== Error Handlers =====
TieError:'FILE TIE ERROR'[]SIGNAL 18 0 +0

```

```

fn NErse tn;fp;fh;OpenFile
^ Erase native file (must be tied).
fp+_nties[;[]IO];tn 0 +(fp>p_nties)+Error
fh+_nties[fp;1+[]IO]
fp+_lclose fh
^ Remove from file ties and Zap it ...
_nties;+tn+_nties[;[]IO]
[]NA'I kernel.P16|OpenFile <OT={U1 U1 U U1[4] T[128]} U'
fh>OpenFile fn(136 0 0(4p0)(128+' '))512
+0
Error:'FILE TIE ERROR'[]SIGNAL 18

```

```

bytes+NRead arg;tn;tr;get;ofs;fh;ct;size
^ Get bytes (in appropriate translation)
^ Number of bytes to read is mandatory!
bytes+' ' 0 +(3>parg)+LenError
tn tr get ofs+4+arg 0 +get+0
fh+_nties[;[]IO];tn 0 +(fh>p_nties)+TieError
fh+_nties[fh;1+[]IO] 0 size+NSize tn
^ Check if we fell off the end
+(size<ofs+get*2*tr+. =163 645 645 645)+InxError
^ Jump to offset if given (mode-independent!!)
+(3>parg)+Read
ct+_llseek fh ofs 0
Read:+( (82 163 645=tr),1)/ReadByte,ReadInt,ReadFloat,DmnError

ReadByte:ct bytes+_lread82 fh get get
bytes+Ascii[bytes+[]IO] 0 +0
ReadInt:ct bytes+_lread163 fh get(2*get) 0 +0
ReadFloat:ct bytes+_lread645 fh get(8*get) 0 +0

^ ===== Error Handlers =====
TieError:'FILE TIE ERROR'[]SIGNAL 18 0 +0
InxError:'FILE INDEX ERROR'[]SIGNAL 20 0 +0
LenError:[]SIGNAL 5 0 +0
DmnError:[]SIGNAL 11 0 +0

```

```

bytes NReplace arg;size;tn;fp;fh;ct;len;ofs
^ Write to file starting at given byte
+(2*parg)+LenError 0 tn ofs+arg
fp+_nties[;[]IO];tn 0 +(fp>p_nties)+TieError
fh+_nties[fp;1+[]IO]

size+NSize tn 0 +(ofs>size)+InxError

```

```

a Jump to the right spot ...
  ct+_llseek fh ofs 0

a Check data type and use correct function ...
  +(2<|=bytes)+DmnError
  +(0<1+0pbytes)+Char
  +((32768^>|bytes)^(bytes^>|bytes))+PlusInt
  +Float

Char:bytes+(-_IO)+255|Ascii\bytes
  len+pbytes o ct+_lwrite82 fh bytes len o +0
PlusInt:len+2*pbytes o ct+_lwrite163 fh bytes len o +0
Float:len+8*pbytes o ct+_lwrite645 fh bytes len o +0

a ===== Error Handlers =====
TieError:'FILE TIE ERROR'\_SIGNAL 18 o +0
InxError:'FILE INDEX ERROR'\_SIGNAL 20 o +0
LenError:_SIGNAL 5 o +0
DmnError:_SIGNAL 11 o +0

```

```

r+NNames
a File names currently natively tied
r+_nties[;2+_IO]

```

```

r+NNums
a Files currently natively tied
r+_nties[;_IO]

```

The component file emulation (such as it is) simply uses the above functions to do the donkey-work! Note that to maintain separation between native and component files, it would probably be a good idea to stay with the PLUS/PC standard of negative tie-numbers for native files.

```

fi FTie tn;_TRAP
a Tie component file to given <tn>
  _TRAP+(18 24)'E' '\_SIGNAL _EN'
  fi,+(-'.efi)/'.ASF'
  fi NTie tn

size+FSIZE tn;fp
a File size of component file (always starts from 1)
  fp+_nties[;_IO]\tn o +(fp>p_nties)+Error
a Allows 128 components! There must be a better way!!
  size+1,1+4+~1:~0 0 0 0<9+NRead tn 163 512 0
  size,+(NSize tn),0

```

```

→0
Error:'FILE TIE ERROR'[] SIGNAL 18

FUntie tn
a Untie component file(s)
NUntie tn

-----

z←FRead arg;tn;cn;fh;sa;sh;t;tr;[]IO;0;[]TRAP
a Read PC component file ties as <tn>
[]IO+0 ◊ →(2*parg)+LenError ◊ tn cn+arg
fh←_nties[;0];tn ◊ →(fh>p_nties)+TieError
fh←_nties[fh;1]

0←_llseek fh,(8*cn+1),0
sa+φ1←_lread82 fh 4 4 ◊ →(sa^.=0)+InxError

sa+256i(128|1+sa),1+sa
0←_llseek fh,(sa+9),0 ◊ tr+1←_lread82 fh 2 2 ◊ t+tr[0]>8
tr[0]←(82 163 645)[1 2 8;16|tr[0]]

a Long-stop trap for index errors
[]TRAP+20 'E' '[] SIGNAL []EN'

sh←NRead tn,(163 323)[t],tr[1],sa+11
z+shpNRead tn,tr[0],(×/sh),sa+11+tr[1]×(2 4)[t] ◊ →0

a ===== Error Handlers =====
TieError:'FILE TIE ERROR'[] SIGNAL 18 ◊ →0
InxError:'FILE INDEX ERROR'[] SIGNAL 20 ◊ →0
LenError:[] SIGNAL 5 ◊ →0

```

Limitations

It is not obvious to me what happens to a component file with more than 128 components! The PLUS/PC strategy is to allocate space on the index for 64 components when the file is created. When you write the 65th, it clears space for another 64 (re-locating anything that was in the way to the end) and so on. The above *FRead* function will cope quite nicely, but I cannot see an easy way for *FSize* to know how far to look into the file for the 'one beyond the last' marker of 4 consecutive zeros! To save searching the entire file redundantly, I just give up at 128.

PS

Apologies to non-Dyalog users for the gratuitous use of 'slash-commute-gets' in several of the functions above. I got a little carried away, but I am better now.

Index to Advertisers

Causeway Graphical Systems Ltd	4
Dyadic Systems Ltd	2,4
MicroAPL	16
APL Booklist (Renaissance Data Systems)	32
Vector Back Numbers	8

All queries regarding advertising in VECTOR should be made to Gill Smith, at 01439-788385, Compuserve: 100331,644.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Anthony Camacho,
11 Auburn Road, Redland,
BRISTOL, BS6 6LS
Tel: 0117-973 0036
Email: acamacho@cix.compulink.co.uk

Authors wishing to use Windows Write or Word for Windows should contact Vector Production for a copy of the Vector APL TrueType font and Vector APL typebox.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK YO6 4JJ.

Tel: 01439-788385
Compuserve: 100331,644.

British APL Association: Membership Form

Membership is open to anyone interested in APL. The membership year normally runs from 1st May to 30th April, but new members may join from 1st August, November or February if preferred. The British APL Association is a special interest group of the British Computer Society, Reg. Charity No. 292,786

Name: _____

Address: _____

Postcode / Country: _____

Telephone Number: _____

Email Address: _____

Category (please tick box) to run from: 1st May ☐ August ☐ Nov ☐ Feb ☐

UK private membership £12 ☐

Overseas private membership £14 ☐

Airmail supplement (not needed for Europe) £4 ☐

UK Corporate membership £100 ☐

Corporate membership overseas £135 ☐

Sustaining membership £430 ☐

Non-voting UK member (student/OAP/unemployed only) £6 ☐

PAYMENT — in Sterling or by Visa/Mastercard/JCB only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard, Visa or JCB number.

I authorise you to debit my Visa/Mastercard/JCB account

Number: Expiry date: |

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON E3 5HU, UK

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1995/96 Committee

Chairman:	Dr Alan Mayer 01792-205678x4274 a.d.mayer@swansea.ac.uk	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Secretary:	Sylvia Camacho 0117-973 0036 100612.1057@compuserve.com	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Treasurer:	Nicholas Small 0181-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Anthony Camacho 0117-973 0036 acamacho@cix.compulink.co.uk	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Activities:	Vacant Post	
Education:	Dr Ian Clark 01388-527190 100021.3073@compuserve.com	9 Hill End, Frosterley Bishop Auckland Co. Durham DL13 2SX
Technical:	Vacant Post	
Publicity:	David Eastwood 0171-922 8866 MicroAPL@microapl.demon.co.uk	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Recruitment:	Jon Sandles 01904-411635 100257.1756@compuserve.com	22a Arthur Street, Lawrence Street, York YO1 3EL
Administration:	Rowena Small 0181-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Anthony Camacho	0117-973 0036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (01439-788385)
Support Team:	Jonathan Barman (01488-648575), Duncan Pearson (01653-618900), Richard and Adam Weber (01302-539761), Sylvia Camacho, Ray Cannon (01252-874697), John Searle (0181-858 6811), David Ziemann (0171-267 8032), Jon Sandles	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

English MFL Association and is distributed to Association members in the form of a programme — an interactive computer programme. It is supported on

SUSTAINING MEMBERS

English MFL Association wish to acknowledge the generous financial

Compass R&D Ltd
10 Frederick Sanger Road
Surrey Research Park
Guildford, Surrey GU2 6YD
Tel: 01483 902249
Fax: 01483 302279

HMW Trading Systems Ltd

1 Temple Avenue
LONDON EC4Y 0HA
Tel: 0171 353 8800
Fax: 0171 353 8325

Email: 00020.2692@compuserve.com

Manjistics

215 East Wilson St
Rockville

MARYLAND 20852 USA
Tel: +1 (301) 984-5412
Fax: +1 (301) 984-5094

Email: intl@manj.com (international)

Soliton Associates Ltd

6000 Blankenburg St
Northridge

10824 Vanowen St
Tel: +1 (310) 496-4775
Fax: +1 (310) 496-4705

Email: info@soliton.com